尺度核查与独立评价下的啤酒感官预测及约束风格迁移

摘 要

针对啤酒感官评分、关键因素与风格生成问题，构建尺度审计、分范围预测和独立代理评价流程。以250款酒的231个输入和50个目标建模，明确30个Aroma/Flavour核心维度并保留全表。三折五次重复留出中，ET核心宏平均MAE为0.3496，训练内收缩融合为0.3466；五次独立角色分割的平均逐维连续覆盖为90.06%，但相对等级不等于原始0-6分。加入无图及打乱图对照后，真实图与无图误差几乎相同，全数据内层选择图惩罚为0，不支持额外预测收益。150条因素候选中39条满足当前描述条件，化学证据分级保留。生成阶段去除五个未核实综合量，以100款搜索、100款评价、50款真实留出隔离，比较四类生成方法；非复制优势未转化为全面分布距离优势。半径1下，真实留出源酒的10条重复迁移记录仅2条获独立风格及陈旧酒花代理共同支持。研究提供任务化建模与证据边界，不主张新的基础算法、绝对评分恢复或真实品评改善。

关键词：感官预测；相对分级；嵌套验证；网络消融；条件生成；独立评价

目 录

1 问题重述与研究目标 1

2 相关工作、贡献边界与模型假设 1

3 数据审计与处理 2

4 第一问：连续预测、相对分级与不确定性 2

5 第二问：网络消融、稳定因素与化学证据 6

6 第三问：受约束生成与独立迁移诊断 8

7 结果评价、贡献与限制 10

8 结论与建议 11

参考文献 11

AI工具使用与研究贡献声明 12

附录A 完整Top-3、复现与结果索引 12

附录B 本项目完整源程序 15

1 问题重述与研究目标

本题要求由化合物测量给出感官评分、逐维 Top-3 及稳定性和化学解释，并按风格生成化合物谱和完成缺陷不恶化的迁移。与附件高度吻合的原始研究已用化学资料预测感官并开展加标试验[1]；其数据和已有发现是研究背景，不计作本研究新增贡献。

本文以实际 250 款啤酒、231 个数值输入和 50 个监督目标为基础。保留全部目标输出，另把 A_ 与 F_ 前缀的 30 个核心维度作为预先明确的主要比较范围，其他 20 项按评分卡章节补充报告。这个范围选择不代表官方只要求 30 项，也不把题面“约 40”强行解释为一个与附件不符的固定数量。

三项研究目标分别为：明确连续标签与 0-6 等级的可解释范围；检验网络和候选因素的预测信息是否稳定；把统计可行的风格调整与独立模型支持、真实化学和品评证据区分。无法获得的新测量不以模型输出补造。

2 相关工作、贡献边界与模型假设

ExtraTrees[2]、梯度提升回归[3]及利用交叉验证预测学习组合权重的 stacking[4]均为成熟方法。图拉普拉斯约束回归已有系统研究[5]，分割保形预测也已有理论和方法[6]。本文采用这些方法并作任务化组合，不主张首次发明基础算法、收缩融合、Pareto 优化或新的保形理论。

本文的可核查贡献定位于应用建模和证据评估：逐字段尺度及评分位置核查；在明确目标范围内对标准模型及收缩融合公平比较；加入无图和打乱图对照，检验网络的额外价值；分离生成搜索资料与评价资料，并如实报告代理判断分歧和失败。贡献是否成立由具体实验决定，不要求所有复杂设计获胜。

假设一，beer 唯一标识相同酒，跨表正确对应，经集合、唯一性和风格一致性支持。假设二，训练内插补、截尾和标准化提供可检验的统计处理，但不证明极端值错误或缺失随机。假设三，相关网络只提供共变结构，系数不是因果效应。假设四，风格内统计插值与经验支持约束不充分保证酿造可行。假设五，覆盖解释依赖相应交换性与测量条件，不能自动推广到自适应搜索后的分布外样本。

符号	定义
X、Y	231 个输入与 50 个连续目标；生成主模型使用 226 个非聚合字段
Z、B、L	训练预处理后的输入、回归系数及有符号图拉普拉斯
w_j、η	目标 j 的 GBR 权重及向全局权重收缩的惩罚
g、[L_y,U_y]	固定训练分位等级映射与连续预测区间
G、J、T	生成/搜索训练组、独立评价训练组、真实留出组
r、D	标准化改动半径和四项指定化学缺陷字段集合

3 数据审计与处理

3.1 对齐、字段与评分卡核查

实际 1.csv 为 250×234，含 3 列元信息；4.csv 含 50 个目标。beer_id 在感官文件精度受损，不作为主键。程序按 beer 对齐并核对风格；原件以 SHA-256 绑定版本，未修改附件。2.csv 与重算 Spearman 的平均绝对差约 1.58×10^-16，而与 Pearson 约差 0.0873，因此按秩相关解释。

评分卡 summary 与监督目标的三个显式别名为 A_ester_fruity→A_esters_fruity、F_ester_fruity→F_esters_fruity、4vg→X4vg。50 个目标均已对应。评分卡另有 Aroma、Flavour 乙醛位置而没有对应的独立监督列，保留缺标签说明，未补零。12 处 Barnyard 汇总引用 AG76，表单标签却对应 AG90，列为疑似源公式问题；这不自动证明 4.csv 继承该问题。

资料或范围	数量/用途	处理决定
核心 Aroma / Flavour	15+15 个前缀目标	主比较，全部保留输出
其他感官目标	20 个	口感、缺陷、香料、Lambic 与总体等另报
输入字段	231 个	预测阶段保持附件尺度
生成主输入	226 个非聚合描述字段	五个未核实综合量不独立生成
3.xlsx 原评分卡	12 张 beer 页及 summary	输入空白和公式缓存零不作为评分真值

3.2 尺度、缺失和异常处理

逐列比对可比的原始正值配对，有 37 个字段支持 log10 关系；其他字段存在多数恒等、少数不匹配或缺参考。正值匹配不证明零替代值、检测限或全部量纲均已恢复，因此不把全表统一反对数。DMS、4VG 的正值变换证据与双乙酰、trans-2-nonenal 的不匹配另列。

所有预测预处理只在当前训练部分估计：1%/99% 截尾、中位数插补、均值和标准差标准化，使用 Pipeline 管理[7]。验证和测试只执行已拟合变换。去截尾、2.5% 截尾和模拟缺失作为消融，不能把被截值称为错误测量。

$$z_{ij} = \frac{\mathrm{clip}(x_{ij}; a_j, b_j) - m_j}{s_j} \tag{1}$$

4 第一问：连续预测、相对分级与不确定性

4.1 绝对评分的信息缺口与交付定义

原始研究按品评人员中心化和标准化。若逐人均值、标准差、出席情况及汇总权重缺失，同一聚合连续值可能对应多组原始评分，无法唯一反演真实 0-6 分。空白评分卡不能提供锚点。本文不把经验映射包装成真实品评等级。

$$z_{it} = \frac{s_{it} - \mu_t}{\sigma_t}, \qquad s_{it} = \mu_t + \sigma_t z_{it} \tag{2}$$

实际交付包括全 50 维连续留出预测和带 ASSUMPTION 标记的相对 0-6 等级。相对 0 是训练参考的最低一档，不等于真实未检出；相对 6 也不等于真实压倒性。若题目要求真实绝对等级，仍需原始锚点或转换参数；本文诚实保留这一未解决的测量信息条件。

4.2 模型与严格训练内融合

比较均值、岭回归、ET、GBR、固定半数 ET/GBR、全局权重、无收缩逐维权重和收缩逐维权重。ET 200 棵树、最小叶样本 3、特征比例 0.8；GBR 每目标 100 棵深度 2 的树，学习率 0.04、最小叶样本 5、特征比例 0.7。岭回归 α 在 {1,10,100,1000} 中以内层核心 MAE 选择；元信息和风格不作为第一问预测输入。

$$\widehat{B} = \arg\min_{B} \left\{ \|Y_c - ZB\|_F^2 + \alpha \|B\|_F^2 \right\} \tag{3}$$

融合权重的具体目标是内层留出预测的均方误差加向全局权重的二次惩罚，最终 η 由训练内验证的核心 MAE 选择。设 d=p_GBR-p_ET、e=y-p_ET，得到受限闭式权重：

$$w_j = \mathrm{clip}\left(\frac{\mathrm{mean}_i(d_{ij}e_{ij}) + \eta w_0}{\mathrm{mean}_i(d_{ij}^2) + \eta}, 0, 1 \right) \tag{4}$$

$\eta \in \{0, 0.01, 0.1, 1, 10\}$。对每个外层训练集，再分 S/V；只在 S 内三折产生基础预测并拟合权重，重训 S 后预测 V、选择 η。V 标签不进入 S 内拟合。选定 η 后在全部外层训练内重新产生 OOF、学习权重，重训基础模型后预测外层测试。该嵌套结构避免直接在外层测试上拼接最优维度。

4.3 同范围验证与实际结果

按风格三折、五次重复，共 15 个外层划分；内层使用三折 KFold。每个样本每次恰被留出一次，所有方法共享外层角色。报告先对每个 beer 的五次预测平均、再计算逐维宏指标的聚合 OOF 成绩；它不同于未来单次拟合模型的误差，单折成绩另存。重复折共享资料，不能当独立样本简单作 t 检验[8]。

$$\mathrm{MAE} = \frac{1}{nq} \sum_{i=1}^{n} \sum_{k=1}^{q} |y_{ik} - \hat{y}_{ik}| \tag{5}$$

方法	核心30维 MAE	全50维 MAE
均值	0.41164	0.41698
岭回归	0.36326	0.35339
ET	0.34964	0.33987
GBR	0.35046	0.33534
固定半数融合	0.34648	0.33359
训练内全局融合	0.34672	0.33433
无收缩逐维融合	0.34709	0.33402
收缩逐维融合	0.34664	0.33389

ET 的核心 MAE 为 0.34964，全维为 0.33987；收缩融合分别为 0.34664、0.33389。核心相对 ET 的变化为 -0.86%。固定半数融合核心 MAE 为 0.34648，略优于收缩方案；当前没有证明更复杂的权重学习必要。本文不把全维平均最优推导为核心维度最优。主交付的相对等级与区间采用事先固定的 ET 基线，复杂融合仅在其实际收益和代价范围内评价。

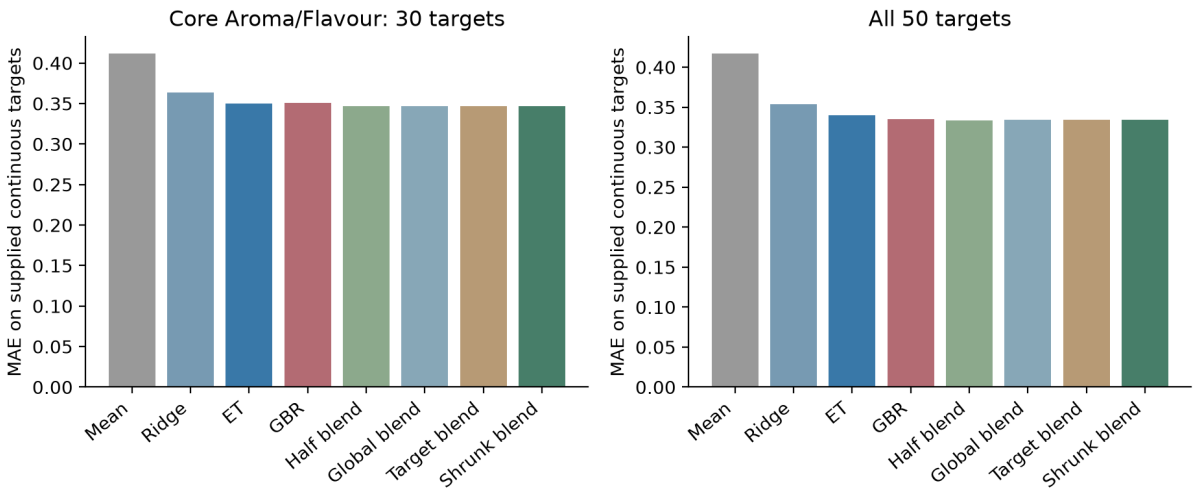


图1 相同外层划分的聚合留出误差：核心30维与全50维分开报告

配对比较按 beer 重采样，保持同一酒两模型的误差配对及风格构成。2000 次固定 OOF 条件重采样范围只反映保存预测的样本敏感性，不包含训练和选模不确定性。完整差值、范围、逐维 R^2 及失败目标随结果表提供，未把条件范围当作总体算法优势的严格证明。

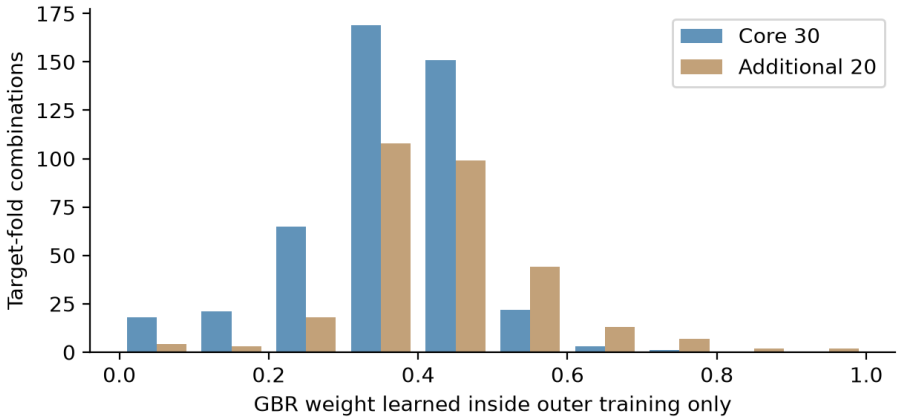


图2 权重完全在外层训练内部学习，反映目标异质性而非外层成绩事后挑选

4.4 固定相对分级与预测集合

每个训练参考的六个边界取 k/7 分位。固定边界定义 g(y)，对留出连续预测给出相对等级。全 250 款酒的交付表使用五次留出等级众数；各折参考稍有差别，因此该表是相对画像，不是共同绝对强度量表。

$$g(y)=\sum_{k=1}^{6}\mathbf{1}(y>t_k),\quad \mathcal{C}(x)=\{g(L_y(x)),\ldots,g(U_y(x))\} \tag{6}$$

另作五次 150 训练、50 校准、50 测试的分割保形示例。ET 和等级边界只在 150 个训练样本拟合；50 个校准残差的第 ceil[51×0.9]=46 顺序统计量给出逐维连续区间，再映射为相对等级集合包络。重复边界可能让包络包含不可达的中间等级，但不缩窄包络。

$$r_{ik}=|y_{ik}-f_k(x_i)|,\quad r_k^{*}=r_{(\lceil (m+1)(1-\alpha)\rceil),k} \tag{7}$$

$$C_k(x)=\left[f_k(x)-r_k^{*},\ f_k(x)+r_k^{*}\right] \tag{8}$$

五次分割、50 个目标的平均连续测试覆盖为 90.06%，平均区间宽 1.4842；固定规则下的相对等级集合覆盖为 98.56%，平均集合大小 6.49。这不是原始等级准确率或 50 维同时覆盖保证；

同一批酒重复出现也不构成五套外部数据。

平均集合接近全部七档，说明目前很难在可靠性要求下细分感官等级；较高集合覆盖伴随宽集合，不能解读为七级预测很精确。细化等级需要更好的测量锚点、更多样本或更小的连续预测误差。

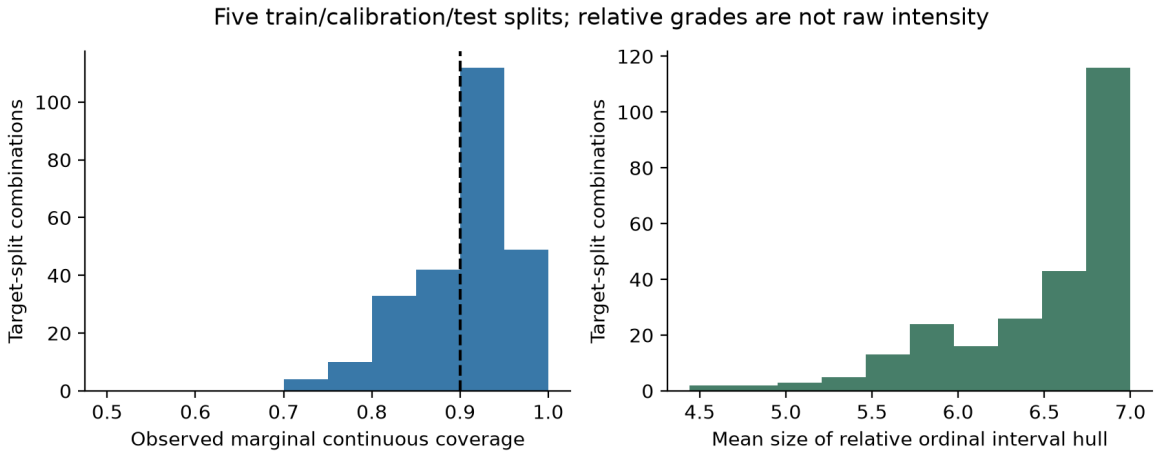


图3 五次分割的逐维覆盖与相对等级集合大小；真实绝对评分仍未标定

beer	麦芽香	酒花香	酯香	麦芽味	酒花味	酯味
10	3	1	1	2	1	1
86	3	1	0	2	1	1
2	4	1	0	3	1	0

上表只展示全量输出中的三个样本和六个整体属性，数值均为相对档位；完整 250×50 等级、连续预测和区间角色保存在支撑文件。真实评分锚点获取路径未执行，不虚报绝对等级准确率。

4.5 截尾、缺失压力与失败分析

模拟缺失情境	额外比例	ET核心 MAE
MCAR	0%	0.35205
MCAR	5%	0.35463
MCAR	10%	0.35739
MCAR	20%	0.36152
low_value_preferential	0%	0.35205
low_value_preferential	5%	0.35492
low_value_preferential	10%	0.35804
low_value_preferential	20%	0.36669

低值优先遮蔽使训练 20% 分位以下的观测被遮蔽权重为其他值的四倍；它是检测限相关缺失的压力情境，不是对真实缺失机制的确认。每种情境中 ET 与融合使用完全相同的遮蔽位置。截尾对照使用 ET，未把它冒充全部方法都完成的消融。

截尾比例 0、1%、2.5% 的 ET 单折核心 MAE 均值依次为 0.35232、0.35205、0.35250。额外缺失表为逐折误差均值，与前述重复预测平均后的误差不是同一种汇总，不能混作同一成绩提升。

5 第二问：网络消融、稳定因素与化学证据

5.1 描述图与训练折图

按给定秩相关矩阵建立 231 节点的描述图，$|r|\geq 0.7$ 时有 109 条边。预测验证中的网络必须由当前训练数据重算，不能直接使用含外层样本信息的全数据图。符号拉普拉斯及图约束回归定义如下；图用于正则化，不是反应通路。

$$W_{ij} = r_{ij}\,\mathbf{1}(|r_{ij}| \geq \tau), \quad L = \operatorname{diag}\left(\sum_j |W_{ij}|\right) - W \tag{9}$$

$$\widehat{B} = \arg\min_B \left\{ \|Y_c - ZB\|_F^2 + \alpha\|B\|_F^2 + \lambda \operatorname{tr}(B^\top LB) \right\} \tag{10}$$

$\alpha \in \{1, 10, 100, 1000\}$，$\lambda \in \{0, 1, 5, 20\}$，非零 λ 时 $\tau \in \{0.6, 0.7, 0.8\}$，全部在内层以核心 MAE 选择。无图基线使用相同 α 候选；负对照将训练图节点标签随机置换，保留无标签图的边权、符号和度分布。每个拟合使用 10 个随机图，汇总各图的误差而非平均预测形成额外集成。

5.2 网络的额外收益检验

模型	核心30维单折 MAE	全50维单折 MAE
无图	0.366058	0.356312
真实图	0.366061	0.356314
打乱图	0.367004	0.357329

真实图与无图误差几乎相同，全数据内层选择 α=1000、λ=0。因此当前数据和网格不支持“图正则化改善预测”的结论。网络仍可组织共变关系，但最终参数退化到普通岭回归时，Top-3 不能归功于图惩罚。该表是单折均值，不能与第一问聚合 OOF 数值直接比较来判断模型优劣。

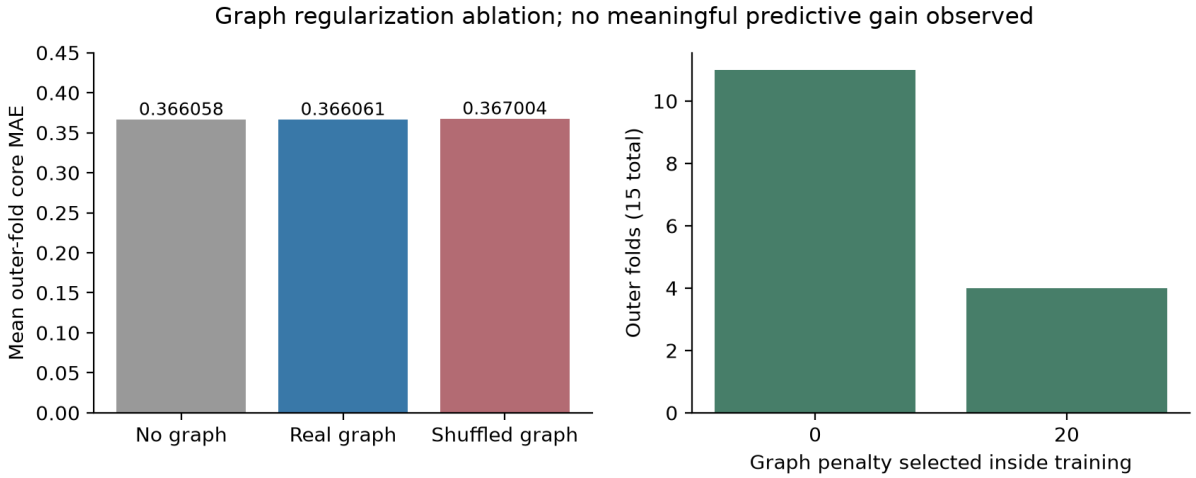


图4 有图、无图和打乱图的对照及内层选择：真实图未显示实用预测收益

5.3 跨拟合贡献、风格控制与模块替代

每个外层训练内选择每目标三个化合物/元素候选，排除 pH、色度及聚合字段。留出贡献用冻结模型的特征置换 MAE 增量评价，重复 20 次；另在相同风格内置换，以及联合置换整个相关模块、删去模块重训。风格内置换不是完整条件于全部化合物的严格条件置换，模块与单变量贡献也不是因果效应。

$$\pi_{jk} = \frac{1}{R}\sum_{b=1}^{R} \mathbf{1}\left(j \in \mathrm{Top3}_k^{(b)}\right) \tag{11}$$

$$\Delta_{jk} = \mathrm{MAE}_k\left(\hat{Y}^{\mathrm{perm}(j)}\right) - \mathrm{MAE}_k\left(\hat{Y}\right) \tag{12}$$

全数据描述模型的 200 次 bootstrap 每次重建预处理与图，但固定全数据选定的超参数，所以频率是该参数条件下的稳定性。当前描述性支持条件为频率≥0.5、至少 5 个外层训练入选、这些入选折的平均普通与风格内贡献均为正，共 39/150 条。它与前阶段 11 条采用的模型和判据不同，不能把数量变化解释为直接的可靠性提升；筛后汇总仍有选择适应性，不作独立显著性检验。

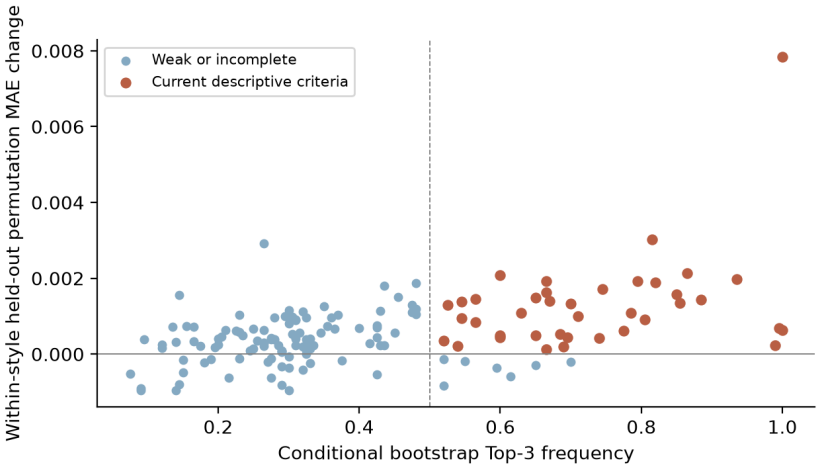


图5 150 条最终候选的条件重采样稳定性及风格内留出贡献：正贡献不代表因果

5.4 化学合理性、证据层级与反例

X4vg 与 X4VG 为较一致的候选。已有啤酒陈化研究观察 4VG 降低并讨论丁香向香草印象变化[9]，可支持气味合理性，但不是本数据因果验证。iron 与 metallic 的关系有铁离子诱发口腔脂质氧化的实验依据[10]，该研究使用金属溶液且不同价态反应不同，因此不能直接给本附件总 iron 套用统一啤酒阈值。

coriander 与 linalool 的关联可结合低醇小麦啤酒添加芫荽籽的研究核查；该研究的 GC-O 结果指出芫荽籽中 linalool 等组分有较高气味活性[11]。这只支持机制相容性，不能证明本附件 coriander 标签由 linalool 单独造成。酸度与乳酸、醋酸以及酒精感与乙醇可作测量相容性的检查，其余未知联系不补写机理。

关联示例	可支持的解释	仍缺什么
X4vg / X4VG	啤酒中丁香相关香气有原始研究支持	本批次加标或独立测量
metallic / iron	金属离子可诱发口腔金属味机制	价态、基质、浓度和本附件因果
coriander / linalool	芫荽籽与啤酒研究的香气依据	标签特异性与浓度验证
bitternes / iron	统计贡献可能反映风格或共变	不能据此称铁是苦味的直接来源
clove / linalool	可重复关联但不等于典型丁香来源	共线替代与特异性证据

完整 150 条结果附方向、频率、入选折数、风格内贡献、单位和附件气味描述。独立文献只支持少量明确案例，其余化学解释逐条标为未充分核实；不以统计支持状态冒充机制通过。论文附录逐维给出 Top-3，弱项和反例均保留。

6 第三问：受约束生成与独立迁移诊断

6.1 三组隔离与226字段模型

每个种子将资料按风格分层为 G=100、J=100、T=50。G 拟合生成器、Logistic 风格搜索器、岭感官代理和 ET 感官一致性代理；J 拟合不参与搜索的 ET 风格评价器及另一感官代理；T 只评价真实样本诊断能力。每次 G/J/T 不重叠，但所有酒在前两阶段研究中均已出现，因此只是本次拟合角色隔离，不是新增外部验证。

五个未核实综合量不独立生成，生成器、搜索器与评价器全部重训为相同 226 个非聚合字段。字段仍含理化描述量，不能当作 226 种可任意混合的纯物质；未核实综合量输出为待核实，而不是补造数值。候选及顺序先固定并计算哈希，再拟合 J 评价模型，J/T 结果不用于重新挑候选。

五次真实 T 诊断中，搜索器与独立评价器的平均平衡准确率分别为 0.416、0.470。因此独立评价者本身也并非完美风格鉴定。两类模型的概率未经共同校准，不能仅以概率数值高低比较其可靠性；另报目标类别预测比例和代理约束。

6.2 生成方法与相同核带宽比较

比较风格内 bootstrap、局部凸组合、十维条件 PCA 高斯及受约束局部凸组合，每种风格每种方法每划分生成 40 个样本。PCA 协方差按 n_c/(n_c+10) 部分汇聚。局部凸组合避免直接复制，但不等于物理混配。

$$h \mid c \sim \mathcal{N}(\mu_c, w_c\Sigma_c + (1-w_c)\Sigma_{\text{pool}}), \quad w_c = \frac{n_c}{n_c+10} \tag{13}$$

$$x_{\text{new}} = ux_a + (1-u)x_b, \quad u \sim \text{Uniform}(0.1, 0.9), \quad a, b \in c \tag{14}$$

受约束版本在同风格最近三个邻居间取权重；DMS 和 4VG 按已支持的正值 log10 关系先回到正值域插值再返回，其他字段仍在附件尺度统计插值。检测限替代值的解释不完整，因此前者也只是有证据条件的建模变换。投影训练范围后，以同风格训练近邻距离的95%分位限制联合支持；每组最多尝试 40×250 次，失败不伪装为新样本。

MMD^2[12] 的表示和核带宽只用 G 训练资料确定，所有方法共用；真实参考为 T 的非缺失测量，另报 KS、复制率、J 模型目标类别判断及范围违反。风格留出常只有两款，不能以小距离或不显著证明分布一致。

风格	方法	MMD^2	J目标类别比例
Kriek	PCA高斯	0.2832	59.5%
Lambic	PCA高斯	0.2095	38.5%
Tripel	PCA高斯	0.1088	28.5%
Kriek	重采样	0.2871	80.0%
Lambic	重采样	0.2226	88.5%
Tripel	重采样	0.1034	56.5%
Kriek	受约束凸组合	0.2988	95.0%
Lambic	受约束凸组合	0.2181	99.5%

风格	方法	MMD^2	J目标类别比例
Tripel	受约束凸组合	0.1158	69.5%
Kriek	局部凸组合	0.3005	95.5%
Lambic	局部凸组合	0.2206	97.5%
Tripel	局部凸组合	0.1135	75.5%

受约束方法在已检查的训练范围内违反率为零，未精确复制训练点；但 MMD 未全面优于重采样，且与普通局部凸组合各有取舍。J 的类别判断也不能作为生成酒真实标签。完整生成谱和感官代理画像保存于支撑文件，未将其称为重新测得的感官结果。

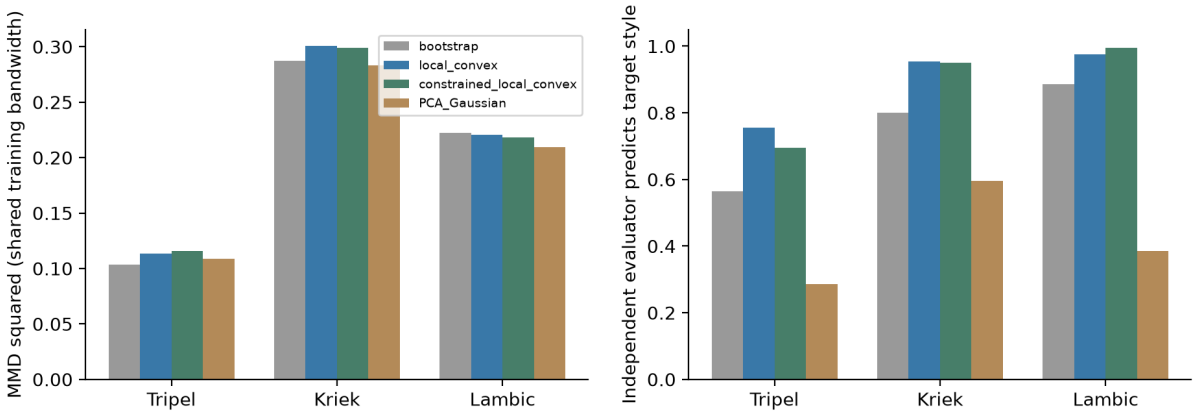


图6 固定训练核带宽下的分布比较及未参与搜索的J模型判断，五次划分平均

6.3 信赖域、Pareto选择与逐项缺陷

迁移沿源样本与 Tripel 的 G 训练锚点各取30个权重。候选投影范围后逐项限制 DMS、双乙酰、4VG、trans-2-nonenal 不增加，并同时要求岭与 ET 两个 G 感官代理的 stale_hops 不增加。搜索 Tripel 概率至少0.5，且超过原样和仅截尾对照至少0.01；同时满足目标经验支持与指定改动半径。

$$x_{\text{candidate},d} \le x_{\text{source},d}, \qquad \forall d \in D \tag{15}$$

$$f_{\text{stale}}(x_{\text{candidate}}) \le f_{\text{stale}}(x_{\text{source}}) \tag{16}$$

$$\frac{\|z_{\text{candidate}} - z_{\text{source}}\|_2}{\sqrt{p}} \le r, \quad r \in \{0.25, 0.5, 1, 2\} \tag{17}$$

排除违反硬约束的候选后，对标准化改变量、目标训练支持距离、DMS/4VG正值变换下的阈值代理超额作 Pareto 筛选，再从非支配候选选择最小改动者。19 ug/L 与0.2 mg/L 来自附件中位阈值，仅作为这两项正值代理，不替代逐样本浓度和检测限核验。trans-2-nonenal 阈值缺失；双乙酰等配对不匹配未完全解释，所以没有任何候选获“完整绝对化学认证”。

源缺陷缺失时仅以训练中位数参与条件搜索并单独标记，不能作真实源上界。源值非缺失也不表示已验证浓度、基质或感官阈值。找不到非平凡候选时保留原样且记失败；原样满足不增加不计作迁移成功。

改动半径	搜索非平凡可行	全部记录J代理支持	T源记录J代理支持
0.25	1/55	0/55	0/10

改动半径	搜索非平凡可行	全部记录J代理支持	T源记录J代理支持
0.5	5/55	1/55	0/10
1.0	51/55	11/55	2/10
2.0	51/55	11/55	2/10

半径1的搜索可行记录中，10条依赖缺失源值插补条件，41条具有四项附件源字段观测。后者仍不等于原浓度、全部阈值或真实缺陷已认证。

半径1时，T源样本的 10 条重复划分记录对应 7 款不同酒，只有 2 条同时满足 J 的 Tripel 概率改善、目标类别判断及 stale_hops 代理不增加。G/J训练源记录另报，不与T源合并冒充独立结果。这个比例是跨模型代理一致性，不是真实品评成功率；不同半径重复使用同一源酒，也不能累加为独立样本。

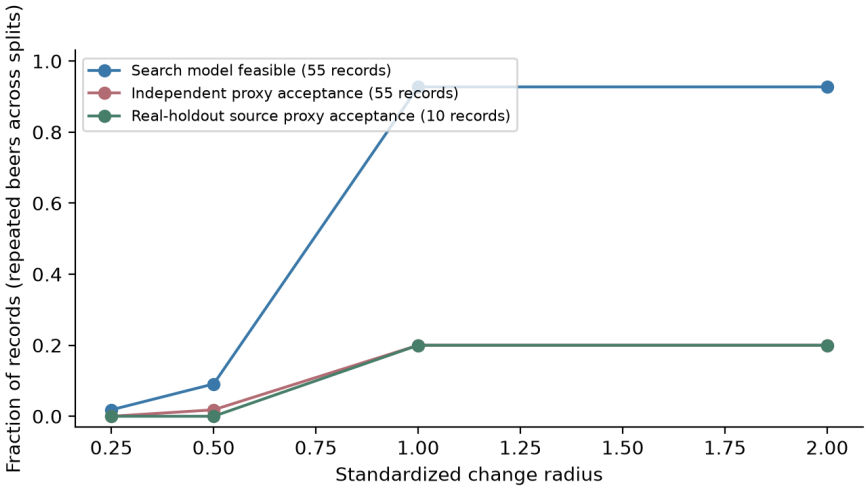


图7 搜索可行率与独立代理支持率的差异；分母为重复划分记录而非新增酒款

与前阶段只追求单个分类器高分相比，本阶段增加了最小改动、双代理、联合支持和J评价。这些更严格条件带来更低的通过率，不能与旧稿11/11直接当同定义成绩比较。若某候选利用模型弱点，其失败本身是重要结果。自适应优化后的候选也不能继承普通逐点保形区间的无条件覆盖解释。

7 结果评价、贡献与限制

预测对照揭示目标范围和维度异质性，收缩融合的收益必须与实际误差及复杂度一起判断；网络消融没有支持额外预测优势，因此不把网络图形复杂性当作贡献。独立迁移诊断揭示搜索成绩与跨模型支持之间的明显差距。主要贡献是形成可核查、不过度宣称的任务化证据，而不是创造新的基础算法。

当前 40 项程序检查通过，涵盖主键、原件、训练内预处理、嵌套预测角色、图零惩罚等价、风格置换、固定核MMD、生成范围、G/J/T隔离、缺失源标记、失败迁移与校准角色。检查验证实现性质，不能证明全部科学假设成立。配置、源程序与原件哈希、划分和结果均保存。

剩余限制包括：缺少真实绝对评分锚点；原作者预处理和检测限未完全恢复；大部分化学机理尚无目标特异性文献或加标验证；稀有风格留出过少；生成的理化量未形成完整酿造方程；独立评价仍是统计代理且无新外部酒或真实品评。拟议0.5/1/2倍阈值敏感性、所有聚合公式恢复和新增

真实锚点并未完成，不能写成已验证。

8 结论与建议

在当前附件连续尺度及明确核心范围下，ET 的聚合留出 MAE 为 0.34964，收缩融合为 0.34664。相对分级与区间集合给出可复核画像，但不能恢复真实原始0-6分。图约束未提供实用预测增益，因素解释应结合风格内贡献、模块替代和化学证据。

受约束局部生成实现了无精确复制和已检范围可行，但分布拟合没有全面超越简单重采样。半径1的T源迁移记录仅 2/10 获得独立代理条件共同支持，尚无完整绝对化学或真实品评认证。后续优先获取评分锚点和明确检测限/阈值，补充目标特异性化学证据，再以新酒或盲评确认效果；不建议仅继续增加模型名称。

参考文献

[1] Schreurs M, Piampongsant S, Roncoroni M, et al. Predicting and improving complex beer flavor through machine learning. Nature Communications, 15:2368, 2024. DOI:10.1038/s41467-024-46346-0.

[2] Geurts P, Ernst D, Wehenkel L. Extremely randomized trees. Machine Learning, 63:3-42, 2006. DOI:10.1007/s10994-006-6226-1.

[3] Friedman J H. Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5):1189-1232, 2001. DOI:10.1214/aos/1013203451.

[4] Breiman L. Stacked Regressions. Machine Learning, 24:49-64, 1996. DOI:10.1007/BF00117832.

[5] Li C, Li H. Variable selection and regression analysis for graph-structured covariates with an application to genomics. The Annals of Applied Statistics, 4(3):1498-1516, 2010. DOI:10.1214/10-AOAS332.

[6] Lei J, G'Sell M, Rinaldo A, et al. Distribution-Free Predictive Inference for Regression. Journal of the American Statistical Association, 113(523):1094-1111, 2018. DOI:10.1080/01621459.2017.1307116.

[7] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12:2825-2830, 2011.

[8] Bengio Y, Grandvalet Y. No Unbiased Estimator of the Variance of K-Fold Cross-Validation. Journal of Machine Learning Research, 5:1089-1105, 2004.

[9] Vanbeneden N, Saison D, Delvaux F, Delvaux F R. Decrease of 4-vinylguaiacol during beer aging and formation of apocynol and vanillin in beer. Journal of Agricultural and Food Chemistry, 56(24):11983-11988, 2008. DOI:10.1021/jf8019453.

[10] Omur-Ozbek P, Dietrich A M, Duncan S E, Lee Y. Role of lipid oxidation, chelating agents, and antioxidants in metallic flavor development in the oral cavity. Journal of Agricultural and Food Chemistry, 60(9):2274-2280, 2012. DOI:10.1021/jf204277v.

[11] Pater A, Satora P, Januszek M. Effect of Coriander Seed Addition at Different Stages of Brewing on Selected Parameters of Low-Alcohol Wheat Beers. Molecules, 29(4):844, 2024. DOI:10.3390/molecules29040844.

[12] Gretton A, Borgwardt K M, Rasch M J, et al. A Kernel Two-Sample Test. Journal of Machine Learning Research, 13:723-773, 2012.

[13] OpenAI. Codex（系统提示模型家族 GPT-6，精确可公开版本待实际界面核验），使用日期：2026-10-24 至 2026-10-25. 重要交互、采纳与未完成的人工作业见随附 AI 工具使用详情说明阶段版。

AI工具使用与研究贡献声明

本参考研究由 OpenAI Codex 辅助资料核对、代码实现、实验运行、图表与稿件编写[13]。所述实验确已运行，但尚无证据表明队员已独立完成或复核核心工作，不能据此作相反声明。精确公开模型版本、队员实际贡献及最终采纳修改需据实补充；随附阶段版说明保留重要提示词、关键回复和实际执行范围。

附录A 完整Top-3、复现与结果索引

以下为全部50个目标的描述性Top-3。顺序按全数据描述模型系数绝对值，括号为固定全数据所选参数条件下的bootstrap入选频率；不等于致因概率。逐折选择与贡献、化学证据层级、全部连续预测和250×50相对等级另存，原始赛题数据不随公开支撑包分发。

目标	Top-3 字段及条件入选频率
A_malt_all	X2.acetylfuran (0.85)；MeSH (0.30)；ethyl.furoate (0.27)
A_malt_grain	car.3.en.2.one (0.55)；X5.methylfurfural (0.32)；gamma.nonalactone (0.36)
A_malt_bread	furfural (0.55)；gamma.nonalactone (0.15)；X1.10.di.epi.cubenol (0.28)
A_malt_cara	X2.acetylfuran (0.63)；octyl_acetate (0.27)；SO2.mg.L. (0.27)
A_malt_burn	methylpyrazine (0.74)；X2.acetylfuran (0.65)；MeSH (0.15)
A_hops_all	iron (0.53)；DMS (0.29)；X6.methyl.5.hepten.2.one (0.28)
A_hops_citrus	X6.methyl.5.hepten.2.one (0.45)；geraniol (0.29)；terpinolene (0.10)
A_hops_tropical	DMS (0.52)；beta.phellandrene (0.17)；citronellol (0.28)
A_hops_noble	ethyl_octanoate (0.35)；ethyl_decanoate (0.29)；iron (0.28)
A_hops_woody	ethyl_2.methyl_butyrate (0.33)；iron (0.25)；X.E..ortho.methoxy.cinnamaldehyde (0.20)
A_esters_all	acetaldehyde (0.82)；X.E..beta.ionone (0.23)；methylpyrazine (0.26)
A_esters_ethac	acetaldehyde (0.42)；X2E.6Z.farnesal (0.21)；ethyl_acetate (0.12)
A_esters_isoaa	ambroxide (0.33)；n.hexanol (0.23)；ethyl.hexadecanoate (0.19)
A_esters_flower	X.E..alpha.ionone (0.43)；beta.phellandrene (0.31)；X.E..beta.ionone (0.31)
A_esters_fruity	beta.phellandrene (0.69)；X1.10.di.epi.cubenol (0.47)；X.E..beta.ionone (0.14)
F_malt_all	acetaldehyde (0.78)；X2.acetylfuran (0.67)；X.E..alpha.ionone (0.27)
F_malt_grain	X2.3.butanedione (0.37)；terpinen.4.ol (0.38)；X3.methyl.2.butenal (0.30)
F_malt_bread	caryophylla.4.12..8.13..dien.5.alpha.ol (0.30)；terpinen.4.ol (0.32)；acetaldehyde (0.20)
F_malt_cara	acetaldehyde (0.65)；isobutyl.hexanoate (0.29)；ethyl.nicotinate (0.24)
F_malt_burn	X2.acetylfuran (0.48)；methylpyrazine (0.52)；MeSH (0.42)
F_hops_all	cis.dihydro.beta.terpineol (0.65)；iron (0.70)；ethyl_isovalerate (0.43)
F_hops_citrus	geosmin (0.32)；geraniol (0.25)；X2.acetylfuran (0.20)

目标	Top-3 字段及条件入选频率
F_hops_tropical	DMS (0.33)；X6.methyl.5.hepten.2.one (0.33)；beta.phellandrene (0.12)
F_hops_noble	iron (0.69)；cis.dihydro.beta.terpineol (0.33)；X14.hydroxy.alpha.humulene (0.29)
F_hops_woody	manool (0.67)；cis.dihydro.beta.terpineol (0.31)；isobutanol (0.14)
F_esters_all	acetaldehyde (0.89)；X1.10.di.epi.cubenol (0.32)；phenethyl_alcohol (0.33)
F_esters_ethac	acetaldehyde (0.35)；X2E.6Z.farnesal (0.30)；glycerol (0.20)
F_esters_isoaa	acetaldehyde (0.17)；phenethyl_alcohol (0.21)；isopentyl_acetate (0.14)
F_esters_flower	beta.phellandrene (0.36)；eucalyptol (0.30)；benzyl.alcohol (0.17)
F_esters_fruity	X1.10.di.epi.cubenol (0.56)；beta.phellandrene (0.28)；X.E..beta.ionone (0.14)
acidity	lactic_acid.mg.L. (1.00)；acetic_acid.g.L. (0.79)；ethyl.phenylacetate (0.07)
bitternes	iron (0.94)；ethyl_octanoate (0.67)；cis.dihydro.beta.terpineol (0.52)
sweetness	ethyl_decanoate (0.48)；iron (0.43)；ethyl_octanoate (0.23)
X4vg	X4VG (1.00)；delta.terpinyl.acetate (0.30)；ethyl_isovalerate (0.14)
diacetyl	beta.phellandrene (0.33)；X.E..beta.ionone (0.18)；pentyl.octanoate (0.23)
dms	SO2.mg.L. (0.60)；caryophylla.4.12..8.13..dien.5.alpha.ol (0.28)；X.Z..dihydro.apofarnesal (0.09)
metallic	iron (0.86)；ethyl_isovalerate (0.54)；isopentyl.butanoate (0.27)
stale_hops	X2.butyloctanoate (0.31)；iron (0.34)；X.E..ortho.methoxy.cinnamaldehyde (0.23)
t2n	germacrene.D (0.69)；isopentyl.hexanoate (0.34)；glycerol (0.29)
orange	X1.alpha.terpineol (0.81)；limonene (0.40)；geraniol (0.48)
coriander	x.methyl.1.octanol (0.80)；linalool (0.60)；geraniol (0.46)
clove	linalool (0.74)；eucalyptol (0.55)；limonene (0.47)
lactic	lactic_acid.mg.L. (0.99)；acetic_acid.g.L. (0.61)；ethyl.phenylacetate (0.09)
acetic	lactic_acid.mg.L. (0.99)；acetic_acid.g.L. (0.70)；DES (0.16)
barnyard	gamma.eudesmol (0.33)；X2.butyloctanoate (0.27)；iron (0.28)
alcohol	acetaldehyde (0.85)；ethanol..v.v. (0.81)；ethyl_octanoate (0.42)
aftertaste	ethyl_isovalerate (0.56)；glycerol (0.41)；ethanol..v.v. (0.33)
body	glycerol (0.71)；acetaldehyde (0.59)；X3.methyl.2.hexanol (0.43)
co2	ethyl_octanoate (0.67)；X3.methyl.2.hexanol (0.60)；ethyl_hexanoate (0.30)
overall	ethyl_acetate (0.42)；X2.butyloctanoate (0.28)；X.E..beta.ocimene (0.15)

复现顺序：使用官方A附件并核对哈希；安装 requirements-lock.txt；运行 run_stage3.py q1、q2、q3；运行 pytest；然后生成图表与本稿。全部角色和参数位于 outputs/stage3/，执行版配置为 config_stage3.json。软件为Python 3.12、NumPy、pandas、SciPy、scikit-learn、

NetworkX、Matplotlib、openpyxl；精确运行版本见各问run_manifest。文档另用ReportLab、pypdf和Pillow。

材料	内容与相对路径
评分核查	outputs/stage3_design/scoring_field_map.json 等，只读核查
第一问	outputs/stage3/q1：分范围成绩、嵌套权重、相对评分、区间、缺失与截尾
第二问	outputs/stage3/q2：图消融、跨拟合Top-3、模块与风格诊断
第三问	outputs/stage3/q3：G/J/T角色、候选锁、生成谱、画像、迁移及未知项
验证与来源	outputs/stage3/validation_summary.json；docs/SOURCES.md
历史记录	stage1、stage2及旧稿保留，不混作同定义新成绩

附录B 本项目完整源程序

以下列出本项目自编计算、核查、图表、排版、打包和测试源程序，以及执行配置。外部依赖库不逐行重印，名称版本见锁定文件和环境记录。长行会为版面自动折行；极少数字体不支持字符以Unicode转义显示。可执行原文件随支撑包保留，文件哈希列于附录索引。

源程序 1 src/__init__.py

```
"""Research support code. See docs/AI_USAGE.md before competition use."""
```

源程序 2 src/audit.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24."""
from pathlib import Path

import numpy as np
import pandas as pd
from openpyxl import load_workbook

from .common import progress, save_csv, sha256, write_json
from .data import META


def run_audit(data, out, cfg):
    dest = Path(out) / "audit"
    dest.mkdir(parents=True, exist_ok=True)
    progress("Audit: checking source hashes, sample keys, scales and sensory templates")
    hashes, shapes = [], []
    for p in sorted(data.root.iterdir()):
        if p.is_file():
            hashes.append({"file": p.name, "bytes": p.stat().st_size, "sha256": sha256(p)})
            if p.suffix == ".csv":
                frame = pd.read_csv(p)
                shapes.append({"file": p.name, "rows": len(frame), "columns_including_metadata":
len(frame.columns), "missing_cells": int(frame.isna().sum().sum())})
    save_csv(pd.DataFrame(hashes), dest / "source_manifest.csv")
    save_csv(pd.DataFrame(shapes), dest / "file_shapes.csv")
    save_csv(data.meta.groupby("tasting_category_fine").size().rename("n").reset_index(), dest / "style_counts.csv")
    save_csv(data.chemical_meta, dest / "chemical_dictionary.csv")
    names = data.sensory_names[data.sensory_names.dataset == "Trained
panel"].drop_duplicates("R_name").set_index("R_name")
    targets = []
    for c in data.Y:
        alias = "X4vg" if c == "X4vg" else c
        targets.append({"field": c, "label": names.at[alias, "CleanName_PanelFeedback"] if alias in names.index
else c,
                        "group": "Aroma" if c.startswith("A_") else "Flavour" if c.startswith("F_") else "Other
sensory",
                        "observed_scale": "provided centered/standardized continuous values",
                        "absolute_0_6_calibration": "unresolved"})
    save_csv(pd.DataFrame(targets), dest / "sensory_dictionary.csv")
    records = []
    for col in data.X:
        a = data.X[col]
        raw = data.calibrated[col] if col in data.calibrated else None
        row = {"field": col, "n_missing": int(a.isna().sum()), "minimum": float(a.min()), "maximum": float(a.max()),
               "q01": float(a.quantile(.01)), "q99": float(a.quantile(.99)), "transform_evidence": "not_available",
               "raw_identity_fraction": np.nan, "raw_log10_fraction": np.nan, "n_raw_positive_pairs": 0}
        if raw is not None:
            mask = a.notna() & raw.notna() & (raw > 0)
            if mask.sum():
                identity = np.isclose(a[mask], raw[mask], atol=1e-7, rtol=1e-6)
                logarithm = np.isclose(a[mask], np.log10(raw[mask]), atol=1e-7, rtol=1e-6)
                fi, fl = float(identity.mean()), float(logarithm.mean())
                row.update(raw_identity_fraction=fi, raw_log10_fraction=fl, n_raw_positive_pairs=int(mask.sum()))
                row["transform_evidence"] = "verified_log10_positive_pairs" if fl >= .999 else "
verified_identity_positive_pairs" if fi >= .999 else "mostly_identity_with_mismatches" if fi >= .9 else "
mixed_or_unresolved"
        records.append(row)
    scales = pd.DataFrame(records)
    save_csv(scales, dest / "field_scales.csv")
    # Inspect actual input cells, not formula caches that evaluate blanks to zero.
    w = load_workbook(data.root / "3.xlsx", data_only=False)
    input_rows = [7, 20, 33, 45, 50, 54, 58, 62, 68, 72, 76, 82, 86, 90, 94]
    input_cols = [3, 5, 7, 9, 11, 13, 15, 19, 21, 23, 25, 27, 29, 31]
    template = []
```

```
    for sh in w:
        if not sh.title.startswith("beer_"):
            continue
        marks = [sh.cell(r, c).coordinate for r in input_rows for c in input_cols if sh.cell(r, c).value is not None]
        template.append({"sheet": sh.title, "nonempty_main_scoring_inputs": len(marks), "input_cells": ",".join(marks)})
    save_csv(pd.DataFrame(template), dest / "sensory_template_audit.csv")
    key_stats = []
    for n in [1, 4, 7]:
        d = pd.read_csv(data.root / f"{n}.csv")
        key_stats.append({"file": f"{n}.csv", "unique_beer": int(d.beer.nunique()), "unique_beer_id": int(d.beer_id.nunique())})
    save_csv(pd.DataFrame(key_stats), dest / "join_key_audit.csv")
    values = data.given_corr.to_numpy()
    evidence = {
        "pearson_matrix_mean_absolute_difference": float(np.nanmean(np.abs(data.X.corr().to_numpy() - values))),
       "spearman_matrix_mean_absolute_difference": float(np.nanmean(np.abs(data.X.corr(method="spearman").to_numpy() - values))),
        "n_samples": len(data.X), "n_features": data.X.shape[1], "n_targets": data.Y.shape[1],
        "n_styles": int(data.meta.tasting_category_fine.nunique()),
        "grade_calibration": "UNRESOLVED: no observed paired absolute 0-6 anchors confirmed",
        "template_summary_barnyard_formula": str(w["summary"]["AX2"].value),
        "template_barnyard_expected_cell": "beer_1!AG90 based on the form's Barnyard label; team review required",
        "scope": "source evidence verified by computation; model conclusions remain exploratory",
    }
    write_json(dest / "verified_facts.json", evidence)
```

源程序 3 src/common.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
Team verification and modification are required; see docs/AI_USAGE.md.
"""
from __future__ import annotations

import hashlib
import json
import platform
from datetime import datetime
from importlib.metadata import version
from pathlib import Path
from zoneinfo import ZoneInfo

import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler


def write_json(path, value):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    Path(path).write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")


def save_csv(frame, path):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    frame.to_csv(path, index=False, encoding="utf-8-sig", float_format="%.9g")


def sha256(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()


def environment():
    return {
        "timestamp": datetime.now(ZoneInfo("Asia/Shanghai")).isoformat(),
        "python": platform.python_version(),
        "packages": {p: version(p) for p in ["numpy", "pandas", "scipy", "scikit-learn", "networkx", "matplotlib", "openpyxl"]},
    }


class TrainQuantileClipper(TransformerMixin, BaseEstimator):
    """Learn bounds on training samples only. Missing values remain missing."""

    def __init__(self, lower=0.01, upper=0.99):
        self.lower = lower
        self.upper = upper

```

```
    def fit(self, X, y=None):
        a = np.asarray(X, dtype=float)
        self.lower_ = np.nanquantile(a, self.lower, axis=0)
        self.upper_ = np.nanquantile(a, self.upper, axis=0)
        self.n_features_in_ = a.shape[1]
        return self

    def transform(self, X):
        a = np.asarray(X, dtype=float)
        if a.shape[1] != self.n_features_in_:
            raise ValueError("Feature count differs from training")
        return np.clip(a, self.lower_, self.upper_)


def preprocessing():
    return Pipeline([
        ("clip", TrainQuantileClipper()),
        ("impute", SimpleImputer(strategy="median", keep_empty_features=True)),
        ("scale", StandardScaler()),
    ])


def ecdf_grades(reference, values):
    """ASSUMPTION: seven empirical bins, NOT calibrated sensory intensity."""
    bounds = np.quantile(np.asarray(reference), np.arange(1, 7) / 7, axis=0)
    values = np.asarray(values)
    return (values[:, None, :] > bounds[None, :, :]).sum(axis=1).astype(int)


def progress(text):
    print(text, flush=True)
```

源程序 4 src/data.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
All source files are read-only. Never join on precision-damaged beer_id.
"""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd

META = ["beer", "beer_id", "tasting_category_fine"]
AGGREGATES = {"kcalperc", "esters_sum", "aroma_hops_sum", "acids_sum", "sulfur_sum"}
NON_MOLECULES = {"color.EBC.", "betaglucan.mg.L.", "pH", "protein.g.L.", "bitterness", "GFS", "CO2.PSI.", "
unfermentables"} | AGGREGATES
CHEMICAL_DEFECTS = ["DMS", "X2.3.butanedione", "X4VG", "X.E..2.nonenal"]


@dataclass
class BeerData:
    root: Path
    meta: pd.DataFrame
    X: pd.DataFrame
    Y: pd.DataFrame
    given_corr: pd.DataFrame
    thresholds: pd.DataFrame
    sensory_names: pd.DataFrame
    calibrated: pd.DataFrame
    chemical_meta: pd.DataFrame


def find_data(argument=None):
    if argument:
        root = Path(argument).resolve()
        if not (root / "1.csv").exists():
            raise FileNotFoundError(f"Expected 1.csv in {root}")
        return root
    repo = Path(__file__).resolve().parents[1]
    for root in [repo / "data/raw", repo.parent / "materials/A/附件"]:
        if (root / "1.csv").exists():
            return root
    raise FileNotFoundError("Supply --data-dir pointing to the original A attachments")


def align_by_beer(chemical, sensory):
    for label, frame in [("chemical", chemical), ("sensory", sensory)]:
        if frame["beer"].isna().any() or frame["beer"].duplicated().any():
```

```
            raise ValueError(f"{label}: beer must be non-null and unique")
    if set(chemical.beer) != set(sensory.beer):
        raise ValueError("Chemical and sensory sample sets differ")
    sensory = sensory.set_index("beer").loc[chemical.beer].reset_index()
    if not chemical.tasting_category_fine.reset_index(drop=True).equals(sensory.tasting_category_fine):
        raise ValueError("Style labels do not agree after beer alignment")
    return sensory


def load_data(root):
    root = Path(root)
    chemical = pd.read_csv(root / "1.csv")
    sensory = align_by_beer(chemical, pd.read_csv(root / "4.csv"))
    X = chemical.drop(columns=META).astype(float)
    Y = sensory.drop(columns=META).astype(float)
    if Y.isna().any().any():
        raise ValueError("Targets contain missing values; target handling needs explicit review")
    corr = pd.read_csv(root / "2.csv", index_col=0)
    if corr.shape != (X.shape[1], X.shape[1]):
        raise ValueError("Correlation matrix dimensions do not match chemical fields")
    if list(corr.index) != list(corr.columns):
        raise ValueError("Correlation row/column names differ")
    # The positional alias map is empirically checked over the full matrix.
    # Thus it is not inferred from just matching the number of columns.
    mismatch = np.nanmean(np.abs(X.corr(method="spearman").to_numpy() - corr.to_numpy()))
    if mismatch > 0.002:
        raise ValueError("Cannot verify ordered chemical aliases against sample correlations")
    book = root / "8.xlsx"
    thresholds = pd.read_excel(book, sheet_name="Sensory thresholds", na_values="NA")
    names = pd.read_excel(book, sheet_name="Sensory names", na_values="NA")
    calibrated = pd.read_excel(book, sheet_name="Calibrated chemical dataset", na_values="NA")
    if calibrated.beer.duplicated().any():
        raise ValueError("Calibrated reference has duplicate beer keys")
    calibrated = calibrated.set_index("beer").loc[chemical.beer].reset_index()
    cm = thresholds.dropna(subset=["field_name_R"]).drop_duplicates("field_name_R").set_index("field_name_R")
    rows = []
    for col, alias in zip(X.columns, corr.index):
        row = {"field": col, "matrix_alias": alias, "kind": "aggregate" if col in AGGREGATES else "
physical_or_mixture" if col in NON_MOLECULES else "compound_or_element"}
        for name in ["chemical_name", "origin_paper", "unit", "off_flavors", "threshold_median", "threshold_mean"]:
            row[name] = cm.at[col, name] if col in cm.index else np.nan
        rows.append(row)
    return BeerData(root, chemical[META], X, Y, corr, thresholds, names, calibrated, pd.DataFrame(rows))
```

源程序 5 src/q1.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
Nested preprocessing/tuning and paired out-of-fold prediction. No raw-score claim.
"""
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn.base import clone
from sklearn.cross_decomposition import PLSRegression
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV, KFold, RepeatedStratifiedKFold
from sklearn.pipeline import Pipeline

from .common import ecdf_grades, preprocessing, progress, save_csv, write_json


def model_specs(cfg):
    def pipe(model):
        return Pipeline([("prep", preprocessing()), ("model", model)])
    return {
        "mean": (pipe(DummyRegressor(strategy="mean")), {}),
        "ridge": (pipe(Ridge()), {"model__alpha": cfg["ridge_alphas"]}),
        "pls": (pipe(PLSRegression(scale=False)), {"model__n_components": cfg["pls_components"]}),
        "extra_trees": (pipe(ExtraTreesRegressor(n_estimators=cfg["extra_trees"], min_samples_leaf=3,
max_features=.8, random_state=cfg["seed"], n_jobs=2)), {}),
    }


def fitted_model(name, X, y, cfg):
    estimator, params = model_specs(cfg)[name]
    if params:
```

```
        estimator = GridSearchCV(estimator, params, scoring="neg_mean_absolute_error", cv=KFold(cfg["inner_folds"], shuffle=True, random_state=cfg["seed"]), n_jobs=1, refit=True)
    estimator.fit(X, y)
    return estimator.best_estimator_ if params else estimator, estimator.best_params_ if params else {}


def metric_rows(y, pred, targets, model):
    return [{"model": model, "target": c,
             "mae": float(mean_absolute_error(y[:, j], pred[:, j])),
             "rmse": float(np.sqrt(mean_squared_error(y[:, j], pred[:, j]))),
             "r2": float(r2_score(y[:, j], pred[:, j]))}
            for j, c in enumerate(targets)]


def run_q1(data, out, cfg):
    dest = Path(out) / "q1"
    dest.mkdir(parents=True, exist_ok=True)
    (dest / "private").mkdir(exist_ok=True)
    X, Y = data.X.to_numpy(), data.Y.to_numpy()
    targets = data.Y.columns.to_list()
    split = RepeatedStratifiedKFold(n_splits=cfg["outer_folds"], n_repeats=cfg["outer_repeats"], random_state=cfg["seed"])
    sums = {m: np.zeros_like(Y) for m in model_specs(cfg)}
    count = np.zeros(len(X), dtype=int)
    assignments, selected, fold_metrics, robustness = [], [], [], []
    grades = np.zeros_like(Y)
    for fold, (train, test) in enumerate(split.split(X, data.meta.tasting_category_fine)):
        progress(f"Q1: outer fold {fold + 1}/{cfg['outer_folds'] * cfg['outer_repeats']}")
        count[test] += 1
        assignments.extend({"repeat": fold // cfg["outer_folds"], "fold": fold % cfg["outer_folds"], "beer": int(data.meta.iloc[i].beer), "role": "validation"} for i in test)
        for name in sums:
            est, params = fitted_model(name, X[train], Y[train], cfg)
            pred = est.predict(X[test])
            sums[name][test] += pred
            selected.append({"outer_fold": fold, "model": name, "parameters": str(params)})
            fold_metrics.append({"outer_fold": fold, "model": name, "mae": float(mean_absolute_error(Y[test], pred)), "macro_r2": float(r2_score(Y[test], pred)), "n_validation": len(test)})
            if name == "ridge":
                grades[test] += ecdf_grades(Y[train], pred)
                for rate in cfg["missing_rates"]:
                    corrupted = X[test].copy()
                    rng = np.random.default_rng(cfg["seed"] + fold)
                    artificial_missing = rng.random(corrupted.shape) < rate
                    corrupted[artificial_missing] = np.nan
                    pp = est.predict(corrupted)
                    robustness.append({"outer_fold": fold, "additional_missing_fraction": rate, "mae": float(mean_absolute_error(Y[test], pp)), "mask_seed": cfg["seed"] + fold})
    if not np.all(count == cfg["outer_repeats"]):
        raise AssertionError("Every sample must have exactly one held-out prediction per repeat")
    predictions = {name: value / count[:, None] for name, value in sums.items()}
    rows = []
    for name, pred in predictions.items():
        rows += metric_rows(Y, pred, targets, name)
        frame = pd.concat([data.meta[["beer", "tasting_category_fine"]], pd.DataFrame(pred, columns=targets)], axis=1)
        save_csv(frame, dest / f"oof_{name}_continuous.csv")
    metrics = pd.DataFrame(rows)
    save_csv(metrics, dest / "metrics_by_target.csv")
    save_csv(metrics.groupby("model")[["mae", "rmse", "r2"]].mean().reset_index(), dest / "metrics_summary.csv")
    save_csv(pd.DataFrame(fold_metrics), dest / "metrics_by_fold.csv")
    save_csv(pd.DataFrame(assignments), dest / "split_assignments.csv")
    save_csv(pd.DataFrame(selected), dest / "inner_selected_parameters.csv")
    save_csv(pd.DataFrame(robustness), dest / "missingness_robustness.csv")
    styles = []
    for style in sorted(data.meta.tasting_category_fine.unique()):
        ix = np.flatnonzero(data.meta.tasting_category_fine.to_numpy() == style)
        styles.append({"style": style, "n": len(ix), "ridge_mae": float(mean_absolute_error(Y[ix], predictions["ridge"][ix]))})
    save_csv(pd.DataFrame(styles), dest / "metrics_by_style.csv")
    # Predeclared ridge is used downstream; do not choose the winner using OOF scores.
    model, params = fitted_model("ridge", X, Y, cfg)
    joblib.dump({"model": model, "features": data.X.columns.tolist(), "targets": targets}, dest / "private/model.joblib")
    save_csv(pd.DataFrame(model.named_steps["model"].coef_, index=targets, columns=data.X.columns).reset_index(names="target"), dest / "final_ridge_coefficients.csv")
    if cfg["export_assumption_grades"]:
        gg = np.rint(grades / count[:, None]).astype(int)
        frame = pd.concat([data.meta[["beer"]], pd.DataFrame(gg, columns=targets)], axis=1)
        frame["status"] = "ASSUMPTION_ONLY_training_ECDF_bins_not_absolute_sensory_scores"
        save_csv(frame, dest / "ASSUMPTION_percentile_grades_0_6.csv")
    write_json(dest / "status.json", {
```

```
        "continuous_prediction": "computed_repeated_out_of_fold_exploratory_baselines",
        "absolute_0_6_prediction": "UNRESOLVED_missing_raw_score_calibration",
        "proxy_grades": "empirical equal-frequency bins, not a recovery of the original 0-6 intensity",
        "primary_model": "ridge selected before evaluation; inner tuning only",
        "final_inner_parameters": params,
        "external_generalization": "not established; supplied targets already preprocessed by source authors",
        "intervals": "not calibrated; no nominal coverage claim is made",
    })
```

源程序 6 src/q2.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
Descriptive graph-regularized attribution, not causal identification.
"""
from pathlib import Path

import networkx as nx
import numpy as np
import pandas as pd
from scipy.linalg import solve
from scipy.stats import rankdata

from .common import preprocessing, progress, save_csv, write_json
from .data import NON_MOLECULES


def signed_laplacian(corr, threshold):
    W = np.asarray(corr, float).copy()
    W[~np.isfinite(W)] = 0
    W[np.abs(W) < threshold] = 0
    np.fill_diagonal(W, 0)
    W = (W + W.T) / 2
    return np.diag(np.abs(W).sum(axis=1)) - W


def graph_coefficients(Z, Y, corr, cfg, graph=True):
    L = signed_laplacian(corr, cfg["graph_threshold"]) if graph else np.zeros((Z.shape[1], Z.shape[1]))
    yc = Y - Y.mean(axis=0)
    return solve(Z.T @ Z + cfg["graph_ridge_alpha"] * np.eye(Z.shape[1]) + cfg["graph_ridge_lambda"] * L, Z.T @ yc,
assume_a="pos")


def cross_spearman(X, Y):
    a, b = rankdata(X, axis=0), rankdata(Y, axis=0)
    a -= a.mean(axis=0)
    b -= b.mean(axis=0)
    denom = np.sqrt((a * a).sum(axis=0)[:, None] * (b * b).sum(axis=0)[None, :])
    return np.divide(a.T @ b, denom, out=np.zeros((a.shape[1], b.shape[1])), where=denom > 0)


def top_indices(values, eligible):
    return eligible[np.argsort(-np.abs(values[eligible]), axis=0, kind="stable")[:3]]


def run_q2(data, out, cfg):
    dest = Path(out) / "q2"
    dest.mkdir(parents=True, exist_ok=True)
    X, Y = data.X.to_numpy(), data.Y.to_numpy()
    features, targets = data.X.columns.tolist(), data.Y.columns.tolist()
    eligible = np.array([j for j, c in enumerate(features) if c not in NON_MOLECULES])
    given = data.given_corr.to_numpy()
    graph = nx.Graph()
    graph.add_nodes_from(features)
    edge_rows = []
    for i in range(len(features)):
        for j in range(i + 1, len(features)):
            r = float(given[i, j])
            if abs(r) >= cfg["graph_threshold"]:
                graph.add_edge(features[i], features[j], weight=abs(r), correlation=r)
                edge_rows.append({"source": features[i], "target": features[j], "spearman_r": r, "absolute_weight":
abs(r)})
    save_csv(pd.DataFrame(edge_rows), dest / "coexpression_edges.csv")
    communities = list(nx.community.greedy_modularity_communities(graph, weight="weight"))
    modules = {node: i for i, group in enumerate(communities) for node in group}
    save_csv(pd.DataFrame([{"field": c, "module": modules[c], "degree": graph.degree(c), "weighted_degree":
graph.degree(c, weight="weight")} for c in features]), dest / "network_nodes.csv")
    graph_stats = []
    for threshold in [.6, .7, .8, .9]:
        gg = nx.Graph()
        gg.add_nodes_from(features)
```

```
        gg.add_edges_from((features[i], features[j]) for i in range(len(features)) for j in range(i + 1,
len(features)) if abs(given[i, j]) >= threshold)
        graph_stats.append({"threshold": threshold, "edges": gg.number_of_edges(), "components":
nx.number_connected_components(gg), "isolated_nodes": len(list(nx.isolates(gg)))})
    save_csv(pd.DataFrame(graph_stats), dest / "network_threshold_sensitivity.csv")
    prep = preprocessing()
    Z = prep.fit_transform(X)
    beta = graph_coefficients(Z, Y, given, cfg)
    ridge = graph_coefficients(Z, Y, given, cfg, graph=False)
    marginal = cross_spearman(Z, Y)
    frequencies = np.zeros_like(beta)
    rng = np.random.default_rng(cfg["seed"])
    for b in range(cfg["bootstrap_repeats"]):
        if b % 20 == 0:
            progress(f"Q2: bootstrap {b}/{cfg['bootstrap_repeats']}")
        ix = rng.integers(0, len(X), len(X))
        zb = preprocessing().fit_transform(X[ix])
        # Refit the network too. Do not merely resample previously computed ranks.
        corrb = np.corrcoef(rankdata(zb, axis=0), rowvar=False)
        bb = graph_coefficients(zb, Y[ix], corrb, cfg)
        for k in range(Y.shape[1]):
            frequencies[top_indices(bb[:, k], eligible), k] += 1
    frequencies /= cfg["bootstrap_repeats"]
    meta = data.chemical_meta.set_index("field")
    top_rows, agreement = [], []
    for k, target in enumerate(targets):
        chosen = top_indices(beta[:, k], eligible)
        other = top_indices(ridge[:, k], eligible)
        univariate = top_indices(marginal[:, k], eligible)
        def jac(a, b):
            return len(set(a) & set(b)) / len(set(a) | set(b))
        agreement.append({"target": target, "graph_vs_ridge_jaccard": jac(chosen, other), "
graph_vs_spearman_jaccard": jac(chosen, univariate), "ridge_vs_spearman_jaccard": jac(other, univariate)})
        for rank, j in enumerate(chosen, 1):
            c = features[j]
            origin = meta.at[c, "origin_paper"]
            top_rows.append({"target": target, "rank": rank, "field": c, "chemical_name": meta.at[c, "
matrix_alias"],
                             "graph_coefficient": beta[j, k], "association_spearman": marginal[j, k],
                             "bootstrap_top3_frequency": frequencies[j, k], "module": modules[c],
                             "origin": origin, "unit": meta.at[c, "unit"], "known_off_flavors": meta.at[c, "
off_flavors"],
                             "stability_flag": "low_selection_frequency" if frequencies[j, k] < .5 else "
frequently_selected_in_this_resampling",
                             "chemical_argument": f"Reference origin: {origin}. Verify target-specific mechanism
and units; coefficient is association, not causation.",
                             "team_mechanism_review": "required"})
    save_csv(pd.DataFrame(top_rows), dest / "top3_by_sensory_dimension.csv")
    save_csv(pd.DataFrame(agreement), dest / "method_jaccard.csv")
    save_csv(pd.DataFrame(frequencies, index=features, columns=targets).reset_index(names="field"), dest / "
bootstrap_selection_frequencies.csv")
    write_json(dest / "status.json", {
        "scope": "descriptive_exploratory_full_dataset_association",
        "nodes": graph.number_of_nodes(), "edges": graph.number_of_edges(), "modules": len(communities),
        "bootstrap_repeats": cfg["bootstrap_repeats"], "bootstrap_rebuilds_preprocessing_and_graph": True,
        "causal_status": "not_identified",
        "chemical_mechanism_status": "reference metadata attached; target-specific human literature review
required",
        "regularization_parameters": "fixed research defaults, not optimized for causal recovery",
        "eligible_compounds": len(eligible),
    })
```

源程序 7 src/q3.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
Small-sample conditional Gaussian reference and explicitly model-based transfers.
"""
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from scipy.stats import ks_2samp
from sklearn.covariance import LedoitWolf
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import balanced_accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline

```

```
from .common import ecdf_grades, preprocessing, progress, save_csv, write_json
from .data import CHEMICAL_DEFECTS


class ConditionalPCAGaussian:
    def __init__(self, components=10, seed=20261024):
        self.components, self.seed = components, seed

    def fit(self, X, labels):
        self.prep = preprocessing()
        z = self.prep.fit_transform(X)
        self.pca = PCA(n_components=min(self.components, len(z) - 1, z.shape[1]), random_state=self.seed)
        latent = self.pca.fit_transform(z)
        self.low = np.nanquantile(X, .01, axis=0)
        self.high = np.nanquantile(X, .99, axis=0)
        self.median = np.nanmedian(X, axis=0)
        pooled = LedoitWolf().fit(latent).covariance_
        self.means, self.covariances = {}, {}
        labels = np.asarray(labels)
        for style in np.unique(labels):
            a = latent[labels == style]
            self.means[style] = a.mean(axis=0)
            local = np.atleast_2d(np.cov(a, rowvar=False)) if len(a) > 1 else pooled
            weight = len(a) / (len(a) + 10)
            self.covariances[style] = weight * local + (1 - weight) * pooled + 1e-8 * np.eye(pooled.shape[0])
        return self

    def encode(self, X):
        return self.pca.transform(self.prep.transform(X))

    def decode(self, latent):
        z = self.pca.inverse_transform(latent)
        values = self.prep.named_steps["scale"].inverse_transform(z)
        return np.clip(values, self.low, self.high)

    def sample(self, style, n, rng):
        latent = rng.multivariate_normal(self.means[style], self.covariances[style], size=n)
        return self.decode(latent)


def chemical_nonincrease(source, candidate, positions, tolerance=1e-9):
    return np.all(np.asarray(candidate)[..., positions] <= np.asarray(source)[..., positions] + tolerance, axis=-1)


def biased_mmd(a, b):
    """RBF MMD squared; bandwidth from real reference only."""
    if len(a) < 2 or len(b) < 2:
        return np.nan
    dist = cdist(b, b, metric="sqeuclidean")
    positive = dist[dist > 0]
    bandwidth = np.median(positive) if len(positive) else 1.
    return float(np.exp(-cdist(a, a, "sqeuclidean") / (2 * bandwidth)).mean() + np.exp(-dist / (2 *
bandwidth)).mean() - 2 * np.exp(-cdist(a, b, "sqeuclidean") / (2 * bandwidth)).mean())


def run_q3(data, out, cfg):
    dest = Path(out) / "q3"
    dest.mkdir(parents=True, exist_ok=True)
    path = Path(out) / "q1/private/model.joblib"
    if not path.exists():
        raise FileNotFoundError("Q3 needs the Q1 reference predictor; run q1 first")
    bundle = joblib.load(path)
    predictor = bundle["model"]
    features, targets = data.X.columns.tolist(), data.Y.columns.tolist()
    if bundle["features"] != features or bundle["targets"] != targets:
        raise ValueError("Q1 predictor schema mismatch")
    X = data.X.to_numpy()
    labels = data.meta.tasting_category_fine.to_numpy()
    train, test = train_test_split(np.arange(len(X)), test_size=cfg["generation_holdout_fraction"],
stratify=labels, random_state=cfg["seed"])
    save_csv(pd.DataFrame({"beer": data.meta.beer, "role": ["train" if i in set(train) else "holdout" for i in
range(len(X))]}), dest / "generator_split.csv")
    progress("Q3: fitting conditional PCA Gaussian on generator training samples only")
    generator = ConditionalPCAGaussian(cfg["generation_components"], cfg["seed"]).fit(X[train], labels[train])
    classifier = Pipeline([("prep", preprocessing()), ("model", LogisticRegression(C=1, max_iter=2000,
class_weight="balanced", random_state=cfg["seed"]))])
    classifier.fit(X[train], labels[train])
    class_index = {s: j for j, s in enumerate(classifier.classes_)}
    holdout_balanced = float(balanced_accuracy_score(labels[test], classifier.predict(X[test])))
    rng = np.random.default_rng(cfg["seed"])
    generated, metrics, portraits = [], [], []
    for style in cfg["generation_styles"]:
```

```
        values = generator.sample(style, cfg["generation_samples_per_style"], rng)
        ids = [f"{style}_{i + 1:03d}" for i in range(len(values))]
        frame = pd.DataFrame(values, columns=features)
        frame.insert(0, "style", style)
        frame.insert(0, "generated_id", ids)
        generated.append(frame)
        pred = predictor.predict(values)
        portrait = pd.DataFrame(pred, columns=targets)
        portrait.insert(0, "style", style)
        portrait.insert(0, "generated_id", ids)
        portraits.append(portrait)
        real = X[test[labels[test] == style]]
        zs = generator.prep.transform(values)
        zr = generator.prep.transform(real)
        ztrain = generator.prep.transform(X[train])
        distances = cdist(zs, ztrain)
        train_style = X[train[labels[train] == style]]
        bootstrap = train_style[rng.integers(0, len(train_style), len(values))]
        # Natural missingness in held-out measurements must not erase all KS diagnostics.
        ks_values = []
        for j in range(X.shape[1]):
            observed = real[:, j][np.isfinite(real[:, j])]
            if len(observed):
                ks_values.append(ks_2samp(values[:, j], observed, method="auto").statistic)
        # Both metrics use independent real holdout samples; small counts are explicit.
        metrics.append({"style": style, "n_generated": len(values), "n_training_style": len(train_style), "
n_holdout_style": len(real),
                        "mean_marginal_KS_holdout": float(np.mean(ks_values)), "n_KS_fields_observed":
len(ks_values),
                        "RBF_MMD2_holdout": biased_mmd(zs, zr),
                        "bootstrap_baseline_MMD2_holdout": biased_mmd(generator.prep.transform(bootstrap), zr),
                        "mean_target_style_probability": float(classifier.predict_proba(values)[:,
class_index[style]].mean()),
                        "nearest_train_distance_mean": float(distances.min(axis=1).mean()),
                        "exact_training_copy_fraction": float((distances.min(axis=1) < 1e-8).mean()),
                        "heldout_reference_warning": "few independent style examples; descriptive distances only,
no quality guarantee"})
    all_generated = pd.concat(generated, ignore_index=True)
    save_csv(all_generated, dest / "generated_compound_profiles.csv")
    save_csv(pd.concat(portraits, ignore_index=True), dest / "generated_sensory_continuous.csv")
    save_csv(pd.DataFrame(metrics), dest / "generation_diagnostics.csv")
    if cfg["export_assumption_grades"]:
        a = pd.concat(portraits, ignore_index=True)
        grade = ecdf_grades(data.Y.to_numpy()[train], a[targets].to_numpy())
        proxy = pd.concat([a[["generated_id", "style"]], pd.DataFrame(grade, columns=targets)], axis=1)
        proxy["status"] = "ASSUMPTION_ONLY_ECDF_not_absolute_sensory_scores"
        save_csv(proxy, dest / "ASSUMPTION_generated_percentile_grades.csv")
    positions = [features.index(c) for c in CHEMICAL_DEFECTS]
    stale = targets.index("stale_hops")
    target = cfg["transfer_target"]
    source = cfg["transfer_source"]
    transfers, diagnostics, checks, transfer_sensory = [], [], [], []
    for i in np.flatnonzero(labels == source):
        original = X[i]
        x = np.where(np.isfinite(original), original, generator.median)
        base_pred = predictor.predict(x[None])[0]
        base_prob = classifier.predict_proba(x[None])[0, class_index[target]]
        cleaning_only = np.clip(x, generator.low, generator.high)
        cleaning_only[positions] = np.minimum(cleaning_only[positions], x[positions])
        cleaning_prob = classifier.predict_proba(cleaning_only[None])[0, class_index[target]]
        latent = generator.encode(x[None])[0]
        desired = generator.decode((latent + generator.means[target] - generator.means[source])[None])[0]
        candidates = []
        for t in np.linspace(.025, 1.25, 50):
            candidate = np.clip(x + t * (desired - x), generator.low, generator.high)
            candidate[positions] = np.minimum(candidate[positions], x[positions])
            candidates.append(candidate)
        candidates = np.asarray(candidates)
        candidate_pred = predictor.predict(candidates)
        chemical_ok = chemical_nonincrease(x, candidates, positions)
        stale_ok = candidate_pred[:, stale] <= base_pred[stale] + 1e-9
        target_prob = classifier.predict_proba(candidates)[:, class_index[target]]
        # Separate genuine style movement from the effects of clipping source outliers.
        feasible = chemical_ok & stale_ok & (target_prob > max(base_prob, cleaning_prob) + .01)
        if feasible.any():
            eligible = np.flatnonzero(feasible)
            chosen = eligible[np.argmax(target_prob[eligible])]
            value, pp, probability = candidates[chosen], candidate_pred[chosen], target_prob[chosen]
            status = "nontrivial_model_feasible_candidate"
        else:
            value, pp, probability = x.copy(), base_pred.copy(), base_prob
            status = "no_nontrivial_candidate_found_source_retained"
```

```
        beer = int(data.meta.iloc[i].beer)
        missing_defects = [c for c, j in zip(CHEMICAL_DEFECTS, positions) if not np.isfinite(original[j])]
        for c, j in zip(CHEMICAL_DEFECTS, positions):
            checks.append({"beer": beer, "defect": c, "source_value": x[j], "candidate_value": value[j], "
nonincrease": bool(value[j] <= x[j] + 1e-9), "scale": "supplied_feature_scale", "source_observed":
bool(np.isfinite(original[j])), "absolute_threshold_certified": False})
        checks.append({"beer": beer, "defect": "stale_hops", "source_value": base_pred[stale], "candidate_value":
pp[stale], "nonincrease": bool(pp[stale] <= base_pred[stale] + 1e-9), "scale": "Q1_model_predicted_continuous_sensory", "
source_observed": False, "absolute_threshold_certified": False})
        transfers.append({"beer": beer, "source_style": source, "target_style": target, **dict(zip(features,
value))})
        transfer_sensory.append({"beer": beer, **dict(zip(targets, pp))})
        diagnostics.append({"beer": beer, "generator_role": "holdout" if i in set(test) else "train", "status":
status,
                            "source_target_probability": base_prob, "candidate_target_probability": probability,
                            "cleaning_only_target_probability": cleaning_prob,
                            "supplied_scale_change_l2": float(np.linalg.norm(value - x)),
                            "chemical_nonincrease": bool(chemical_nonincrease(x, value, positions)),
                            "predicted_stale_hops_nonincrease": bool(pp[stale] <= base_pred[stale] + 1e-9),
                            "missing_source_defects": ",".join(missing_defects),
                            "out_of_training_envelope_fields": int(((value < generator.low - 1e-9) | (value >
generator.high + 1e-9)).sum()),
                            "guarantee_scope": "measurement nonincrease plus learned stale proxy; physical and
sensory experiment not performed"})
    save_csv(pd.DataFrame(transfers), dest / "pils_to_tripel_profiles.csv")
    save_csv(pd.DataFrame(transfer_sensory), dest / "pils_to_tripel_sensory_continuous.csv")
    save_csv(pd.DataFrame(diagnostics), dest / "transfer_diagnostics.csv")
    save_csv(pd.DataFrame(checks), dest / "defect_nonincrease_checks.csv")
    write_json(dest / "status.json", {
        "model": "conditional_PCA_Gaussian_with_partial_covariance_pooling",
        "generator_train_n": len(train), "generator_holdout_n": len(test),
        "style_classifier_holdout_balanced_accuracy": holdout_balanced,
        "primary_feature_scale": "supplied_mixed_transforms; see audit/field_scales.csv",
        "marginal_bounds": "training 1%-99% descriptor range; not a thermodynamic or brewing-feasibility guarantee",
        "aggregate_consistency": "aggregate descriptors generated jointly; exact chemical sum identities not
established",
        "sensory_portraits": "Q1 full-data fitted predictor, model-based only; not an independent tasting
validation",
        "defect_thresholds": "absolute thresholds incomplete and some field scales unresolved; no complete OAV
certification",
        "stale_hops_constraint": "prediction proxy only; actual sensory non-worsening unverified",
        "zero_change_transfers": "explicitly reported as no nontrivial feasible candidate, never counted as
successful style transfer",
    })
```

源程序 8 src/reporting.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24."""
import json
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from .common import progress, sha256, write_json


def make_report(data, out, cfg):
    out = Path(out)
    figures = out / "figures"
    figures.mkdir(exist_ok=True)
    progress("Report: preparing experiment summaries and scientific figures")
    metrics = pd.read_csv(out / "q1/metrics_summary.csv").set_index("model")
    per_target = pd.read_csv(out / "q1/metrics_by_target.csv")
    robustness = pd.read_csv(out / "q1/missingness_robustness.csv")
    top = pd.read_csv(out / "q2/top3_by_sensory_dimension.csv")
    jaccard = pd.read_csv(out / "q2/method_jaccard.csv")
    generation = pd.read_csv(out / "q3/generation_diagnostics.csv")
    transfers = pd.read_csv(out / "q3/transfer_diagnostics.csv")
    checks = pd.read_csv(out / "q3/defect_nonincrease_checks.csv")
    plt.rcParams.update({"font.size": 10, "axes.spines.top": False, "axes.spines.right": False, "figure.dpi": 140, "
savefig.facecolor": "white"})
    fig, ax = plt.subplots(1, 2, figsize=(11, 4.5), layout="constrained")
    order = ["mean", "ridge", "pls", "extra_trees"]
    ax[0].bar(order, metrics.loc[order, "mae"], color=["#9b9b9b", "#286eaa", "#549778", "#bd833a"])
    ax[0].set(ylabel="Macro MAE (provided continuous scale)", title="Repeated out-of-fold prediction")
    byrate = robustness.groupby("additional_missing_fraction").mae.agg(["mean", "std"])
    ax[1].errorbar(byrate.index * 100, byrate["mean"], yerr=byrate["std"], marker="o", capsize=4, color="#286eaa")
```

```
    ax[1].set(xlabel="Additional missing cells (%)", ylabel="Fold MAE", title="Ridge missingness sensitivity")
    ax[1].text(.02, .98, "Bars: fold SD, not a confidence interval", transform=ax[1].transAxes, va="top",
fontsize=8)
    fig.savefig(figures / "q1_validation.png", dpi=170)
    plt.close(fig)
    fig, ax = plt.subplots(1, 2, figsize=(11, 4.5), layout="constrained")
    ax[0].hist(top.bootstrap_top3_frequency, bins=np.linspace(0, 1, 11), color="#286eaa", edgecolor="white")
    ax[0].set(xlabel="Bootstrap Top-3 inclusion frequency", ylabel="Number of selected factors", title="Stability
of 150 selected factors")
    cols = ["graph_vs_ridge_jaccard", "graph_vs_spearman_jaccard", "ridge_vs_spearman_jaccard"]
    ax[1].boxplot([jaccard[c] for c in cols], tick_labels=["Graph / ridge", "Graph / rank corr.", "Ridge / rank
corr."])
    ax[1].set(ylabel="Top-3 Jaccard", ylim=(-.05, 1.05), title="Agreement across methods")
    fig.savefig(figures / "q2_stability.png", dpi=170)
    plt.close(fig)
    portraits = pd.read_csv(out / "q3/generated_sensory_continuous.csv")
    selected = ["A_malt_all", "A_hops_all", "A_esters_all", "F_malt_all", "F_hops_all", "F_esters_all", "acidity", "
bitternes", "sweetness", "body", "diacetyl", "dms", "stale_hops"]
    values = portraits.groupby("style")[selected].mean().reindex(cfg["generation_styles"])
    fig, ax = plt.subplots(figsize=(12, 3.4), layout="constrained")
    cap = max(.5, float(np.max(np.abs(values.to_numpy()))))
    im = ax.imshow(values, aspect="auto", cmap="RdBu_r", vmin=-cap, vmax=cap)
    ax.set_xticks(range(len(selected)), selected, rotation=45, ha="right")
    ax.set_yticks(range(len(values)), values.index)
    ax.set_title("Generated sensory portraits: model predictions on supplied continuous scale")
    fig.colorbar(im, ax=ax, label="Predicted continuous intensity", shrink=.85)
    fig.savefig(figures / "q3_sensory_portraits.png", dpi=170)
    plt.close(fig)
    fig, ax = plt.subplots(1, 2, figsize=(11, 4.5), layout="constrained")
    x = np.arange(len(transfers))
    ax[0].plot(x, transfers.source_target_probability, "o--", color="#999999", label="Source")
    ax[0].plot(x, transfers.candidate_target_probability, "o-", color="#286eaa", label="Candidate")
    ax[0].set_xticks(x, transfers.beer.astype(str), rotation=45)
    ax[0].set(xlabel="Source beer", ylabel="Tripel probability (model only)", title="Candidate style movement")
    ax[0].legend()
    groups = checks.groupby("defect").nonincrease.agg(["sum", "count"])
    ax[1].barh(groups.index, groups["sum"] / groups["count"], color="#549778")
    ax[1].set(xlim=(0, 1.05), xlabel="Nonincrease fraction in checked quantity", title="Chemical measurements and
stale-hops proxy")
    ax[1].text(.02, -.2, "Unchanged failures are included; actual tasting unverified", transform=ax[1].transAxes,
fontsize=8)
    fig.savefig(figures / "q3_transfer_checks.png", dpi=170)
    plt.close(fig)
    facts = json.loads((out / "audit/verified_facts.json").read_text(encoding="utf-8"))
    q3status = json.loads((out / "q3/status.json").read_text(encoding="utf-8"))
    successful = int((transfers.status == "nontrivial_model_feasible_candidate").sum())
    heldout = transfers[transfers.generator_role == "holdout"]
    heldout_success = int((heldout.status == "nontrivial_model_feasible_candidate").sum())
    baseline = float(metrics.loc["mean", "mae"])
    ridge_mae = float(metrics.loc["ridge", "mae"])
    improvement = (baseline - ridge_mae) / baseline
    low = int((top.bootstrap_top3_frequency < .5).sum())
    report = f"""# A 题阶段成果报告

本阶段为 AI 辅助研究基线，三问均有可运行实现和阶段产物。数值只对应当前配置与当前附件。原始 0–6 分标定、真实感官改善及化学因果仍未完成验证，不能将整份报告直接作为最终参赛结论。

## 已核实的数据事实

实际有 {facts['n_samples']} 个样本、{facts['n_features']} 个化学或理化数值字段、{facts['n_targets']} 个连续感官目标和 {facts['n_styles']} 种风格。以 beer 唯一键一对一关联，不使用受损 beer_id。附件相关矩阵与重算 Spearman 的平均绝对差为 {facts['spearman_matrix_mean_absolute_difference']:.6g}，与 Pearson 为 {facts['pearson_matrix_mean_absolute_difference']:.6g}。逐字段变换证据、模板检查、哈希和字典保存在 audit/。这些是文件证据，关于合理性的解释仍需队内复核。

## 第一问 处理 方法与结果

输入为附件 1 的数值字段，目标为附件 4 的 50 个连续感官字段。样本编号、条码和风格标签均不作为化合物预测特征。训练折内拟合 1%–99% 截尾、缺失中位数与标准化。采用按风格分层的 {cfg['outer_folds']} 折、{cfg['outer_repeats']} 次重复外层验证，岭回归和 PLS 的超参数在外层训练数据内再做 {cfg['inner_folds']} 折选择。每个样本的重复留出预测取平均。预先指定岭回归作为下游参考，未根据外层成绩选择冠军再声称独立测试性能。

| 方法 | 宏平均 MAE | 宏平均 RMSE | 宏平均 R² |
|---|---:|---:|---:|
"""
    for name in order:
        v = metrics.loc[name]
        report += f"| {name} | {v.mae:.6f} | {v.rmse:.6f} | {v.r2:.6f} |\n"
    report += f"""
岭回归相对于训练均值参照的宏平均 MAE 变化为 {improvement:.2%} 的下降。该结果衡量给定连续标签的插值预测，不能当作真实 0–6 分精度，也不证明对全新风格的迁移能力。逐维误差、各风格误差、折分配和附加缺失压力实验均已保存。图中的折间标准差不是置信区间。
```

`ASSUMPTION_percentile_grades_0_6.csv` 采用训练折目标的七等频经验分位边界，只是有明确假设的参考等级。它不能恢复“未感知”等绝对评分语义，集中于同一值的目标也会出现重复分位边界。绝对标定仍为 UNRESOLVED。最终模型的全数据拟合系数仅供复现与下游画像，不用于报告样本外精度。

![第一问验证](figures/q1_validation.png)

第二问 处理 方法与结果

按全矩阵秩相关一致性核对附件 1 与 2 的变量别名。网络边保留绝对相关大于等于 {cfg['graph_threshold']} 的关系，保存符号与绝对权重。用符号拉普拉斯约束岭回归，求解 $(Z^{T}Z + \alpha I + \lambda L)B = Z^{T}(Y - \text{均值})$。L 的非对角元素为负的有符号相关权，度数取绝对权和，因而为半正定。固定 α={cfg['graph_ridge_alpha']}、λ={cfg['graph_ridge_lambda']}；这是探索参数，尚未证明最优。

Top-3 仅从明确的化合物或元素字段中选择，排除 pH、色度、综合总量及混合指标。每个目标给出 3 项，共 {len(top)} 条，附方向、秩相关、网络模块、来源元数据及人工机理核查字段。{cfg['bootstrap_repeats']} 次样本 bootstrap 中重新拟合预处理和网络，再计算入选频率。与无图岭回归、边际秩相关 Top-3 比较 Jaccard。

当前选定因素中，入选频率低于 0.5 的有 {low}/{len(top)} 条，提示单个因素排序存在明显不稳定性。Graph/ridge 的平均 Jaccard 为 {jaccard.graph_vs_ridge_jaccard.mean():.4f}，Graph/Spearman 为 {jaccard.graph_vs_spearman_jaccard.mean():.4f}。这些差异应保留，不宜只展示一致的维度。来源与缺陷描述来自附件元数据，每个目标的具体化学机理论证仍需队员查阅原始文献并复核。全数据关联分析不构成独立预测验证或因果识别。

较高频候选包括 4VG 感官目标与 X4VG、金属味与 iron、烘烤麦芽感官与 methylpyrazine；它们仍只是当前重采样设计下的候选。某些目标排名出现反直觉化合物，说明模型和化学论证需要继续改进，不能把完整 Top-3 表当作已经确定的机理清单。

![第二问稳定性](figures/q2_stability.png)

第三问 处理 方法与结果

分层留出 20% 样本作为生成模型诊断参考。训练数据上拟合预处理和 {cfg['generation_components']} 维 PCA；各风格采用独立均值，协方差按 n/(n+10) 与全体训练协方差部分汇聚。采样后还原到附件特征尺度并限制在训练 1%–99% 范围，生成每种指定风格 {cfg['generation_samples_per_style']} 组化合物或理化描述符。全数据拟合的第一问参考模型输出感官画像，属于模型推断。

风格分类器在生成模型留出集上的平衡准确率为 {q3status['style_classifier_holdout_balanced_accuracy']:.4f}。该分类器与生成模型共享训练资料，并不是外部独立鉴定；其保留样本表现与生成物风格概率仅作模型诊断。Kriek、Lambic 等风格的真实留出样本很少，KS/MMD 指标无法证明完整分布已被可靠学会。KS 仅比较各字段实际非缺失的留出观测，字段数量另列；MMD 使用训练预处理后的表示。详见 generation_diagnostics.csv，并保留真实训练样本重采样的参照 MMD。

跨风格迁移先在潜空间施加风格均值差，再搜索连续插值候选。DMS、双乙酰、4VG 和 trans-2-nonenal 的附件字段逐项不得增加；Stale hops 没有直接化合物观测，以第一问的预测值不增加作为单独代理约束。候选还需使固定分类器的 Tripel 概率相对于原始样本以及仅做截尾的对照均增加超过 0.01，避免把异常清理效果混为风格迁移。未找到满足条件的非零改变候选时，保留原样并标为未找到，不计成功。

当前共有 {len(transfers)} 个 Pils/Lager 来源样本，找到非平凡模型内可行候选 {successful} 个；其中生成模型留出来源样本为 {len(heldout)} 个，找到候选 {heldout_success} 个。任何缺陷源值缺失、越出训练经验边界或仅能依赖预测代理的情况均在 transfer_diagnostics.csv 标注。所有字段检查保存在 defect_nonincrease_checks.csv。即使这些检查全部通过，也只能说明被检查的测量量或模型代理满足条件，不能声称实际品评无恶化。

![第三问感官画像](figures/q3_sensory_portraits.png)

![第三问迁移检查](figures/q3_transfer_checks.png)

保留结果与下一阶段

可保留：原件哈希和样本键审计、逐字段尺度证据、既定验证下的误差表、Top-3 与重采样频率、生成样本与诊断、逐项迁移约束核查。它们均绑定 config.json 与运行环境。

需要继续研究：真实评分标定锚点；未确定尺度与异常值的来源；目标相关的化学解释；网络与生成参数敏感性；新的独立验证；聚合指标的物理一致性；完整阈值和真实感官实验。原始研究或其他项目中的已发表成绩不属于本次实测结果。所有采纳的 AI 辅助成果须依赛事要求披露并由队内独立验证。

方法详见 docs/，来源详见 docs/SOURCES.md，运行日志详见 docs/RESEARCH_LOG.md。
"""
    (out / "STAGE_REPORT.md").write_text(report, encoding="utf-8")
    checksums = [{"file": str(p.relative_to(out)).replace("\\", "/"), "sha256": sha256(p)} for p in sorted(out.rglob("*")) if p.is_file() and "private" not in p.parts and p.suffix in {".csv", ".json", ".md", ".png"} and p.name not in ["result_checksums.json", "run_manifest.json"]]
    write_json(out / "result_checksums.json", checksums)

源程序 9 src/stage2.py

```
"""AI-assisted extension: OpenAI Codex / GPT-6, 2026-10-24.
Scientific limitations and all failed experiments must remain visible.
"""
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from scipy.stats import ks_2samp, norm
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import balanced_accuracy_score, mean_absolute_error
from sklearn.model_selection import RepeatedStratifiedKFold, train_test_split
from sklearn.multioutput import MultiOutputRegressor
from sklearn.pipeline import Pipeline

from .audit import run_audit
from .common import ecdf_grades, preprocessing, progress, save_csv, sha256, write_json
from .data import CHEMICAL_DEFECTS
from .q1 import fitted_model, metric_rows
from .q3 import ConditionalPCAGaussian, biased_mmd, chemical_nonincrease


def conformal_quantile(errors, alpha):
    n = len(errors)
    rank = int(np.ceil((n + 1) * (1 - alpha)))
    if rank > n:
        raise ValueError("Calibration set too small for requested coverage")
    return np.sort(np.asarray(errors), axis=0)[rank - 1]


def assumption_grades(reference, pred):
    reference, pred = np.asarray(reference), np.asarray(pred)
    empirical = ecdf_grades(reference, pred)
    mu, sd = reference.mean(0), np.maximum(reference.std(0), 1e-8)
    gaussian = np.minimum(6, np.floor(7 * norm.cdf((pred - mu) / sd))).astype(int)
    low, high = np.quantile(reference, [.05, .95], axis=0)
    linear = np.rint(6 * np.clip((pred - low) / np.maximum(high - low, 1e-8), 0, 1)).astype(int)
    return np.stack([empirical, gaussian, linear])


def local_convex_samples(real, n, rng):
    """Conditional empirical local mixing; not a physical composition law."""
    real = np.asarray(real)
    scale = np.maximum(np.std(real, axis=0), 1e-8)
    dist = cdist(real / scale, real / scale)
    np.fill_diagonal(dist, np.inf)
    neighbors = np.argsort(dist, axis=1)[:, :min(3, len(real) - 1)]
    out = []
    for _ in range(n):
        anchor = rng.integers(len(real))
        partner = rng.choice(neighbors[anchor]) if len(real) > 1 else anchor
        weight = rng.uniform(.1, .9)
        out.append(weight * real[anchor] + (1 - weight) * real[partner])
    return np.asarray(out)


def run_extension(data, base, out, cfg, ext):
    manifest = pd.read_csv(base / "audit/source_manifest.csv")
    for row in manifest.itertuples():
        if sha256(data.root / row.file) != row.sha256:
            raise ValueError("Source version differs from stage one")
    run_audit(data, out, cfg)
    X, Y = data.X.to_numpy(), data.Y.to_numpy()
    labels = data.meta.tasting_category_fine.to_numpy()
    targets, features = data.Y.columns.tolist(), data.X.columns.tolist()
    for sub in ["q1", "q2", "q3"]:
        (out / sub).mkdir(exist_ok=True)
    splits = list(RepeatedStratifiedKFold(n_splits=3, n_repeats=2, random_state=cfg["seed"]).split(X, labels))
    sums = np.zeros_like(Y)
    counts = np.zeros(len(Y))
    permutation_sums = np.zeros((X.shape[1], Y.shape[1]))
    positive_folds = np.zeros_like(permutation_sums)
    grade_min, grade_max = np.full_like(Y, 6), np.zeros_like(Y)
    grade_disagreement, assignments = [], []
    for fold, (train, test) in enumerate(splits):
        progress(f"Stage 2 Q1/Q2: held-out fold {fold + 1}/6")
        model = Pipeline([("prep", preprocessing()), ("model",
MultiOutputRegressor(GradientBoostingRegressor(n_estimators=ext["gbr_estimators"],
learning_rate=ext["gbr_learning_rate"], max_depth=ext["gbr_depth"], min_samples_leaf=5, max_features=.7,
random_state=cfg["seed"]), n_jobs=2))])
        # Threads avoid process resource-tracker failures on Unicode Windows temp paths.
        with joblib.parallel_backend("threading"):
            model.fit(X[train], Y[train])
            prediction = model.predict(X[test])
        sums[test] += prediction
        counts[test] += 1
        assignments.extend({"repeat": fold // 3, "fold": fold % 3, "beer": int(data.meta.iloc[i].beer)} for i in
test)
        ridge, _ = fitted_model("ridge", X[train], Y[train], cfg)
        ridge_pred = ridge.predict(X[test])
        grades = assumption_grades(Y[train], ridge_pred)
        grade_min[test] = np.minimum(grade_min[test], grades.min(axis=0))
```

```
        grade_max[test] = np.maximum(grade_max[test], grades.max(axis=0))
        grade_disagreement.append({"fold": fold, "any_rule_disagreement_fraction": float((grades.max(0) !=
grades.min(0)).mean()), "mean_grade_spread": float((grades.max(0) - grades.min(0)).mean())})
        # Exact linear predictor permutation. The trained preprocessing is frozen.
        Z = ridge.named_steps["prep"].transform(X[test])
        coefficients = ridge.named_steps["model"].coef_
        baseline_error = np.abs(Y[test] - ridge_pred).mean(axis=0)
        rng = np.random.default_rng(cfg["seed"] + fold)
        deltas = np.zeros_like(permutation_sums)
        for j in range(X.shape[1]):
            for _ in range(ext["permutation_repeats"]):
                permuted = Z[rng.permutation(len(Z)), j] - Z[:, j]
                pp = ridge_pred + permuted[:, None] * coefficients[:, j]
                deltas[j] += np.abs(Y[test] - pp).mean(axis=0) - baseline_error
        deltas /= ext["permutation_repeats"]
        permutation_sums += deltas
        positive_folds += deltas > 0
    gbr = sums / counts[:, None]
    existing = pd.read_csv(base / "q1/metrics_by_target.csv")
    additional = pd.DataFrame(metric_rows(Y, gbr, targets, "gbr_fixed"))
    ridge_oof = pd.read_csv(base / "q1/oof_ridge_continuous.csv")[targets].to_numpy()
    tree_oof = pd.read_csv(base / "q1/oof_extra_trees_continuous.csv")[targets].to_numpy()
    blend = (ridge_oof + tree_oof) / 2
    additional = pd.concat([additional, pd.DataFrame(metric_rows(Y, blend, targets, "fixed_ridge_tree_blend"))],
ignore_index=True)
    all_metrics = pd.concat([existing, additional], ignore_index=True)
    save_csv(all_metrics, out / "q1/metrics_by_target.csv")
    save_csv(all_metrics.groupby("model")[["mae", "rmse", "r2"]].mean().reset_index(), out / "
q1/metrics_summary.csv")
    save_csv(pd.concat([data.meta[["beer"]], pd.DataFrame(gbr, columns=targets)], axis=1), out / "
q1/oof_gbr_continuous.csv")
    save_csv(pd.DataFrame(assignments), out / "q1/split_assignments.csv")
    save_csv(pd.DataFrame(grade_disagreement), out / "q1/grade_rule_sensitivity.csv")
    grade_ranges = pd.concat([data.meta[["beer"]], pd.DataFrame(grade_min.astype(int), columns=[c + "__min" for c
in targets]), pd.DataFrame(grade_max.astype(int), columns=[c + "__max" for c in targets])], axis=1)
    grade_ranges["status"] = "range_over_assumed_rules_and_CV_fits_not_a_confidence_interval"
    save_csv(grade_ranges, out / "q1/ASSUMPTION_grade_ranges.csv")

    # Proper train / calibration / test split for one demonstrative conformal experiment.
    train, rest = train_test_split(np.arange(len(X)), test_size=.4, random_state=cfg["seed"] + 71)
    calibration, test = train_test_split(rest, test_size=.5, random_state=cfg["seed"] + 72)
    ridge, _ = fitted_model("ridge", X[train], Y[train], cfg)
    q = conformal_quantile(np.abs(Y[calibration] - ridge.predict(X[calibration])), ext["conformal_alpha"])
    pp = ridge.predict(X[test])
    coverage = (np.abs(Y[test] - pp) <= q).mean(axis=0)
    save_csv(pd.DataFrame({"target": targets, "nominal_marginal_coverage": .9, "empirical_test_coverage": coverage, "
interval_width": 2 * q, "calibration_n": len(calibration), "test_n": len(test)}), out / "q1/conformal_coverage.csv")
    save_csv(pd.DataFrame({"beer": data.meta.beer, "role": ["train" if i in set(train) else "calibration" if i in
set(calibration) else "test" for i in range(len(X))]}), out / "q1/conformal_split.csv")

    progress("Stage 2 Q2: assembling held-out attribution evidence")
    importances = permutation_sums / len(splits)
    rows = [{"target": target, "field": field, "heldout_permutation_MAE_increase": importances[j, k], "
positive_fold_fraction": positive_folds[j, k] / len(splits)} for k, target in enumerate(targets) for j, field in
enumerate(features)]
    importance_frame = pd.DataFrame(rows)
    save_csv(importance_frame, out / "q2/heldout_permutation_importance.csv")
    top = pd.read_csv(base / "q2/top3_by_sensory_dimension.csv").merge(importance_frame, on=["target", "field"],
validate="one_to_one")
    top["evidence_status"] = np.where((top.bootstrap_top3_frequency >= .5) & (top.heldout_permutation_MAE_increase
> 0) & (top.positive_fold_fraction >= 2 / 3), "comparatively_supported_association", "weak_or_unstable_association")
    top["causality"] = "not_established"
    save_csv(top, out / "q2/top3_evidence_review.csv")

    progress("Stage 2 Q3: comparing conditional generators across held-out splits")
    rng = np.random.default_rng(cfg["seed"])
    results = []
    final_generator, final_classifier, generated_final = None, None, []
    first_train = None
    for repeat in range(ext["generation_repeats"]):
        train, test = train_test_split(np.arange(len(X)), test_size=.2, stratify=labels, random_state=cfg["seed"] +
repeat)
        generator = ConditionalPCAGaussian(10, cfg["seed"]).fit(X[train], labels[train])
        clf = Pipeline([("prep", preprocessing()), ("model", LogisticRegression(C=1, max_iter=2000,
class_weight="balanced", random_state=cfg["seed"]))]).fit(X[train], labels[train])
        acc = balanced_accuracy_score(labels[test], clf.predict(X[test]))
        for style in ext["generation_styles"]:
            real = X[test[labels[test] == style]]
            real_train = generator.prep.named_steps["scale"].inverse_transform(generator.prep.transform(X[train[labe
ls[train] == style]]))
            samples = {"bootstrap": real_train[rng.integers(0, len(real_train), ext["generation_n"])], "
local_convex": local_convex_samples(real_train, ext["generation_n"], rng), "PCA_Gaussian": generator.sample(style,
```

```
ext["generation_n"], rng)}
            for method, values in samples.items():
                Z = generator.prep.transform(values)
                reference = generator.prep.transform(real)
                distances = cdist(Z, generator.prep.transform(X[train]))
                ks = [ks_2samp(values[:, j], real[:, j][np.isfinite(real[:, j])]).statistic for j in
range(X.shape[1]) if np.isfinite(real[:, j]).any()]
                results.append({"repeat": repeat, "style": style, "method": method, "n_real_holdout": len(real), "
KS_mean": np.mean(ks), "MMD2": biased_mmd(Z, reference), "exact_training_copy_fraction": float((distances.min(1) <
1e-8).mean()), "target_style_probability": clf.predict_proba(values)[:, list(clf.classes_).index(style)].mean(), "
style_classifier_holdout_balanced_accuracy": acc})
                if repeat == 0 and method == "local_convex":
                    frame = pd.DataFrame(values, columns=features)
                    frame.insert(0, "style", style)
                    frame.insert(0, "generated_id", [f"LC_{style}_{j+1:03d}" for j in range(len(values))])
                    generated_final.append(frame)
        if repeat == 0:
            final_generator, final_classifier, first_train = generator, clf, train
    diagnostics = pd.DataFrame(results)
    save_csv(diagnostics, out / "q3/generator_comparison_by_split.csv")
    save_csv(diagnostics.groupby(["method", "style"])[["KS_mean", "MMD2", "exact_training_copy_fraction", "
target_style_probability"]].mean().reset_index(), out / "q3/generator_comparison_mean.csv")
    generated = pd.concat(generated_final, ignore_index=True)
    save_csv(generated, out / "q3/local_convex_generated_profiles.csv")
    bundle = joblib.load(base / "q1/private/model.joblib")
    predictor = bundle["model"]
    portraits = pd.concat([generated[["generated_id", "style"]],
pd.DataFrame(predictor.predict(generated[features].to_numpy()), columns=targets)], axis=1)
    save_csv(portraits, out / "q3/local_convex_sensory_continuous.csv")
    transfer_results = expanded_transfer(data, final_generator, final_classifier, first_train, predictor, ext)
    save_csv(transfer_results[0], out / "q3/expanded_transfer_diagnostics.csv")
    save_csv(transfer_results[1], out / "q3/expanded_transfer_profiles.csv")
    save_csv(transfer_results[2], out / "q3/expanded_transfer_defect_checks.csv")
    write_json(out / "status.json", {
        "absolute_score_calibration": "not_identifiable_from_provided_aggregated_per_taster_standardized_scores_with
out_additional_anchors",
        "grade_sensitivity_disagreement": float(pd.DataFrame(grade_disagreement).any_rule_disagreement_fraction.mean
()),
        "conformal_nominal_marginal_coverage": .9, "conformal_empirical_average_coverage": float(coverage.mean()),
        "conformal_caveat": "single random split; marginal coverage on provided continuous labels, not simultaneous
or original-scale coverage",
        "comparatively_supported_top3_associations": int((top.evidence_status == "
comparatively_supported_association").sum()),
        "top3_total": len(top), "generation_repeated_splits": ext["generation_repeats"],
        "expanded_transfer_nontrivial": int((transfer_results[0].status == "
nontrivial_model_feasible_candidate").sum()),
        "physical_or_tasting_validation": "not_performed", "research_stage": "
exploratory_extension_after_stage_one_not_an_untouched_external_test",
    })


def expanded_transfer(data, generator, classifier, train, predictor, ext):
    X = data.X.to_numpy()
    features, targets = data.X.columns.tolist(), data.Y.columns.tolist()
    labels = data.meta.tasting_category_fine.to_numpy()
    positions = [features.index(c) for c in CHEMICAL_DEFECTS]
    stale = targets.index("stale_hops")
    target_train = X[train[labels[train] == "Tripel"]]
    target_train = generator.prep.named_steps["scale"].inverse_transform(generator.prep.transform(target_train))
    target_class = list(classifier.classes_).index("Tripel")
    diagnostics, profiles, checks = [], [], []
    for i in np.flatnonzero(labels == "Pils/Lager"):
        x = np.where(np.isfinite(X[i]), X[i], generator.median)
        source_pred = predictor.predict(x[None])[0]
        base_prob = classifier.predict_proba(x[None])[0, target_class]
        cleaned = np.clip(x, generator.low, generator.high)
        cleaned[positions] = np.minimum(cleaned[positions], x[positions])
        clean_prob = classifier.predict_proba(cleaned[None])[0, target_class]
        candidates = []
        for target in target_train:
            for weight in np.linspace(.025, 1, ext["transfer_weights"]):
                value = np.clip(x + weight * (target - x), generator.low, generator.high)
                value[positions] = np.minimum(value[positions], x[positions])
                candidates.append(value)
        candidates = np.asarray(candidates)
        pp = predictor.predict(candidates)
        probabilities = classifier.predict_proba(candidates)[:, target_class]
        valid = chemical_nonincrease(x, candidates, positions) & (pp[:, stale] <= source_pred[stale] + 1e-9) &
(probabilities > max(base_prob, clean_prob) + .01)
        if valid.any():
            ids = np.flatnonzero(valid)
            j = ids[np.argmax(probabilities[ids])]
```

```
            value, portrait, prob, status = candidates[j], pp[j], probabilities[j], "
nontrivial_model_feasible_candidate"
        else:
            value, portrait, prob, status = x, source_pred, base_prob, "
no_nontrivial_candidate_found_source_retained"
        beer = int(data.meta.iloc[i].beer)
        missing = [c for c, j in zip(CHEMICAL_DEFECTS, positions) if not np.isfinite(X[i, j])]
        diagnostics.append({"beer": beer, "status": status, "generator_role": "train" if i in set(train) else "
holdout", "source_trip_probability": base_prob, "candidate_trip_probability": prob, "cleaning_only_probability":
clean_prob, "change_l2": np.linalg.norm(value - x), "missing_source_defects": ",".join(missing), "
all_source_defects_observed": not missing, "stale_constraint": "predicted_proxy_only"})
        profiles.append({"beer": beer, **dict(zip(features, value))})
        for name, j in zip(CHEMICAL_DEFECTS, positions):
            checks.append({"beer": beer, "defect": name, "source": x[j], "candidate": value[j], "nonincrease":
bool(value[j] <= x[j] + 1e-9), "source_observed": bool(np.isfinite(X[i, j])), "scale": "provided_measurement"})
        checks.append({"beer": beer, "defect": "stale_hops", "source": source_pred[stale], "candidate":
portrait[stale], "nonincrease": bool(portrait[stale] <= source_pred[stale] + 1e-9), "source_observed": False, "scale": "
learned_sensory_proxy"})
    return pd.DataFrame(diagnostics), pd.DataFrame(profiles), pd.DataFrame(checks)
```

源程序 10 src/stage3_algorithms.py

```
"""Stage-three algorithm prototypes; AI-assisted, OpenAI Codex, 2026-10-25.

These helpers validate mathematical properties. They are not evidence that the
proposed full study improves real prediction or chemical feasibility.
"""
import numpy as np


def fixed_grade_boundaries(training_targets):
    """Fit once on training labels; never refit with calibration/test labels."""
    y = np.asarray(training_targets, float)
    if y.ndim != 2 or not np.isfinite(y).all() or len(y) < 2:
        raise ValueError("Expected at least two finite training rows")
    return np.quantile(y, np.arange(1, 7) / 7, axis=0)


def grade_with_fixed_boundaries(values, boundaries):
    values, boundaries = np.asarray(values, float), np.asarray(boundaries, float)
    if values.ndim != 2 or boundaries.shape != (6, values.shape[1]):
        raise ValueError("Grade dimensions do not agree")
    if not np.isfinite(values).all() or not np.isfinite(boundaries).all() or np.any(np.diff(boundaries, axis=0) <
0):
        raise ValueError("Need finite values and ordered boundaries")
    return (values[:, None, :] > boundaries[None, :, :]).sum(axis=1).astype(int)


def ordinal_interval_image(lower, upper, boundaries):
    """Conservative image of a continuous interval under a fixed monotone map.

    Coverage, if applicable, is for this operationally defined grade only.
    Tied boundaries can make the interval hull include unreachable categories.
    """
    lower, upper = np.asarray(lower, float), np.asarray(upper, float)
    if lower.shape != upper.shape or np.any(lower > upper):
        raise ValueError("Invalid continuous intervals")
    return grade_with_fixed_boundaries(lower, boundaries), grade_with_fixed_boundaries(upper, boundaries)


def shrunk_blend_weights(y_inner_oof, et_inner_oof, gbr_inner_oof, penalty):
    """Fit using inner OOF predictions only; choose penalty within inner training.

    w[j] multiplies GBR and (1-w[j]) multiplies ExtraTrees. The closed-form
    objective is mean squared error plus penalty*(w[j]-global_weight)**2.
    """
    y, et, gbr = [np.asarray(a, float) for a in (y_inner_oof, et_inner_oof, gbr_inner_oof)]
    if y.ndim != 2 or et.shape != y.shape or gbr.shape != y.shape or penalty < 0:
        raise ValueError("Invalid blend inputs")
    if not all(np.isfinite(a).all() for a in (y, et, gbr)):
        raise ValueError("Blend arrays must be finite")
    diff, residual = gbr - et, y - et
    denom = np.mean(diff * diff)
    global_weight = float(np.clip(np.mean(diff * residual) / denom, 0, 1)) if denom > 1e-14 else 0.5
    local_denom = np.mean(diff * diff, axis=0) + penalty
    numerator = np.mean(diff * residual, axis=0) + penalty * global_weight
    weights = np.divide(numerator, local_denom, out=np.full(y.shape[1], global_weight), where=local_denom > 1e-14)
    return np.clip(weights, 0, 1), global_weight


def paired_oof_bootstrap(y, pred_a, pred_b, strata, repeats=2000, seed=20261025):
```

```
    """Stratified beer-level resampling of fixed predictions; NOT a retraining CI.

    Inputs must have one row per unique beer, with repeats already averaged.
    Negative differences favour A. Macro MAE is average over rows and targets.
    """
    y, a, b = [np.asarray(x, float) for x in (y, pred_a, pred_b)]
    labels = np.asarray(strata)
    if y.ndim != 2 or a.shape != y.shape or b.shape != y.shape or len(labels) != len(y):
        raise ValueError("Paired dimensions differ")
    if repeats < 1 or not all(np.isfinite(x).all() for x in (y, a, b)):
        raise ValueError("Need finite paired predictions and positive repeats")
    per_beer = (np.abs(y-a)-np.abs(y-b)).mean(axis=1)
    groups = [np.flatnonzero(labels == label) for label in np.unique(labels)]
    rng = np.random.default_rng(seed)
    draws = np.array([np.mean(per_beer[np.concatenate([rng.choice(g, len(g), replace=True) for g in groups])]) for
_ in range(repeats)])
    return {"difference_A_minus_B": float(per_beer.mean()), "conditional_percentile_95": np.quantile(draws, [.025, .
975]).tolist(),
            "scope": "conditional_on_fixed_OOF_predictions_does_not_include_model_retraining_or_selection_uncertaint
y",
            "n_unique_beers": len(y), "resamples": repeats}


def pareto_minimize(objectives):
    """Keep non-dominated finite candidates; no weighting can hide a violation."""
    values = np.asarray(objectives, float)
    if values.ndim != 2:
        raise ValueError("Expected candidates by objectives")
    keep = np.isfinite(values).all(axis=1)
    for i in np.flatnonzero(keep):
        dominated = np.all(values <= values[i], axis=1) & np.any(values < values[i], axis=1) &
np.isfinite(values).all(axis=1)
        if dominated.any():
            keep[i] = False
    return keep


def threshold_excess(concentration, threshold):
    """Use only after unit and scale verification; invalid inputs stay unknown."""
    concentration, threshold = np.broadcast_arrays(np.asarray(concentration, float), np.asarray(threshold, float))
    valid = np.isfinite(concentration) & np.isfinite(threshold) & (concentration >= 0) & (threshold > 0)
    out = np.full(concentration.shape, np.nan)
    out[valid] = np.maximum(concentration[valid] / threshold[valid] - 1, 0)
    return out
```

源程序 11 src/stage3_generation.py

```
"""Disjoint-search/evaluation conditional generation and Pareto transfer.
AI-assisted reference implementation; no actual brewing or tasting validation.
"""
from pathlib import Path
import hashlib
import time
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from scipy.stats import ks_2samp
from sklearn.ensemble import ExtraTreesClassifier, ExtraTreesRegressor
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.model_selection import train_test_split
from sklearn.metrics import balanced_accuracy_score
from sklearn.pipeline import Pipeline
from .common import preprocessing, save_csv, write_json, progress
from .data import AGGREGATES, CHEMICAL_DEFECTS
from .q3 import ConditionalPCAGaussian
from .stage2 import local_convex_samples
from .stage3_algorithms import pareto_minimize, threshold_excess


def mmd_fixed_bandwidth(a,b,bandwidth):
    if len(a)<2 or len(b)<2: return np.nan
    return float(np.exp(-cdist(a,a,'sqeuclidean')/(2*bandwidth)).mean()+np.exp(-cdist(b,b,'sqeuclidean')/(2*bandwidt
h)).mean()-2*np.exp(-cdist(a,b,'sqeuclidean')/(2*bandwidth)).mean())


def loo_support_radius(z):
    if len(z)<2: return np.nan
    distance=cdist(z,z); np.fill_diagonal(distance,np.inf)
    return float(np.quantile(distance.min(1),.95))
```

```
def constrained_convex(real,generator,style_positions,n,rng):
    Z=generator.prep.transform(real); distance=cdist(Z,Z); np.fill_diagonal(distance,np.inf)
    neighbors=np.argsort(distance,axis=1)[:,:min(3,len(real)-1)]; radius=loo_support_radius(Z); result=[]; attempts=0
    if len(real)<2: return real[np.zeros(n,dtype=int)].copy(),{'attempts':n,'singleton_fallback':True,'support_radius':None}
    while len(result)<n and attempts<n*250:
        attempts+=1; a=rng.integers(len(real)); b=rng.choice(neighbors[a]); weight=rng.uniform(.1,.9)
        value=weight*real[a]+(1-weight)*real[b]
        for j in style_positions:
            value[j]=np.log10(weight*10**real[a,j]+(1-weight)*10**real[b,j])
        value=np.clip(value,generator.low,generator.high)
        if cdist(generator.prep.transform(value[None]),Z).min()<=radius+1e-9: result.append(value)
    return np.asarray(result).reshape((-1,real.shape[1])),{'attempts':attempts,'singleton_fallback':False,'support_radius':radius}


def locked_transfer_candidates(X,labels,beers,G,generator,searcher,sensory_a,consensus_a,features,targets,cfg):
    positions=[features.index(t) for t in CHEMICAL_DEFECTS]; logpos=[features.index(t) for t in ['DMS','X4VG']]
    target_class=list(searcher.classes_).index('Tripel'); stale=targets.index('stale_hops')
    target_data=generator.prep.named_steps['scale'].inverse_transform(generator.prep.transform(X[G[labels[G]=='Tripel']]))
    target_Z=generator.prep.transform(target_data); support=loo_support_radius(target_Z)
    result=[]
    for i in np.flatnonzero(labels=='Pils/Lager'):
        source=np.where(np.isfinite(X[i]),X[i],generator.median); sourceZ=generator.prep.transform(source[None])
        source_p=searcher.predict_proba(source[None])[0,target_class];
stale_a=sensory_a.predict(source[None])[0,stale]; stale_c=consensus_a.predict(source[None])[0,stale]
        cleaned=np.clip(source,generator.low,generator.high);
cleaned[positions]=np.minimum(cleaned[positions],source[positions]);
clean_p=searcher.predict_proba(cleaned[None])[0,target_class]
        values=[]
        for anchor in target_data:
            for weight in np.linspace(.025,1,cfg['transfer_weights']):
                v=np.clip(source+weight*(anchor-source),generator.low,generator.high);
v[positions]=np.minimum(v[positions],source[positions]); values.append(v)
        values=np.asarray(values); Z=generator.prep.transform(values);
changes=np.linalg.norm(Z-sourceZ,axis=1)/np.sqrt(len(features)); support_distance=cdist(Z,target_Z).min(1)
        probabilities=searcher.predict_proba(values)[:,target_class]; pa=sensory_a.predict(values);
pc=consensus_a.predict(values)
        bounds=((values>=generator.low-1e-9)&(values<=generator.high+1e-9)).all(1)
        nonincrease=(values[:,positions]<=source[positions]+1e-9).all(1)
        known_proxy_risk=np.maximum(threshold_excess(10**values[:,logpos[0]],19.),threshold_excess(10**values[:,logpos[1]],.2))
        valid=bounds&nonincrease&(pa[:,stale]<=stale_a+1e-9)&(pc[:,stale]<=stale_c+1e-9)&(probabilities>=max(cfg['transfer_target_probability_floor'],source_p+.01,clean_p+.01))&(support_distance<=support+1e-9)&(changes>1e-8)
        for radius in cfg['trust_radii']:
            ids=np.flatnonzero(valid&(changes<=radius)); pareto_ids=[]
            if len(ids):
                nd=pareto_minimize(np.c_[changes[ids],support_distance[ids],known_proxy_risk[ids]]);
pareto_ids=ids[nd]; j=int(pareto_ids[np.argmin(changes[pareto_ids])]); value=values[j];
status='model_feasible_nontrivial' if np.isfinite(X[i,positions]).all() else 'conditional_on_imputed_source_defect'
                p=float(probabilities[j]); change=float(changes[j]); target_distance=float(support_distance[j])
            else:
                value=source.copy(); status='no_feasible_nontrivial_candidate_source_retained'; p=float(source_p);
change=0.; target_distance=float(cdist(sourceZ,target_Z).min())
            result.append({'row':int(i),'beer':int(beers[i]),'radius':radius,'source':source.copy(),'candidate':value.copy(),'status':status,'n_candidates':len(values),'n_valid':len(ids),'n_pareto':len(pareto_ids),
                'search_source_probability':float(source_p),'search_candidate_probability':p,'standardized_change':change,'target_support_distance':target_distance,
               'all_provided_source_defects_observed':bool(np.isfinite(X[i,positions]).all()),'missing_defects':','.join(features[j] for j in positions if not np.isfinite(X[i,j]))})
    return result


def run_generation(data,out,cfg):
    start=time.perf_counter(); dest=Path(out)/'q3'; dest.mkdir(parents=True,exist_ok=True)
    features=[f for f in data.X if f not in AGGREGATES]; targets=list(data.Y); X=data.X[features].to_numpy();
Y=data.Y.to_numpy(); labels=data.meta.tasting_category_fine.to_numpy(); beers=data.meta.beer.to_numpy()
    core=[j for j,t in enumerate(targets) if t.startswith(('A_','F_'))]; log_positions=[features.index(f) for f in ['DMS','X4VG']]; stale=targets.index('stale_hops'); positions=[features.index(f) for f in CHEMICAL_DEFECTS]
    metrics=[]; transfers=[]; checks=[]; roles=[]; evaluator_rows=[]; locked=[]; profiles=[]; portraits=[];
generated_profiles=[]
    for repeat in range(cfg['generation_repeats']):
        seed=cfg['seed']+repeat; rng=np.random.default_rng(seed); progress(f'S3 Q3 disjoint partition {repeat+1}/5')
        dev,T=train_test_split(np.arange(len(X)),test_size=50,stratify=labels,random_state=seed)
        G,J=train_test_split(dev,test_size=100,stratify=labels[dev],random_state=seed+999)
        assert len(G)==len(J)==100 and len(T)==50 and not(set(G)&set(J) or set(G)&set(T) or set(J)&set(T))
        roles.extend({'repeat':repeat,'beer':int(beers[i]),'role':'G_search_training' if i in set(G) else '
J_evaluator_training' if i in set(J) else 'T_real_holdout'} for i in range(len(X)))
        generator=ConditionalPCAGaussian(10,seed).fit(X[G],labels[G])
```

```
        searcher=Pipeline([('prep',preprocessing()),('model',LogisticRegression(C=1,max_iter=2000,class_weight='balanced',random_state=seed))]).fit(X[G],labels[G])
        sensory_a=Pipeline([('prep',preprocessing()),('model',Ridge(alpha=100))]).fit(X[G],Y[G])
        consensus_a=Pipeline([('prep',preprocessing()),('model',ExtraTreesRegressor(n_estimators=200,min_samples_leaf=3,max_features=.8,random_state=seed,n_jobs=2))]).fit(X[G],Y[G])
        generated=[]
        for style in cfg['generation_styles']:
           real=generator.prep.named_steps['scale'].inverse_transform(generator.prep.transform(X[G[labels[G]==style]]))
            constrained,info=constrained_convex(real,generator,log_positions,cfg['generation_n'],rng)
            samples={'bootstrap':real[rng.integers(0,len(real),cfg['generation_n'])],'local_convex':local_convex_samples(real,cfg['generation_n'],rng),
                 'PCA_Gaussian':generator.sample(style,cfg['generation_n'],rng),'constrained_local_convex':constrained}
            for method,values in samples.items(): generated.append({'style':style,'method':method,'values':values,'training_n':len(real),'extra':info if method=='constrained_local_convex' else {}})
        selected=locked_transfer_candidates(X,labels,beers,G,generator,searcher,sensory_a,consensus_a,features,targets,cfg)
        digest=hashlib.sha256()
        for group in generated: digest.update(group['values'].tobytes())
        for row in selected: digest.update(row['candidate'].tobytes())
        lock={'repeat':repeat,'candidate_sha256':digest.hexdigest(),'selection_used_J_or_T':False,'n_generated':sum(len(a['values']) for a in generated),'n_transfer_records':len(selected)}
        locked.append(lock); write_json(dest/f'locked_candidates_{repeat}.json',lock)
        # J models are fitted only after candidates and their order are fixed.
        evaluator=Pipeline([('prep',preprocessing()),('model',ExtraTreesClassifier(n_estimators=200,min_samples_leaf=2,max_features=.7,class_weight='balanced',random_state=seed+3000,n_jobs=2))]).fit(X[J],labels[J])
        sensory_b=Pipeline([('prep',preprocessing()),('model',ExtraTreesRegressor(n_estimators=200,min_samples_leaf=3,max_features=.8,random_state=seed+4000,n_jobs=2))]).fit(X[J],Y[J])
        acc_a=balanced_accuracy_score(labels[T],searcher.predict(X[T])); acc_b=balanced_accuracy_score(labels[T],evaluator.predict(X[T])); y_a=sensory_a.predict(X[T]); y_b=sensory_b.predict(X[T])
        evaluator_rows.append({'repeat':repeat,'searcher_holdout_balanced_accuracy':acc_a,'evaluator_holdout_balanced_accuracy':acc_b,'search_sensory_core30_mae':np.abs(Y[T][:,core]-y_a[:,core]).mean(),'evaluator_sensory_core30_mae':np.abs(Y[T][:,core]-y_b[:,core]).mean()})
        allG=generator.prep.transform(X[G]); distance=cdist(allG,allG,'sqeuclidean'); bandwidth=float(np.median(distance[distance>0]))
        for group in generated:
            style,method,values=group['style'],group['method'],group['values']; test=X[T[labels[T]==style]]; Z=generator.prep.transform(values) if len(values) else np.empty((0,len(features)))
            if not len(values):
                metrics.append({'repeat':repeat,'style':style,'method':method,'n_generated':0,'n_G_style':group['training_n'],'n_T_style':len(test),'status':'generation_budget_exhausted_no_sample'}); continue
            testZ=generator.prep.transform(test); nn=cdist(Z,allG).min(1)
            prob_a=searcher.predict_proba(values)[:,list(searcher.classes_).index(style)]; prob_b=evaluator.predict_proba(values)[:,list(evaluator.classes_).index(style)]
            jstyle=X[J[labels[J]==style]]; jz=evaluator.named_steps['prep'].transform(jstyle); threshold=loo_support_radius(jz); distances_b=cdist(evaluator.named_steps['prep'].transform(values),jz).min(1)
            ks=[ks_2samp(values[:,j],test[:,j][np.isfinite(test[:,j])]).statistic for j in range(X.shape[1]) if np.isfinite(test[:,j]).any()]
            metrics.append({'repeat':repeat,'style':style,'method':method,'n_generated':len(values),'n_G_style':group['training_n'],'n_J_style':len(jstyle),'n_T_style':len(test),
                'MMD2_fixed_training_bandwidth':mmd_fixed_bandwidth(Z,testZ,bandwidth),'KS_mean':np.mean(ks),'exact_copy_fraction':(nn<1e-8).mean(),
                'search_probability':prob_a.mean(),'independent_evaluator_probability':prob_b.mean(),'independent_evaluator_target_prediction_fraction':np.mean(evaluator.predict(values)==style),
                'J_support_fraction':np.mean(distances_b<=threshold) if np.isfinite(threshold) else np.nan,
                'provided_bounds_violation_fraction':np.mean(~((values>=generator.low-1e-9)&(values<=generator.high+1e-9)).all(1)),
                'sensory_proxy_disagreement':np.abs(sensory_a.predict(values)-sensory_b.predict(values)).mean(),
               'bandwidth_from_G':bandwidth,'status':'statistical_profiles_not_actual_tasting','singleton_fallback':group['extra'].get('singleton_fallback',False)})
            if repeat==0:
                ids=[f'S3_{method}_{style}_{i+1:03d}' for i in range(len(values))]
                frame=pd.DataFrame(values,columns=features); frame.insert(0,'style',style); frame.insert(0,'method',method); frame.insert(0,'generated_id',ids); generated_profiles.append(frame)
                if method=='constrained_local_convex':
                    portrait=pd.DataFrame(sensory_a.predict(values),columns=targets); portrait.insert(0,'style',style); portrait.insert(0,'generated_id',ids); portraits.append(portrait)
        for row in selected:
            i=row['row']; source,candidate=row['source'],row['candidate']; pb0=evaluator.predict_proba(source[None])[0,list(evaluator.classes_).index('Tripel')]; pb1=evaluator.predict_proba(candidate[None])[0,list(evaluator.classes_).index('Tripel')]
            sb0=sensory_b.predict(source[None])[0,stale]; sb1=sensory_b.predict(candidate[None])[0,stale]
            nontrivial=row['status']!='no_feasible_nontrivial_candidate_source_retained'
            b_accept=nontrivial and pb1>pb0+.01 and evaluator.predict(candidate[None])[0]=='Tripel' and sb1<=sb0+1e-9
            record={k:v for k,v in row.items() if k not in ['row','source','candidate']}
            record.update(repeat=repeat,source_role='G_search_training' if i in set(G) else 'J_evaluator_training' if i in set(J) else 'T_real_holdout',
                evaluator_source_probability=pb0,evaluator_candidate_probability=pb1,evaluator_predicts_Tripel=bool(evaluator.predict(candidate[None])[0]=='Tripel'),
```

```
                evaluator_stale_source=sb0,evaluator_stale_candidate=sb1,evaluator_proxy_acceptance=bool(b_accept),absolute_chemical_certification=False,real_tasting_validated=False)
            transfers.append(record)
            profiles.append({'repeat':repeat,'beer':int(beers[i]),'radius':row['radius'],'status':row['status'],**dict(zip(features,candidate))})
            for field,j in zip(CHEMICAL_DEFECTS,positions):
                raw=float(data.calibrated.iloc[i][field]) if field in data.calibrated else np.nan
                supported=bool(np.isfinite(raw) and raw>0 and np.isclose(X[i,j],np.log10(raw) if field in ['DMS','X4VG'] else raw,rtol=1e-6,atol=1e-7))
                checks.append({'repeat':repeat,'beer':int(beers[i]),'radius':row['radius'],'field':field,'source_provided':X[i,j],'source_for_search':source[j],'candidate_provided':candidate[j],
                    'provided_nonincrease':bool(candidate[j]<=source[j]+1e-9),'source_provided_observed':bool(np.isfinite(X[i,j])),
                   'source_positive_calibrated_pair_matches':supported,'absolute_threshold_available_in_attachment':field!='X.E..2.nonenal',
                    'complete_absolute_certification':False,'caveat':'scale_and_detection_limit_conditions_not_fully_verified; candidate_has_no_new_measurement'})
        save_csv(pd.DataFrame(metrics),dest/'generation_comparison.csv'); save_csv(pd.DataFrame(transfers),dest/'transfer_independent_evaluation.csv')
    save_csv(pd.DataFrame(roles),dest/'disjoint_roles.csv'); save_csv(pd.DataFrame(evaluator_rows),dest/'real_holdout_diagnostics.csv'); save_csv(pd.DataFrame(checks),dest/'transfer_defect_checks.csv')
    save_csv(pd.DataFrame(profiles),dest/'transfer_profiles.csv'); save_csv(pd.concat(generated_profiles,ignore_index=True),dest/'generated_profiles_first_split.csv'); save_csv(pd.concat(portraits,ignore_index=True),dest/'generated_sensory_proxy_first_split.csv')
    summary=pd.DataFrame(metrics).groupby(['method','style'])[['MMD2_fixed_training_bandwidth','KS_mean','exact_copy_fraction','search_probability','independent_evaluator_probability','independent_evaluator_target_prediction_fraction','provided_bounds_violation_fraction']].mean().reset_index(); save_csv(summary,dest/'generation_summary.csv')
    write_json(dest/'status.json',{'complete':True,'repeats':5,'G_n':100,'J_n':100,'T_n':50,'basic_fields':len(features),'excluded_unverified_aggregates':sorted(AGGREGATES),
        'candidate_locks':locked,'independent_evaluation_scope':'J is disjoint within the current run; all original beers were seen in earlier research stages, not new external validation',
       'absolute_chemical_certification':False,'real_brewing_or_tasting':False,'elapsed_seconds':time.perf_counter()-start})
    progress('S3 Q3 complete')
```

源程序 12 src/stage3_graph.py

```
"""Train-only graph ablations and cross-fitted exploratory explanations.
AI-assisted, OpenAI Codex, 2026-10-25. Graph regularization is an existing method.
"""
from pathlib import Path
import time
import numpy as np
import pandas as pd
import networkx as nx
from scipy.linalg import solve
from scipy.stats import rankdata
from sklearn.model_selection import KFold, RepeatedStratifiedKFold
from .common import preprocessing, progress, save_csv, write_json
from .q2 import signed_laplacian, top_indices
from .data import NON_MOLECULES
from .stage3_algorithms import paired_oof_bootstrap


def train_correlation(Z):
    with np.errstate(invalid='ignore',divide='ignore'):
        corr=np.corrcoef(rankdata(Z,axis=0),rowvar=False)
    return np.nan_to_num(corr)


def solve_coeff(Z,Y,L,alpha,lam):
    return solve(Z.T@Z+alpha*np.eye(Z.shape[1])+lam*L,Z.T@(Y-Y.mean(0)),assume_a='pos',check_finite=False)


def candidates(cfg,kind):
    return [(alpha,lam,tau) for alpha in cfg['ridge_alphas'] for lam in ([0] if kind=='none' else cfg['graph_lambdas']) for tau in ([.7] if lam==0 else cfg['graph_thresholds'])]


def fit_tuned_graph(X,Y,cfg,kind,core,seed):
    choices=candidates(cfg,kind); scores=np.zeros(len(choices))
    for fold,(train,test) in enumerate(KFold(3,shuffle=True,random_state=seed).split(X)):
        prep=preprocessing(); Z=prep.fit_transform(X[train]); V=prep.transform(X[test]); corr=train_correlation(Z)
        lhs=Z.T@Z; rhs=Z.T@(Y[train]-Y[train].mean(0)); identity=np.eye(Z.shape[1])
        layers={tau:signed_laplacian(corr,tau) for tau in cfg['graph_thresholds']}
        perms=[np.random.default_rng(seed+1009*k+fold).permutation(Z.shape[1]) for k in range(cfg['graph_null_repeats'])] if kind=='null' else [None]
        for c,(alpha,lam,tau) in enumerate(choices):
            L=layers[tau] if lam else np.zeros_like(lhs)
            for perm in perms:
```

```
                LL=L[np.ix_(perm,perm)] if perm is not None else L
                beta=solve(lhs+alpha*identity+lam*LL,rhs,assume_a='pos',check_finite=False)
                pp=V@beta+Y[train].mean(0)
                scores[c]+=np.abs(Y[test][:,core]-pp[:,core]).sum()/(len(Y)*len(core)*len(perms))
    best=min(range(len(choices)),key=lambda i:(scores[i],choices[i][1],-choices[i][0]))
    alpha,lam,tau=choices[best]; prep=preprocessing(); Z=prep.fit_transform(X); corr=train_correlation(Z);
L=signed_laplacian(corr,tau)
    if kind=='none': L=np.zeros_like(L)
    betas=[]
    for k in range(cfg['graph_null_repeats'] if kind=='null' else 1):
        perm=np.random.default_rng(seed+20000+k).permutation(Z.shape[1]) if kind=='null' else None
        LL=L[np.ix_(perm,perm)] if perm is not None else L
        betas.append(solve_coeff(Z,Y,LL,alpha,lam))
    return {'prep':prep,'Z':Z,'corr':corr,'beta':betas,'mean':Y.mean(0),'params':{'alpha':alpha,'lambda':lam,'tau':t
au},'inner_core_mae':float(scores[best])}


def within_style_indices(labels,rng):
    labels=np.asarray(labels); indices=np.arange(len(labels))
    for label in np.unique(labels):
        ids=np.flatnonzero(labels==label); indices[ids]=rng.permutation(ids)
    return indices


def run_graph(data,out,cfg):
    started=time.perf_counter(); dest=Path(out)/'q2'; dest.mkdir(parents=True,exist_ok=True)
    X,Y=data.X.to_numpy(),data.Y.to_numpy(); features,targets=list(data.X),list(data.Y);
labels=data.meta.tasting_category_fine.to_numpy()
    core=np.array([j for j,t in enumerate(targets) if t.startswith(('A_','F_'))]); eligible=np.array([j for j,t in
enumerate(features) if t not in NON_MOLECULES])
    counts=np.zeros(len(Y)); totals={kind:np.zeros_like(Y) for kind in ['none','real']}; rows=[]; chosen=[];
effects=[]; modules=[]; style_effects=[]; selected_sets=[]
    for fold,(train,test) in enumerate(RepeatedStratifiedKFold(n_splits=3,n_repeats=cfg['outer_repeats'],random_stat
e=cfg['seed']).split(X,labels)):
        progress(f'S3 Q2 graph fold {fold+1}/15, elapsed {(time.perf_counter()-started)/60:.1f} min')
        fits={kind:fit_tuned_graph(X[train],Y[train],cfg,kind,core,cfg['seed']+fold) for kind in
['none','real','null']}
        for kind,fitted in fits.items():
            V=fitted['prep'].transform(X[test])
            for k,beta in enumerate(fitted['beta']):
                pred=V@beta+fitted['mean']
                for scope,ix in [('core30',core),('all50',np.arange(Y.shape[1]))]:
                    rows.append({'fold':fold,'repeat':fold//3,'graph':'shuffled' if kind=='null' else
kind,'null_rep':k if kind=='null' else -1,'scope':scope,'mae':np.abs(Y[test][:,ix]-pred[:,ix]).mean()})
                if kind in totals: totals[kind][test]+=pred
            chosen.append({'fold':fold,'graph':'shuffled' if kind=='null' else
kind,**fitted['params'],'inner_core_mae':fitted['inner_core_mae']})
        counts[test]+=1
        fit=fits['real']; Z=fit['Z']; V=fit['prep'].transform(X[test]); B=fit['beta'][0]; original=V@B+fit['mean'];
base=np.abs(Y[test]-original).mean(0)
        rng=np.random.default_rng(cfg['seed']+fold)
        fold_chosen=np.stack([top_indices(B[:,k],eligible) for k in range(Y.shape[1])]);
selected_sets.append(fold_chosen)
        for k,target in enumerate(targets):
            for rank,j in enumerate(fold_chosen[k],1):
                all_delta=[]; within_delta=[]
                for repeat in range(cfg['permutation_repeats']):
                    ix=rng.permutation(len(V)); ixw=within_style_indices(labels[test],rng)
                    all_delta.append(np.abs(Y[test,k]-(original[:,k]+(V[ix,j]-V[:,j])*B[j,k])).mean()-base[k])
                    within_delta.append(np.abs(Y[test,k]-(original[:,k]+(V[ixw,j]-V[:,j])*B[j,k])).mean()-base[k])
                effects.append({'fold':fold,'target':target,'rank':rank,'field':features[j],'coefficient':B[j,k],
                    'heldout_delta_mae':np.mean(all_delta),'within_style_delta_mae':np.mean(within_delta),
                    'singleton_test_style_fraction':np.mean([np.sum(labels[test]==label)==1 for label in
labels[test]])})
        # Joint module perturbations preserve internal feature relationships.
        graph=nx.Graph(); graph.add_nodes_from(range(X.shape[1])); W=np.abs(fit['corr'])>=fit['params']['tau'];
np.fill_diagonal(W,False)
        graph.add_edges_from(zip(*np.where(np.triu(W,1))))
        interesting={frozenset(nx.node_connected_component(graph,int(j))) for j in np.unique(fold_chosen)}
        for group in interesting:
            ix=np.array(sorted(group)); keep=np.setdiff1d(np.arange(X.shape[1]),ix)
            L=signed_laplacian(fit['corr'][np.ix_(keep,keep)],fit['params']['tau'])
            reduced=solve_coeff(Z[:,keep],Y[train],L,fit['params']['alpha'],fit['params']['lambda'])
            drop=V[:,keep]@reduced+fit['mean']; joint=np.zeros(Y.shape[1])
            for repeat in range(cfg['permutation_repeats']):
                order=within_style_indices(labels[test],rng)
                pp=original+(V[np.ix_(order,ix)]-V[:,ix])@B[ix]
                joint+=np.abs(Y[test]-pp).mean(0)-base
            for k,t in enumerate(targets):
                if set(fold_chosen[k])&group:
                    modules.append({'fold':fold,'target':t,'module_fields':';'.join(features[j] for j in
ix),'module_n':len(ix),'heldout_module_drop_mae':np.abs(Y[test,k]-drop[:,k]).mean()-base[k],'within_style_joint_permutati
```

```
on_mae':joint[k]/cfg['permutation_repeats']})
        # Style-centering is an explanatory diagnostic, not a Q1 model input.
        zr=Z.copy(); yr=Y[train].copy()
        for label in np.unique(labels[train]):
            ids=np.flatnonzero(labels[train]==label); zr[ids]-=Z[ids].mean(0); yr[ids]-=Y[train][ids].mean(0)
       adjusted=solve_coeff(zr,yr,signed_laplacian(train_correlation(zr),fit['params']['tau']),fit['params']['alpha'
],fit['params']['lambda'])
        for k,t in enumerate(targets):
            adjusted_set=set(top_indices(adjusted[:,k],eligible))
            for j in fold_chosen[k]: style_effects.append({'fold':fold,'target':t,'field':features[j],'still_top3_af
ter_style_centering':j in adjusted_set,'style_adjusted_coefficient':adjusted[j,k]})
        save_csv(pd.DataFrame(rows),dest/'graph_fold_metrics.csv');
save_csv(pd.DataFrame(chosen),dest/'selected_parameters.csv')
    for kind,pred in totals.items(): save_csv(pd.concat([data.meta[['beer']],pd.DataFrame(pred/counts[:,None],column
s=targets)],axis=1),dest/f'oof_graph_{kind}.csv')
    summary=pd.DataFrame(rows).groupby(['graph','scope']).mae.agg(['mean','std']).reset_index();
save_csv(summary,dest/'graph_comparison.csv')
    save_csv(pd.DataFrame(effects),dest/'crossfit_top3_effects.csv');
save_csv(pd.DataFrame(modules),dest/'crossfit_module_effects.csv');
save_csv(pd.DataFrame(style_effects),dest/'style_centering_diagnostics.csv')
    paired=[]
    for scope,ix in [('core30',core),('all50',np.arange(Y.shape[1]))]:
        r=paired_oof_bootstrap(Y[:,ix],totals['real'][:,ix]/counts[:,None],totals['none'][:,ix]/counts[:,None],label
s); r['target_scope']=scope; paired.append(r)
    write_json(dest/'paired_graph_vs_none.json',paired)
    progress('S3 Q2: full descriptive graph and 200 conditional bootstrap replicates')
    full=fit_tuned_graph(X,Y,cfg,'real',core,cfg['seed']+500); beta=full['beta'][0]; frequency=np.zeros_like(beta);
rng=np.random.default_rng(cfg['seed']); p=full['params']
    for b in range(cfg['bootstrap_repeats']):
        ix=rng.integers(0,len(X),len(X)); z=preprocessing().fit_transform(X[ix]);
L=signed_laplacian(train_correlation(z),p['tau']); bb=solve_coeff(z,Y[ix],L,p['alpha'],p['lambda'])
        for k in range(Y.shape[1]): frequency[top_indices(bb[:,k],eligible),k]+=1
    frequency/=cfg['bootstrap_repeats']; effect=pd.DataFrame(effects); adjusted=pd.DataFrame(style_effects);
meta=data.chemical_meta.set_index('field'); top=[]
    for k,t in enumerate(targets):
        for rank,j in enumerate(top_indices(beta[:,k],eligible),1):
            selected=effect[(effect.target==t)&(effect.field==features[j])];
centered=adjusted[(adjusted.target==t)&(adjusted.field==features[j])]
            within=float(selected.within_style_delta_mae.mean()) if len(selected) else None
            delta=float(selected.heldout_delta_mae.mean()) if len(selected) else None
            supported=frequency[j,k]>=.5 and len(selected)>=5 and delta is not None and delta>0 and within is not
None and within>0
            top.append({'target':t,'rank':rank,'field':features[j],'coefficient':beta[j,k],'bootstrap_frequency_cond
itional_on_full_selected_params':frequency[j,k],
                'crossfit_selection_count':len(selected),'heldout_delta_on_selected_folds':delta,'within_style_delta
_on_selected_folds':within,
                'style_centering_retention_fraction':float(centered.still_top3_after_style_centering.mean()) if
len(centered) else None,
                'origin':meta.at[features[j],'origin_paper'],'unit':meta.at[features[j],'unit'],'attachment_off_flav
ors':meta.at[features[j],'off_flavors'],
                'evidence_status':'comparatively_supported_exploratory_association' if supported else '
weak_or_incomplete_evidence',
                'chemical_review':'target_specific_literature_not_yet_complete','causal_status':'not_identified'})
    save_csv(pd.DataFrame(top),dest/'top3_with_evidence.csv');
save_csv(pd.DataFrame(frequency,index=features,columns=targets).reset_index(names='field'),dest/'bootstrap_frequencies.cs
v')
   write_json(dest/'status.json',{'complete':True,'outer_folds':15,'null_graphs_per_fit':cfg['graph_null_repeats'],'
full_graph_params':p,'bootstrap_repeats':cfg['bootstrap_repeats'],'bootstrap_params_reselected':False,
        'bootstrap_caveat':'conditional_on_hyperparameters_selected_on_full_data;
descriptive_stability_not_independent_inference','supported_top3':sum(r['evidence_status'].startswith('comparatively')
for r in top),'scope':'exploratory_association_not_causal','elapsed_seconds':time.perf_counter()-started})
    progress('S3 Q2 complete')
```

源程序 13 src/stage3_prediction.py

```
"""Nested shrinkage blending, ordinal sets and robustness; no novelty claim."""
from pathlib import Path
import time
import joblib
import numpy as np
import pandas as pd
from sklearn.ensemble import ExtraTreesRegressor, GradientBoostingRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.pipeline import Pipeline
from sklearn.model_selection import KFold, RepeatedStratifiedKFold, train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from .common import preprocessing, TrainQuantileClipper, progress, save_csv, write_json
from .q1 import metric_rows
from .stage2 import conformal_quantile
from .stage3_algorithms import fixed_grade_boundaries, grade_with_fixed_boundaries, ordinal_interval_image,
shrunk_blend_weights, paired_oof_bootstrap
```

```
0016
0017
0018 def tree_model(cfg,kind='et',clip=.01):
0019     prep=preprocessing(); prep.set_params(clip=TrainQuantileClipper(clip,1-clip))
0020     if kind=='et':
0021         model=ExtraTreesRegressor(n_estimators=cfg['extra_trees'],min_samples_leaf=3,max_features=.8,random_state=cf
g['seed'],n_jobs=2)
0022     else:
0023         model=MultiOutputRegressor(GradientBoostingRegressor(n_estimators=cfg['gbr_estimators'],learning_rate=.04,ma
x_depth=2,min_samples_leaf=5,max_features=.7,random_state=cfg['seed']),n_jobs=2)
0024     return Pipeline([('prep',prep),('model',model)])
0025
0026
0027 def fit_pair(X,Y,cfg):
0028     with joblib.parallel_backend('threading'):
0029         et=tree_model(cfg).fit(X,Y)
0030         gb=tree_model(cfg,'gbr').fit(X,Y)
0031     return et,gb
0032
0033
0034 def predict_pair(pair,X):
0035     with joblib.parallel_backend('threading'):
0036         return pair[0].predict(X),pair[1].predict(X)
0037
0038
0039 def learn_blend_nested(X,Y,cfg,core,fold_id):
0040     """No outer-test values enter this routine. V labels never enter S fits."""
0041     inner=list(KFold(3,shuffle=True,random_state=cfg['seed']+fold_id).split(X))
0042     pred_et,pred_gb=np.zeros_like(Y),np.zeros_like(Y)
0043     validation_predictions={eta:np.zeros_like(Y) for eta in cfg['blend_penalties']}
0044     for vfold,(s,v) in enumerate(inner):
0045         pe,pg=np.zeros_like(Y[s]),np.zeros_like(Y[s])
0046         for a,b in KFold(3,shuffle=True,random_state=cfg['seed']+vfold+99).split(s):
0047             fitted=fit_pair(X[s[a]],Y[s[a]],cfg)
0048             pe[b],pg[b]=predict_pair(fitted,X[s[b]])
0049         fitted=fit_pair(X[s],Y[s],cfg)
0050         ve,vg=predict_pair(fitted,X[v]); pred_et[v],pred_gb[v]=ve,vg
0051         for eta in cfg['blend_penalties']:
0052             w,_=shrunk_blend_weights(Y[s],pe,pg,eta)
0053             validation_predictions[eta][v]=ve*(1-w)+vg*w
0054     scores={eta:float(np.abs(Y[:,core]-pred[:,core]).mean()) for eta,pred in validation_predictions.items()}
0055     best=min(cfg['blend_penalties'],key=lambda eta:(scores[eta],-eta))
0056     w,glob=shrunk_blend_weights(Y,pred_et,pred_gb,best)
0057     unshrunk,_=shrunk_blend_weights(Y,pred_et,pred_gb,0)
0058     return w,unshrunk,glob,best,scores
0059
0060
0061 def run_prediction(data,out,cfg):
0062     start=time.perf_counter(); dest=Path(out)/'q1'; dest.mkdir(parents=True,exist_ok=True)
0063     X,Y=data.X.to_numpy(),data.Y.to_numpy(); targets=list(data.Y)
0064     core=np.array([j for j,t in enumerate(targets) if t.startswith(('A_','F_'))]);
labels=data.meta.tasting_category_fine.to_numpy()
0065     names=['mean','ridge','extra_trees','gbr','fixed_half_blend','global_oof_blend','unshrunk_target_blend','shrunk_
target_blend']
0066     sums={m:np.zeros_like(Y) for m in names}; count=np.zeros(len(Y)); assignments=[]; fold_rows=[]; weight_rows=[];
stress=[]; clipping=[]
0067     grade_votes=np.zeros((len(Y),Y.shape[1],7),int)
0068     for fold,(train,test) in enumerate(RepeatedStratifiedKFold(n_splits=cfg['outer_folds'],n_repeats=cfg['outer_repe
ats'],random_state=cfg['seed']).split(X,labels)):
0069         progress(f'S3 Q1 fold {fold+1}/15: nested inner weights; elapsed {(time.perf_counter()-start)/60:.1f} min')
0070         w,wu,glob,eta,eta_scores=learn_blend_nested(X[train],Y[train],cfg,core,fold)
0071         pair=fit_pair(X[train],Y[train],cfg); etp,gbp=predict_pair(pair,X[test])
0072         ridge_search=GridSearchCV(Pipeline([('prep',preprocessing()),('model',Ridge())]),{'model__alpha':cfg['ridge_
alphas']},
0073             scoring=lambda est,xx,yy:-np.abs(yy[:,core]-est.predict(xx)[:,core]).mean(),cv=KFold(3,shuffle=True,rand
om_state=cfg['seed']),n_jobs=1)
0074         ridge=ridge_search.fit(X[train],Y[train]).best_estimator_
0075         predictions={'mean':np.broadcast_to(Y[train].mean(0),Y[test].shape),'ridge':ridge.predict(X[test]),'extra_tr
ees':etp,'gbr':gbp,
0076                      'fixed_half_blend':(etp+gbp)/2,'global_oof_blend':etp*(1-glob)+gbp*glob,'unshrunk_target_blend':
etp*(1-wu)+gbp*wu,'shrunk_target_blend':etp*(1-w)+gbp*w}
0077         for name,pred in predictions.items():
0078             sums[name][test]+=pred
0079             for scope,ix in [('core30',core),('all50',np.arange(Y.shape[1]))]:
0080                 fold_rows.append({'fold':fold,'repeat':fold//3,'model':name,'scope':scope,'mae':np.abs(Y[test][:,ix]
-pred[:,ix]).mean(),'n_test':len(test)})
0081         count[test]+=1
0082        assignments.extend({'fold':fold,'repeat':fold//3,'beer':int(data.meta.iloc[i].beer),'role':'outer_validation'
} for i in test)
0083         weight_rows.extend({'fold':fold,'target':target,'weight_GBR':w[j],'global_weight_GBR':glob,'eta':eta} for
j,target in enumerate(targets))
0084         boundaries=fixed_grade_boundaries(Y[train]); grades=grade_with_fixed_boundaries(etp,boundaries)
```

```
        for j in range(Y.shape[1]): np.add.at(grade_votes[:,j],(test,grades[:,j]),1)
        rng=np.random.default_rng(cfg['seed']+fold); low=np.nanquantile(X[train],.2,axis=0)
        for scenario in ['MCAR','low_value_preferential']:
            for rate in [.0,.05,.1,.2]:
                eligible=np.flatnonzero(np.isfinite(X[test]).ravel()); n=int(rate*len(eligible)); a=X[test].copy()
                weights=np.where((a<=low).ravel()[eligible],4.,1.) if scenario!='MCAR' else np.ones(len(eligible))
                if n: a.ravel()[rng.choice(eligible,n,replace=False,p=weights/weights.sum())]=np.nan
                pe,pg=predict_pair(pair,a)
                for name,pred in [('extra_trees',pe),('shrunk_target_blend',pe*(1-w)+pg*w)]:
                    stress.append({'fold':fold,'scenario':scenario,'extra_missing_rate':rate,'model':name,'core30_mae':np.abs(Y[test][:,core]-pred[:,core]).mean(),'all50_mae':np.abs(Y[test]-pred).mean()})
        for lower in [0,.01,.025]:
            alternative=pair[0] if lower==.01 else tree_model(cfg,clip=lower).fit(X[train],Y[train])
            pred=alternative.predict(X[test]); clipping.append({'fold':fold,'lower_clip':lower,'core30_mae':np.abs(Y[test][:,core]-pred[:,core]).mean(),'all50_mae':np.abs(Y[test]-pred).mean()})
        # Save progress atomically per finished outer fold.
        save_csv(pd.DataFrame(fold_rows),dest/'fold_metrics.csv'); save_csv(pd.DataFrame(weight_rows),dest/'blend_weights.csv')
        write_json(dest/'progress.json',{'completed_outer_folds':fold+1,'total_outer_folds':15,'elapsed_seconds':time.perf_counter()-start})
    metric=[]
    for name in names:
        pred=sums[name]/count[:,None]
        save_csv(pd.concat([data.meta[['beer']],pd.DataFrame(pred,columns=targets)],axis=1),dest/f'oof_{name}.csv')
        metric+=metric_rows(Y,pred,targets,name)
    bytarget=pd.DataFrame(metric); save_csv(bytarget,dest/'metrics_by_target.csv')
    scopes=[]
    for scope,fields in [('core30',[targets[j] for j in core]),('all50',targets)]:
        a=bytarget[bytarget.target.isin(fields)].groupby('model')[['mae','rmse','r2']].mean().reset_index(); a.insert(0,'scope',scope); scopes.append(a)
    summary=pd.concat(scopes,ignore_index=True); save_csv(summary,dest/'metrics_by_scope.csv')
    save_csv(pd.DataFrame(assignments),dest/'outer_roles.csv'); save_csv(pd.DataFrame(stress),dest/'missingness_stress.csv');save_csv(pd.DataFrame(clipping),dest/'clipping_ablation.csv')
    save_csv(pd.concat([data.meta[['beer']],pd.DataFrame(grade_votes.argmax(2),columns=targets)],axis=1),dest/'ASSUMPTION_oof_relative_grades.csv')
    paired=[]
    for name in ['gbr','global_oof_blend','unshrunk_target_blend','shrunk_target_blend']:
        for scope,ix in [('core30',core),('all50',np.arange(Y.shape[1]))]:
            r=paired_oof_bootstrap(Y[:,ix],sums[name][:,ix]/count[:,None],sums['extra_trees'][:,ix]/count[:,None],labels)
            r.update(model_A=name,model_B='extra_trees',target_scope=scope); paired.append(r)
    write_json(dest/'paired_conditional_comparison.json',paired)
    intervals=[]; coverage_rows=[]
    for repeat in range(cfg['conformal_repeats']):
        train,rest=train_test_split(np.arange(len(X)),test_size=.4,random_state=cfg['seed']+71+repeat)
        calibration,test=train_test_split(rest,test_size=.5,random_state=cfg['seed']+72+repeat)
        model=tree_model(cfg).fit(X[train],Y[train]); pp=model.predict(X[test]); q=conformal_quantile(np.abs(Y[calibration]-model.predict(X[calibration])),cfg['conformal_alpha'])
        boundaries=fixed_grade_boundaries(Y[train]); lo,hi=ordinal_interval_image(pp-q,pp+q,boundaries); true=grade_with_fixed_boundaries(Y[test],boundaries); point=grade_with_fixed_boundaries(pp,boundaries)
        for j,t in enumerate(targets):
            coverage_rows.append({'repeat':repeat,'target':t,'continuous_coverage':((Y[test,j]>=pp[:,j]-q[j])&(Y[test,j]<=pp[:,j]+q[j])).mean(),'interval_width':2*q[j],
                'relative_grade_set_coverage':((true[:,j]>=lo[:,j])&(true[:,j]<=hi[:,j])).mean(),'mean_grade_set_size':(hi[:,j]-lo[:,j]+1).mean()})
        for i,row in enumerate(test):
            for j,t in enumerate(targets): intervals.append({'repeat':repeat,'beer':int(data.meta.iloc[row].beer),'target':t,'continuous_prediction':pp[i,j],'lower':pp[i,j]-q[j],'upper':pp[i,j]+q[j],'relative_grade':point[i,j],'relative_grade_min':lo[i,j],'relative_grade_max':hi[i,j],'status':'fixed_training_relative_bins_not_absolute_raw_scores'})
        save_csv(pd.DataFrame({'repeat':repeat,'beer':data.meta.beer,'role':['train' if i in set(train) else 'calibration' if i in set(calibration) else 'test' for i in range(len(X))]}),dest/f'conformal_roles_{repeat}.csv')
    save_csv(pd.DataFrame(intervals),dest/'ASSUMPTION_ordinal_sets.csv'); save_csv(pd.DataFrame(coverage_rows),dest/'conformal_coverage.csv')
    write_json(dest/'status.json',{'complete':True,'outer_folds':15,'unique_beers':len(X),'grade_semantics':'relative_training_quantile_not_original_0_6_intensity','absolute_anchors_observed':False,'elapsed_seconds':time.perf_counter()-start,'model_originality':'standard_estimators_and_regularized_stacking_variant_no_first_invention_claim'})
    progress('S3 Q1 complete')
```

源程序 14 scripts/build_paper_pdf.py

```
"""AI-assisted PDF layout: OpenAI Codex / GPT-6, 2026-10-24.
Run with the bundled document runtime, not the numerical research environment.
"""
from pathlib import Path
from xml.sax.saxutils import escape
import json

from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER, TA_JUSTIFY, TA_LEFT
from reportlab.lib.pagesizes import A4
```

```
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.units import cm
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import BaseDocTemplate, Frame, PageTemplate, Paragraph, Spacer, PageBreak, Image,
LongTable, TableStyle, KeepTogether
from reportlab.platypus.tableofcontents import TableOfContents
from PIL import Image as PILImage

ROOT = Path(__file__).resolve().parents[1]
CONTENT = json.loads((ROOT / "paper/paper_content.json").read_text(encoding="utf-8"))
pdfmetrics.registerFont(TTFont("Song", "C:/Windows/Fonts/simsun.ttc", subfontIndex=0))
pdfmetrics.registerFont(TTFont("Hei", "C:/Windows/Fonts/simhei.ttf"))
pdfmetrics.registerFontFamily("Song", normal="Song", bold="Hei", italic="Song", boldItalic="Hei")

styles = {
    "Title": ParagraphStyle("Title", fontName="Hei", fontSize=17, leading=25, alignment=TA_CENTER, spaceAfter=15,
textColor=colors.black),
    "Front": ParagraphStyle("Front", fontName="Hei", fontSize=13, leading=20, alignment=TA_CENTER, spaceBefore=6,
spaceAfter=9),
    "Body": ParagraphStyle("Body", fontName="Song", fontSize=10.5, leading=17, firstLineIndent=21,
alignment=TA_JUSTIFY, wordWrap="CJK", spaceAfter=6),
    "Heading1": ParagraphStyle("Heading1", fontName="Hei", fontSize=13, leading=20, spaceBefore=12, spaceAfter=8,
keepWithNext=True),
    "Heading2": ParagraphStyle("Heading2", fontName="Hei", fontSize=11.5, leading=18, spaceBefore=8, spaceAfter=5,
keepWithNext=True),
    "Equation": ParagraphStyle("Equation", fontName="Song", fontSize=10, leading=17, alignment=TA_CENTER,
wordWrap="CJK", spaceBefore=5, spaceAfter=8),
    "Caption": ParagraphStyle("Caption", fontName="Song", fontSize=9, leading=14, alignment=TA_CENTER,
wordWrap="CJK", spaceBefore=5, spaceAfter=9),
    "Cell": ParagraphStyle("Cell", fontName="Song", fontSize=8.5, leading=12, wordWrap="CJK", alignment=TA_LEFT),
    "CellHeader": ParagraphStyle("CellHeader", fontName="Hei", fontSize=9, leading=13, wordWrap="CJK",
alignment=TA_CENTER),
    "Reference": ParagraphStyle("Reference", fontName="Song", fontSize=9, leading=14, wordWrap="CJK", spaceAfter=6),
}


def text(s):
    return escape(str(s)).replace("Bˆ", "B_hat").replace("Dˆ", "D_hat").replace("\u0302", "_hat").replace("−", "
-").replace("ŷ", "y_hat").replace("ᵀ", "<super>T</super>").replace("²", "<super>2</super>")


class PaperDoc(BaseDocTemplate):
    def afterFlowable(self, flowable):
        if isinstance(flowable, Paragraph) and flowable.style.name in ["Heading1", "Heading2"]:
            level = 0 if flowable.style.name == "Heading1" else 1
            key = flowable.bookmark_key
            self.canv.bookmarkPage(key)
            self.canv.addOutlineEntry(flowable.getPlainText(), key, level=level, closed=False)
            if level == 0:
                self.notify("TOCEntry", (level, flowable.getPlainText(), self.page - 2, key))


def footer(canvas, doc):
    canvas.setAuthor("")
    canvas.setTitle(CONTENT["title"])
    canvas.setSubject("啤酒化合物感官预测与风格迁移参考研究")
    if doc.page >= 3:
        canvas.setFont("Song", 9)
        canvas.drawCentredString(A4[0] / 2, 1.2 * cm, str(doc.page - 2))


output = ROOT / "paper/论文参考稿.pdf"
doc = PaperDoc(str(output), pagesize=A4, leftMargin=2.5*cm, rightMargin=2.5*cm, topMargin=2*cm, bottomMargin=2*cm,
pageCompression=1)
frame = Frame(doc.leftMargin, doc.bottomMargin, doc.width, doc.height, leftPadding=0, rightPadding=0, topPadding=0,
bottomPadding=0)
doc.addPageTemplates([PageTemplate(id="paper", frames=[frame], onPage=footer)])
story = [Spacer(1, .4 * cm), Paragraph(text(CONTENT["title"]), styles["Title"]), Paragraph("摘 要",
styles["Front"]), Paragraph(text(CONTENT["abstract"]), styles["Body"]), Spacer(1,.3*cm), Paragraph("<b>关键词：</b>" +
text(CONTENT["keywords"]), styles["Body"]), PageBreak(), Paragraph("目 录", styles["Front"])]
toc = TableOfContents()
toc.levelStyles = [ParagraphStyle("TOC1", fontName="Song", fontSize=10.5, leading=16, leftIndent=0,
firstLineIndent=0, spaceBefore=3), ParagraphStyle("TOC2", fontName="Song", fontSize=9.5, leading=14, leftIndent=18,
firstLineIndent=0, spaceBefore=1)]
story += [toc, PageBreak()]
in_references = False
for i, block in enumerate(CONTENT["blocks"]):
    kind = block["type"]
    if kind == "heading":
        in_references = block["text"] == "参考文献"
        para = Paragraph(text(block["text"]), styles["Heading1" if block["level"] == 1 else "Heading2"])
```

```
        para.bookmark_key = "heading_%d" % i
        story.append(para)
    elif kind == "paragraph":
        story.append(Paragraph(text(block["text"]), styles["Reference" if in_references else "Body"]))
    elif kind == "equation":
        equation_path = ROOT / "paper/equations" / ("eq_%02d.png" % int(block["number"]))
        with PILImage.open(equation_path) as im:
            width, height = im.size
        ratio = min(72 / 250, (doc.width - 44) / width)
        equation_image = Image(str(equation_path), width=width*ratio, height=height*ratio)
        equation_image.hAlign = "CENTER"
        equation_table = LongTable([[equation_image, Paragraph("（%s）" % block["number"], styles["Equation"])]],
colWidths=[doc.width-38,38])
        equation_table.setStyle(TableStyle([("VALIGN",(0,0),(-1,-1),"MIDDLE"),("ALIGN",(0,0),(0,0),"CENTER"),("TOPPA
DDING",(0,0),(-1,-1),5),("BOTTOMPADDING",(0,0),(-1,-1),8)]))
        story.append(equation_table)
    elif kind == "table":
        rows = [[Paragraph(text(c), styles["CellHeader"]) for c in block["headers"]]]
        rows += [[Paragraph(text(c), styles["Cell"]) for c in row] for row in block["rows"]]
        ratios = block["widths"] or [1/len(rows[0])] * len(rows[0])
        table = LongTable(rows, colWidths=[doc.width*x for x in ratios], repeatRows=1, hAlign="CENTER")
        table.setStyle(TableStyle([("GRID",(0,0),(-1,-1),.45,colors.HexColor("#D9D9D9")),("BACKGROUND",(0,0),(-1,0),
colors.HexColor("#E8EEF2")),("VALIGN",(0,0),(-1,-1),"MIDDLE"),("LEFTPADDING",(0,0),(-1,-1),6),("RIGHTPADDING",(0,0),(-1,-
1),6),("TOPPADDING",(0,0),(-1,-1),6),("BOTTOMPADDING",(0,0),(-1,-1),6)]))
        story.extend([Spacer(1,5),table,Spacer(1,9)])
    elif kind == "figure":
        path = ROOT / block["path"]
        with PILImage.open(path) as im: width,height=im.size
        max_height = (10.5 if "style_counts" in str(path) else 7) * cm
        ratio=min(doc.width/width,max_height/height)
        image = Image(str(path),width=width*ratio,height=height*ratio)
        image.hAlign="CENTER"
        story.append(KeepTogether([Spacer(1,4),image,Paragraph(text(block["caption"]), styles["Caption"])]))
doc.multiBuild(story)
print(json.dumps({"pdf":str(output),"bytes":output.stat().st_size},ensure_ascii=False))
```

源程序 15 scripts/build_revised_pdf.py

```
"""Anonymous PDF with math figures, bookmarks and complete source appendix.
AI-assisted layout, OpenAI Codex, 2026-10-25. Run with document runtime.
"""
from pathlib import Path
from xml.sax.saxutils import escape
import argparse
import hashlib
import json
from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER, TA_JUSTIFY
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.units import cm
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import BaseDocTemplate, Frame, PageTemplate, Paragraph, Spacer, PageBreak, Image,
LongTable, TableStyle, KeepTogether
from reportlab.platypus.tableofcontents import TableOfContents
from PIL import Image as PILImage

ROOT=Path(__file__).resolve().parents[1]
parser=argparse.ArgumentParser(); parser.add_argument('--content',required=True);
parser.add_argument('--output',required=True); args=parser.parse_args()
content=json.loads(Path(args.content).read_text(encoding='utf-8')); output=Path(args.output);
output.parent.mkdir(parents=True,exist_ok=True)
for name,path in [('Song','C:/Windows/Fonts/simsun.ttc'),('Hei','C:/Windows/Fonts/simhei.ttf'),('Code','C:/Windows/F
onts/consola.ttf')]:
    pdfmetrics.registerFont(TTFont(name,path,subfontIndex=0))
pdfmetrics.registerFontFamily('Song',normal='Song',bold='Hei',italic='Song',boldItalic='Hei')
styles={
 'Title':ParagraphStyle('Title',fontName='Hei',fontSize=17,leading=25,alignment=TA_CENTER,spaceAfter=15),
 'Front':ParagraphStyle('Front',fontName='Hei',fontSize=13,leading=20,alignment=TA_CENTER,spaceAfter=12),
 'Body':ParagraphStyle('Body',fontName='Song',fontSize=10.5,leading=17,firstLineIndent=21,alignment=TA_JUSTIFY,wordW
rap='CJK',spaceAfter=6),
 'Heading1':ParagraphStyle('Heading1',fontName='Hei',fontSize=13,leading=20,spaceBefore=12,spaceAfter=8,keepWithNext
=True),
 'Heading2':ParagraphStyle('Heading2',fontName='Hei',fontSize=11.5,leading=18,spaceBefore=8,spaceAfter=5,keepWithNex
t=True),
 'Caption':ParagraphStyle('Caption',fontName='Song',fontSize=9,leading=14,alignment=TA_CENTER,wordWrap='CJK',spaceAf
ter=9),
 'Cell':ParagraphStyle('Cell',fontName='Song',fontSize=8.2,leading=12,wordWrap='CJK'),
 'Header':ParagraphStyle('Header',fontName='Hei',fontSize=8.5,leading=12,wordWrap='CJK',alignment=TA_CENTER),
 'Reference':ParagraphStyle('Reference',fontName='Song',fontSize=9,leading=14,wordWrap='CJK',spaceAfter=5),
```

```
 'Code':ParagraphStyle('Code',fontName='Code',fontSize=6.8,leading=8.8,wordWrap='CJK',spaceAfter=0),
}
def txt(s):
    s=str(s).replace('−','-').replace('^','_hat').replace('ŷ','y_hat').replace('ᵀ','^T').replace('−','-').replace('—','-')
    return escape(s).replace('²','<super>2</super>')

class Doc(BaseDocTemplate):
    def afterFlowable(self,flowable):
        if isinstance(flowable,Paragraph) and flowable.style.name in ['Heading1','Heading2']:
            level=0 if flowable.style.name=='Heading1' else 1; key=flowable.bookmark_key
            self.canv.bookmarkPage(key); self.canv.addOutlineEntry(flowable.getPlainText(),key,level=level,closed=False)
            if level==0: self.notify('TOCEntry',(0,flowable.getPlainText(),self.page-2,key))

def footer(canvas,doc):
    canvas.setAuthor(''); canvas.setTitle(content['title']); canvas.setSubject(content.get('subject','数学建模参考研究'))
    if doc.page>=3: canvas.setFont('Song',9); canvas.drawCentredString(A4[0]/2,1.2*cm,str(doc.page-2))

doc=Doc(str(output),pagesize=A4,leftMargin=2.5*cm,rightMargin=2.5*cm,topMargin=2*cm,bottomMargin=2*cm,pageCompression=1)
frame=Frame(doc.leftMargin,doc.bottomMargin,doc.width,doc.height,leftPadding=0,rightPadding=0,topPadding=0,bottomPadding=0)
doc.addPageTemplates([PageTemplate(id='paper',frames=[frame],onPage=footer)])
story=[Spacer(1,.4*cm),Paragraph(txt(content['title']),styles['Title']),Paragraph(content.get('front_label','摘 要'),styles['Front']),Paragraph(txt(content['abstract']),styles['Body']),Spacer(1,10),Paragraph('<b>'+txt(content.get('keyword_label','关键词'))+'：</b>'+txt(content['keywords']),styles['Body']),PageBreak(),Paragraph('目 录',styles['Front'])]
toc=TableOfContents(); toc.levelStyles=[ParagraphStyle('TOC',fontName='Song',fontSize=10,leading=15,spaceBefore=2)]; story.extend([toc,PageBreak()])

def heading(s,level,key):
    p=Paragraph(txt(s),styles['Heading1' if level==1 else 'Heading2']); p.bookmark_key=key; return p

def code_markup(s):
    # Switch fonts only to preserve characters; no source text is redacted.
    runs=[]; old=None; run=''
    for char in s:
        font='Code' if ord(char) in pdfmetrics.getFont('Code').face.charToGlyph else 'Song'
        if ord(char) not in pdfmetrics.getFont(font).face.charToGlyph:
            char='\\u%04x'%ord(char); font='Code'
        if old is not None and font!=old:
            runs.append(f'<font name="{old}">'+escape(run).replace(' ',' ')+'</font>'); run=''
        old=font; run+=char
    if run: runs.append(f'<font name="{old}">'+escape(run).replace(' ',' ')+'</font>')
    return ''.join(runs)

in_refs=False; source_index=[]
for index,b in enumerate(content['blocks']):
    kind=b['type']
    if kind=='heading':
        in_refs=b['text']=='参考文献'; story.append(heading(b['text'],b['level'],f'h_{index}'))
    elif kind=='paragraph': story.append(Paragraph(txt(b['text']),styles['Reference' if in_refs else 'Body']))
    elif kind=='equation':
        path=ROOT/content.get('equation_dir','paper/equations')/f"eq_{b.get('asset_number',b['number']):02d}.png"
        with PILImage.open(path) as im: w,h=im.size
        scale=min(72/250,(doc.width-44)/w); img=Image(str(path),width=w*scale,height=h*scale)
        table=LongTable([[img,Paragraph(f"（{b['number']}）",styles['Caption'])]],colWidths=[doc.width-38,38]); table.setStyle(TableStyle([('VALIGN',(0,0),(-1,-1),'MIDDLE'),('ALIGN',(0,0),(0,0),'CENTER'),('TOPPADDING',(0,0),(-1,-1),5),('BOTTOMPADDING',(0,0),(-1,-1),8)])); story.append(table)
    elif kind=='table':
        rows=[[Paragraph(txt(c),styles['Header']) for c in b['headers']]]+[[Paragraph(txt(c),styles['Cell']) for c in row] for row in b['rows']]
        widths=b.get('widths') or [1/len(rows[0])]*len(rows[0]); table=LongTable(rows,colWidths=[doc.width*w for w in widths],repeatRows=1)
        table.setStyle(TableStyle([('GRID',(0,0),(-1,-1),.4,colors.HexColor('#D5DCE1')),('BACKGROUND',(0,0),(-1,0),colors.HexColor('#E8EEF2')),('VALIGN',(0,0),(-1,-1),'MIDDLE'),('LEFTPADDING',(0,0),(-1,-1),5),('RIGHTPADDING',(0,0),(-1,-1),5),('TOPPADDING',(0,0),(-1,-1),5),('BOTTOMPADDING',(0,0),(-1,-1),5)])); story.extend([Spacer(1,4),table,Spacer(1,8)])
    elif kind=='figure':
        path=ROOT/b['path']
        with PILImage.open(path) as im: w,h=im.size
        scale=min(doc.width/w,b.get('height_cm',7)*cm/h); img=Image(str(path),width=w*scale,height=h*scale)
        story.append(KeepTogether([img,Paragraph(txt(b['caption']),styles['Caption'])]))
    elif kind=='pagebreak': story.append(PageBreak())
    elif kind=='source_files':
        for number,relative in enumerate(b['paths'],1):
            path=ROOT/relative; raw=path.read_bytes(); source=raw.decode('utf-8-sig'); source_index.append({'file':relative,'sha256':hashlib.sha256(raw).hexdigest(),'lines':len(source.splitlines())})
            story.append(heading(f'源程序 {number} {relative}',2,f'src_{number}'))
            for lineno,line in enumerate(source.splitlines(),1):
                source_line=line.expandtabs(4)
```

```
                story.append(Paragraph(f'<font color="#777777">{lineno:04d} </font>'+code_markup(source_line or ' '),styles['Code']))
            story.append(Spacer(1,10))
doc.multiBuild(story)
if source_index: (output.parent/'source_appendix_manifest.json').write_text(json.dumps(source_index,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps({'output':str(output),'bytes':output.stat().st_size,'source_files':len(source_index)},ensure_ascii=False))
```

源程序 16 scripts/package_reference.py

```
"""Package only authorized research artifacts; preserve exact bytes for publication."""
from pathlib import Path
import argparse
import base64
import hashlib
import json
import zipfile

ROOT = Path(__file__).resolve().parents[1]
parser = argparse.ArgumentParser()
parser.add_argument("--out", required=True)
parser.add_argument("--payload")
args = parser.parse_args()
root_names = {"README.md", ".gitignore", "config.json", "config_stage2.json", "config_stage3_design.json", "config_stage3.json", "run.py", "run_stage2.py", "run_stage3.py", "requirements.txt", "requirements-lock.txt", "requirements-paper.txt"}
directories = {"src", "tests", "docs", "outputs", "paper", "scripts"}
suffixes = {".md", ".py", ".txt", ".json", ".csv", ".png", ".pdf"}
files = []
for path in sorted(ROOT.rglob("*")):
    if not path.is_file():
        continue
    rel = path.relative_to(ROOT)
    if "private" in rel.parts or "__pycache__" in rel.parts:
        continue
    if rel.as_posix() in root_names or (rel.parts[0] in directories and path.suffix.lower() in suffixes):
        assert path.suffix.lower() not in {".xlsx", ".joblib", ".xml"}
        files.append(path)
output = Path(args.out)
output.parent.mkdir(parents=True, exist_ok=True)
with zipfile.ZipFile(output, "w", zipfile.ZIP_DEFLATED) as z:
    for path in files:
        z.write(path, path.relative_to(ROOT).as_posix())
with zipfile.ZipFile(output) as z:
    assert z.testzip() is None and len(z.namelist()) == len(files)
if args.payload:
    entries = []
    for path in files:
        raw = path.read_bytes()
        binary = path.suffix.lower() in {".png", ".pdf"}
        entries.append({"path": path.relative_to(ROOT).as_posix(), "encoding": "base64" if binary else "utf-8",
                        "content": base64.b64encode(raw).decode("ascii") if binary else raw.decode("utf-8"),
                        "blob_sha": hashlib.sha1(b"blob " + str(len(raw)).encode() + b"\0" + raw).hexdigest()})
    Path(args.payload).write_text(json.dumps(entries, ensure_ascii=False, separators=(",", ":")), encoding="utf-8")
print(json.dumps({"zip": str(output), "files": len(files), "bytes": output.stat().st_size,
                  "sha256": hashlib.sha256(output.read_bytes()).hexdigest(),
                  "payload_characters": len(Path(args.payload).read_text(encoding="utf-8")) if args.payload else None}, ensure_ascii=False))
```

源程序 17 scripts/prepare_paper.py

```
"""AI-assisted manuscript preparation: OpenAI Codex / GPT-6, 2026-10-24.
Every reported result is read from saved experiment tables.
"""
from pathlib import Path
import json
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "outputs/stage2"
PAPER = ROOT / "paper"
PAPER.mkdir(exist_ok=True)
status = json.loads((OUT / "status.json").read_text(encoding="utf-8"))
metrics = pd.read_csv(OUT / "q1/metrics_summary.csv").set_index("model")
top = pd.read_csv(OUT / "q2/top3_evidence_review.csv")
gen = pd.read_csv(OUT / "q3/generator_comparison_mean.csv")
transfer = pd.read_csv(OUT / "q3/expanded_transfer_diagnostics.csv")
facts = json.loads((OUT / "audit/verified_facts.json").read_text(encoding="utf-8"))
```

```
coverage = pd.read_csv(OUT / "q1/conformal_coverage.csv")
blocks = []
def H(text, level=1): blocks.append({"type":"heading", "level":level, "text":text})
def P(text): blocks.append({"type":"paragraph", "text":text})
def E(text, number): blocks.append({"type":"equation", "text":text, "number":number})
def T(headers, rows, widths=None): blocks.append({"type":"table", "headers":headers, "rows":rows, "widths":widths})
def F(name, caption, height=6): blocks.append({"type":"figure", "path":"outputs/stage2/figures/"+name, "
caption":caption, "height_cm":height})

baseline = float(metrics.loc["mean", "mae"])
gbr = float(metrics.loc["gbr_fixed", "mae"])
reduction = (baseline-gbr)/baseline
abstract = f"针对啤酒化合物到感官画像的预测、关键因素分析和风格生成问题，构建数据审计、多输出预测、网络关联解释及约束迁移的统一研究流程。依据实际附件，以 beer 为唯一键关联 250 个样本、231 个数值字段和 50 个连续感官目标，逐列核查对数尺度与缺失处理。原始研究采用按品评人员标准化的评分，缺少原始参数时绝对 0–6 分不可唯一恢复，故将连续预测与分级假设分开验证。在训练折内预处理和调参的三折两次重复验证中，固定配置 GBR 的宏平均 MAE 为 {gbr:.4f}，相对均值参照下降 {reduction:.2%}；独立训练、校准和测试分割下，名义 90% 的逐维边际区间获得 {status['conformal_empirical_average_coverage']:.2%} 的平均测试覆盖率。三类分级假设在 {status['grade_sensitivity_disagreement']:.2%} 的预测位置存在分歧。第二问以相关网络和图约束岭回归产生 Top-3 候选，结合 bootstrap 与留出置换贡献，仅 {status['comparatively_supported_top3_associations']}/150 条通过当前描述性证据条件。第三问比较重采样、局部凸组合和 PCA 高斯生成；局部凸组合保留非复制样本优势，但分布距离未全面优于重采样。扩展迁移搜索为 11 个源样本找到模型内可行候选，其中 2 个含缺失缺陷源值。结果支持相对感官预测与受约束的统计设计，尚不支持绝对评分恢复、化学因果或真实酿造效果的结论。"

H("1 问题重述与研究范围")
P("啤酒风味由原料、发酵过程和多种化合物相互作用共同形成。原始研究已将实测化学资料与训练品评联系起来，说明此类数据具有建模价值[1]。本题进一步要求处理缺失、分级评分、稳定因素识别和跨风格生成，因此研究重点不仅是拟合精度，还包括目标定义是否清楚、解释是否稳健，以及生成谱满足什么层次的约束。")
P("第一问以化合物和理化测量为输入，估计品评表各感官维度；第二问建立化合物关系网络并对每维给出三个候选因素；第三问按 tasting_category_fine 生成风格条件样本，并将 Pils/Lager 向 Tripel 调整，要求指定缺陷项不恶化。本文对实际附件的 50 个连续目标完整输出，Aroma 与 Flavour 维度单独标识；最终提交维度应依据官方明确口径核对。")
P("本文研究范围为对给定附件的统计建模与可复现验证。原始 0–6 分锚点、部分浓度尺度、聚合量公式和真实感官试验均存在信息缺口。对这些缺口采用明确假设、敏感性和失败记录处理，保留可核查结果，不以预测代理替代实测。")

H("2 模型假设与符号说明")
H("2.1 基本假设", 2)
P("假设一，beer 能稳定标识同一款啤酒，不同文件按该字段正确对应。该假设经唯一性、集合和风格一致性检查支持；受损的 beer_id 不用于合并。假设二，在相同数据获取机制下，现有样本可用于研究新样本的相对感官预测；此假设不保证未见风格或不同酿造工艺下的外推。")
P("假设三，训练分布的分位截尾与中位数插补可作为噪声、极端值与缺失的研究基线。它们不证明被处理值是错误观测，后续需要针对检测限与非随机缺失改进。假设四，相关网络可作为参数正则化信息，但不具有先验因果含义。假设五，生成与迁移的经验范围和代理模型只定义统计可行性，不构成物理、化学或生产可行性的充分条件。")
H("2.2 主要符号", 2)
T(["符号", "含义"], [["n、p、q", "样本数 250、输入字段数 231、感官目标数 50"], ["X、Y", "附件尺度输入矩阵与连续感官目标矩阵"], ["Z", "仅由训练数据拟合预处理得到的标准化输入"], ["B、α、λ", "回归系数、岭惩罚和网络惩罚强度"], ["W、L", "截断有符号相关权与对应拉普拉斯矩阵"], ["f、g", "连续感官预测器与风格诊断分类器"], ["c、h", "目标风格条件与低维潜变量"], ["D、Dˆ", "缺陷化学字段集合与学习得到的感官缺陷代理"]], [.23,.77])

H("3 数据审计与预处理")
H("3.1 样本与字段核查", 2)
P("实际附件中 1.csv 有 250 行、234 列，前 3 列为元信息，其余 231 列包括化合物、理化和综合字段；4.csv 含 50 个连续感官目标。化合物条码在感官文件中存在精度损失，直接用条码合并可能产生多对多错配。程序要求 beer 非空、唯一、两表集合相同，并在对齐后检查风格标签一致。")
T(["资料", "实际结构", "研究用途"], [["1.csv", "250×234，含 3 列元信息", "化合物及理化输入"], ["2.csv", "去行名后 231×231", "描述性相关网络"], ["3.xlsx", "12 个 beer 页及 summary", "感官定义与空模板审计"], ["4.csv", "250×53，含 50 个目标", "连续感官监督信息"], ["6.csv", "去行名后 44×50", "跨域关联参考，不作新样本"], ["8.xlsx", "阈值、名称、校准等工作表", "尺度和术语证据"]], [.16,.38,.46])
P("完整原件以 SHA-256 绑定版本。字典、字段尺度、缺失统计和样本键检查随支撑文件保存。题面与文件的数量差异按实际证据记录，不补造缺失行列。8.xlsx 中已有 SHAP 与重要度属于既有研究结果，未作为本文重新训练得到的结论，也不用于第一问的外层变量筛选。")
F("style_counts.png", "图1 风格样本数分布 显示小样本与类别不平衡", 8)
H("3.2 量纲与缺失处理", 2)
P("对可比的原始正值配对，逐字段比较恒等关系与 log10 关系。检查支持 37 个字段在这些配对上符合 log10，包括题面列为可能保留原量纲的部分理化指标；另有字段只在多数配对一致，或缺少可比原值。正值配对一致性并不能识别零替代值与检测限。故保留附件尺度建模，不对所有字段统一逆变换。")
P(f"给定相关矩阵与重算 Spearman 的平均绝对差为 {facts['spearman_matrix_mean_absolute_difference']:.6g}，与 Pearson 为 {facts['pearson_matrix_mean_absolute_difference']:.6g}，更支持采用秩相关解释。矩阵的完整结构一致性也用于核对字段别名次序。自然缺失由训练中位数填补，训练 1% 和 99% 分位定义截尾范围，再作训练均值与标准差标准化。所有操作置于 Pipeline 中[2]。")
E("z_ij = [clip(x_ij; a_j, b_j) − m_j] / s_j", 1)
P("式（1）中的边界、插补值、均值和标准差均只在当前训练数据估计。验证与测试样本仅执行变换，不参与参数学习。对极端值的处理属于可检验的建模假设，而非原始观测纠错。")

H("4 第一问 感官预测与分级不确定性")
H("4.1 评分尺度的可识别性", 2)
P("原始研究明确说明按品评人员分别中心化和标准化，并非仅对汇总后的每个属性作一次统一线性变换[1]。设品评人员 t 的原始评分为 s_it，标准化后为 z_it，若其均值 μ_t 与标准差 σ_t 未知，则不能从 z_it 唯一得到 s_it；不同品评人员的出席与汇总权重进一步增加信息需求。")
E("z_it = (s_it − μ_t) / σ_t，s_it = μ_t + σ_t z_it", 2)
P("对于缺少锚点的数据，多组满足原始评分范围的 μ_t、σ_t 都可能产生相同标准化值。空模板公式返回的零也不等于真实打分。因此本文把可验证的连续预测和有假设的相对分级分开，未把简单拉伸、裁剪或经验分位声称为原始评分恢复。")
H("4.2 多输出预测模型与求解", 2)
P("设第 k 个感官目标的预测为 f_k(x)。比较训练均值、岭回归、PLS、ExtraTrees[3]、固定配置梯度提升回归[4]以及固定比例的岭回归与 ExtraTrees 融合。岭回归通过 L2 惩罚控制高维共线，PLS 学习与目标相关的低维方向，树模型捕捉非线性。输入中不包含编号、条码、风格标签或样本的感官评论信息。")
E("Bˆ = argmin_B {∥Yc − ZB∥²_F + α∥B∥²_F}", 3)
P("岭回归 α 在 {1,10,100,1000} 中由内层三折选择；PLS 成分数在 {3,8} 中选择。ExtraTrees 使用 200 棵树、最小叶样本 3、特征比例 0.8。GBR 对每个目标分别拟合 100 棵深度为 2 的树，学习率 0.04，最小叶样本 5，特征比例 0.7。GBR 参数为第二阶段固定探索配置，未根
```

据本阶段外层目标逐折调整。")
H("4.3 验证设计与性能", 2)
P("采用按风格分层的三折、两次重复外层验证。每个样本每次重复仅被留出一次，预处理和调参在外层训练数据内完成。重复留出预测取均值后计算逐维指标，并对 50 维等权宏平均。最少风格有 3 个样本，选用三折避免强行采用超过该样本数的分层折数。第二阶段是在第一阶段观察之后的探索改进，不能视为未接触过数据的外部验证。")
E("MAE = (1/nq) Σ_i Σ_k |y_ik − ŷ_ik|", 4)
T(["方法", "宏平均 MAE", "宏平均 RMSE", "宏平均 R²"], [[name, f"{metrics.loc[key,'mae']:.4f}", f"{metrics.loc[key,'rmse']:.4f}", f"{metrics.loc[key,'r2']:.4f}"] for key,name in [("mean","均值参照"),("ridge","岭回归"),("pls","PLS"),("extra_trees","ExtraTrees"),("gbr_fixed","固定 GBR"),("fixed_ridge_tree_blend","固定比例融合")]], [.28,.24,.24,.24])
P(f"GBR 的 MAE 为 {gbr:.4f}，相对均值参照下降 {reduction:.2%}。它与 ExtraTrees 的差距较小，不能仅据单个平均值断言普遍优越。宏平均 R² 约 {metrics.loc['gbr_fixed','r2']:.3f}，说明仍有较大未解释变异；逐维性能可能差别明显。上述单位均为附件的连续目标尺度，不能改写成 0–6 分的平均误差。")
F("model_comparison.png", "图2 六类方法在相同重复留出划分下的连续目标误差", 6)
H("4.4 分级敏感性与预测区间", 2)
P("在训练目标上分别构造七等频经验分位、正态分布累计概率和以 5%–95% 分位为端点的线性映射三类等级规则。它们均输出 0 至 6，但语义不同：重复分位边界也会影响离散分级。对每个外层留出预测比较三者，完整保存差异与跨规则范围。")
P(f"三种规则在 {status['grade_sensitivity_disagreement']:.2%} 的预测位置给出不同等级，显示绝对标定对假设高度敏感。跨规则的最小值和最大值仅表示假设范围，不是统计置信区间。若获得参考酒原始评分或逐品评人员转换参数，应重新标定和验证，再向原始 0–6 分作实质性解释。")
P("另用 150 个训练、50 个校准、50 个测试样本展示分割保形预测[5]。用训练模型在校准集的绝对残差定义逐维分位，再为测试预测添加对称区间。名义水平为 90%，目标是单个感官维度的边际覆盖，并非所有 50 维同时覆盖。")
E("r_ik = |y_ik − f_k(x_i)|；r*_k = 第 ceil[(m+1)(1−α)] 个校准残差", 5)
E("C_k(x) = [f_k(x) − r*_k, f_k(x) + r*_k]", 6)
P(f"本次平均测试覆盖率为 {status['conformal_empirical_average_coverage']:.2%}，平均区间宽度为 {coverage.interval_width.mean():.4f}。该结果只对应一次分割和当前连续标签；交换性假设、样本筛选与原作者预处理都会限制其外推。不能把边际覆盖率解释为每种风格或每款啤酒都达到同一保障，更不能据此推导原始等级的覆盖率。")
F("calibration_uncertainty.png", "图3 分级规则分歧与逐维连续预测区间的测试覆盖分布", 6)

H("5 第二问 化合物网络与解释证据")
H("5.1 共表达图及图约束回归", 2)
P("以 231 个数值字段为节点，给定相关矩阵中 |r|≥0.7 时建立边，同时保存符号与绝对权重。当前有 109 条边：0.6、0.7、0.8、0.9 阈值的网络变化另存。模块包含孤立节点，不能直接当作生化通路。对每个目标最终只从化合物或元素字段中选择候选，pH、色度、混合和聚合指标不冒充单一化合物。")
E("W_ij = r_ij I(|r_ij|≥τ)，L = diag(Σ_j |W_ij|) − W", 7)
E("B̂ = argmin_B {∥Yc−ZB∥²_F + α∥B∥²_F + λ tr(BᵀLB)}", 8)
P("符号拉普拉斯的二次型为相关边上系数差异或相反关系的加权惩罚，矩阵为半正定。固定 α=20、λ=5 后，求解 (ZᵀZ+αI+λL)B=ZᵀYc。该模型提供一种利用关系结构的正则化基线，尚未证明固定参数最优。每个目标按标准化特征的系数绝对值取前三项，方向和相关性均保留。")
H("5.2 重采样稳定性与留出置换贡献", 2)
P("采用 100 次样本有放回重采样，每次重新估计截尾、插补、标准化和秩相关网络，重新拟合并计数 Top-3。另比较图约束回归、无图岭回归和边际 Spearman 的名单 Jaccard。相关因素可能相互替代，所以单个因素频率低与模块关联存在并不矛盾。")
E("π_jk = bootstrap 中化合物 j 进入目标 k 的 Top-3 次数 / 重采样次数", 9)
P("第二阶段对每个外层训练拟合的岭回归，在其留出样本上单独置换某一特征，观察逐维 MAE 增量，重复 5 次并汇总六个外层划分。对线性预测利用固定预处理后的变量差与系数精确计算，避免反复拟合同一模型。置换贡献取决于模型与相关结构，不是化合物的内在因果效应[6]。")
E("Δ_jk = MAE_k(置换第 j 个变量后的留出预测) − MAE_k(原留出预测)", 10)
P("定义探索性证据条件：入选频率至少 0.5，平均留出 MAE 增量大于零，且至少三分之二外层划分增量为正。阈值为本研究的描述性判据，不是显著性检验或官方评分标准。由于候选名单包含全数据探索信息，筛选后的支持数仍存在选择适应性，不能作未经选择的独立假设检验。")
P(f"150 条候选中，{status['comparatively_supported_top3_associations']} 条同时满足当前条件。第一阶段有 137 条入选频率低于 0.5，说明多数 Top-3 还不适合写成确定因素。完整 150 条及所有变量的贡献保存在支撑表中，弱证据项和反直觉项不删除。")
selected = top[top.evidence_status == "comparatively_supported_association"]
T(["感官目标", "候选字段", "入选频率", "留出 MAE 增量"], [[r.target,r.field,f"{r.bootstrap_top3_frequency:.2f}",f"{r.heldout_permutation_MAE_increase:.5f}"] for r in selected.itertuples()], [.23,.43,.16,.18])
F("factor_evidence.png", "图4 Top-3 候选的重采样频率与留出置换贡献 通过描述条件仍不表示因果", 6)
H("5.3 化学合理性及反例", 2)
P("4VG 感官目标与 X4VG 的联系具有较强的统计一致性。原始化学研究指出 4VG 与特种啤酒的丁香气味相关，并研究了其陈化降解[7]，可作为该关联的外部合理性依据。金属味与 iron、烘烤麦芽与 methylpyrazine 等候选也可结合附件的名称、来源和缺陷描述继续论证，但本文未开展加标、盲评或反应机理实验。")
P("表中也存在不宜直接解释为气味来源的关联，例如苦味与 iron、乳酸相关描述与某酯类字段。它们可能反映风格共变、共线替代或模型不足。源于同一附件的元数据只能提供术语与来源线索，不能独立验证每个目标的机制。对这些结果应保留待证状态，不能人为换成期望化合物来美化模型。")

H("6 第三问 风格生成与约束迁移")
H("6.1 小样本条件生成与对照", 2)
P("按风格分层留出 20% 样本，训练预处理和生成模型仅使用训练部分。比较三类方法：风格内 bootstrap、同风格近邻之间的局部凸组合，以及十维 PCA 中的条件高斯采样。PCA 模型以风格均值为中心，协方差按 n_c/(n_c+10) 将风格内估计与汇聚估计混合。")
E("h|c ~ N(μ_c，w_cΣ_c + (1−w_c)Σ_pool)，w_c = n_c/(n_c+10)", 11)
P("局部凸组合在相同风格中选择一个训练样本和其近邻，以 0.1–0.9 的随机权重插值。该方法是一种条件经验生成模型，生成的新点不等同于化合物可直接混配的物理过程，尤其在对数尺度上只代表统计插值。")
E("x_new = u x_a + (1−u)x_b，u ~ Uniform(0.1，0.9)，a、b 属于同一风格", 12)
H("6.2 生成验证与结果", 2)
P("用五种随机划分重复比较，每次为 Tripel、Kriek、Lambic 各生成 40 个谱。指标包括对真实非缺失留出测量的平均 KS、训练表示上的有偏 RBF MMD²、目标风格分类概率及精确复制比例。分类器与生成器共享训练集，留出表现用于说明诊断模型能力；生成概率并不是外部独立风格鉴定。五次划分也会重复使用同一批酒。")
T(["风格", "方法", "平均 MMD²", "精确复制比例"], [[r.style,r.method,f"{r.MMD2:.4f}",f"{r.exact_training_copy_fraction:.2f}"] for r in gen.itertuples()], [.18,.38,.24,.20])
P("局部凸组合没有精确复制训练样本，目标风格分类概率较高；但其 MMD 未全面优于直接重采样。PCA 高斯的复杂性也没有带来分布距离优势。因每风格只有极少留出样本，尤其 Kriek 和 Lambic 每次通常只有两款，距离指标不能证明整个分布已被学会。生成模型的评价应同时看接近程度、复制风险与可解释约束，而不能只追求分类器高分。")
F("generator_comparison.png", "图5 五次划分下生成方法的分布距离与内部风格概率 误差线是划分标准差", 6)
P("第一种划分的局部凸组合样本及连续感官画像完整保存。画像来自第一问预先指定的全数据岭回归代理，因此不能用作独立的真实感官验证。五个综合指标随谱联合生成，原作者聚合公式尚未完整恢复，精确代数一致性也未被证明。")
H("6.3 逐项缺陷约束与迁移搜索", 2)

```
P("第一阶段采用 PCA 风格均值方向搜索，11 个源样本中只有 7 个找到非平凡候选。第二阶段扩大为目标风格条件训练锚点搜索：对每个源样本，沿每个 Tripel 训练锚点方向取 30 个权重，再作经验范围投影与四项缺陷字段的逐项上界投影。四项分别为 DMS、双乙酰、4VG 和 trans-2-nonenal。相比单一潜方向，该搜索保留更多目标风格变化方向。")
E("x_candidate,d ≤ x_source,d，d∈{DMS、Diacetyl、4VG、trans-2-nonenal}", 13)
E("f_stale(x_candidate) ≤ f_stale(x_source)", 14)
P("同时要求固定分类器的 Tripel 概率超过原始样本和仅做截尾对照至少 0.01。该阈值是操作性的非平凡变化标准，并非真实风格成功阈值。候选选择按模型概率进行，因此会适应诊断分类器；高概率不能作为优化之后仍独立的验证证据。未找到候选时保留原样并记失败，原样不算成功迁移。")
P(f"扩大搜索后，11 个源样本都找到满足当前模型条件的候选，其中 {int(transfer.all_source_defects_observed.sum())} 个具有四项缺陷源字段实测，另 2 个缺少双乙酰值，只能相对插补值检查；Stale hops 对全部样本都是学习得到的代理。源样本中 2 个属于生成器留出集，其余为训练来源。增加搜索范围带来更多模型可行候选，但没有增加真实品评证据。")
F("transfer_comparison.png", "图6 扩展搜索与单一 PCA 方向搜索的内部分类概率比较 缺失源值另作标注", 6)
P("DMS 与其他缺陷的单位并不统一，部分尺度和 trans-2-nonenal 阈值也未确定。本文因此不声称全部绝对 OAV 阈值已通过。测量字段逐项不增加是较保守的模型条件，但若来源值有检测限替代、缺失或尺度歧义，仍需逐项补证。下一步应恢复基本成分的浓度和聚合公式，加入发酵过程与酿造可行性，再以盲评检验感官效应。")

H("7 稳健性 可复现性与模型评价")
H("7.1 已完成的稳健性检查", 2)
P("本流程分别保存了分层重复留出、训练折调参、额外随机缺失、网络阈值变化、bootstrap 网络重建、跨方法排名、留出置换、分级规则变化和生成划分变化。第一阶段与第二阶段分目录存储，保留方法变化前后的成绩和失败样本。23 项程序检查覆盖主键对齐、原件未改、预处理隔离、符号拉普拉斯性质、区间残差排序、留出角色、逐项缺陷和失败标记；这些检查证明实现满足指定性质，不证明全部科学假设成立。")
H("7.2 优点与适用范围", 2)
P("流程的主要价值是把尺度审计、预测验证、关联证据和生成约束分开评估。简单参照使性能提升可比较，弱因素与失败案例保留使解释不过度确定，模型内约束也明确限定在可观测字段或代理。对类似小样本、高维、混合量纲的风味研究，这些步骤比单纯增加模型复杂度更有实际参考价值。")
H("7.3 局限与改进方向", 2)
P("其一，原始逐人评分与绝对锚点缺失使真实 0–6 分不可恢复；分级规则高度分歧，不能把假设等级包装成真值。其二，噪声、检测限、品评差异和有限样本限制预测能力，且源数据预处理范围未完全明确。其三，Top-3 多数不稳定，反直觉项缺少因果证据；图约束参数与解释阈值也需要更系统敏感性。")
P("其四，生成验证的稀有风格留出过少，分类器共享训练信息且会被迁移搜索适应；重复分割不能代替新数据。其五，聚合量公式、物理约束、缺陷阈值和感官实验未完成。其六，第二阶段已经基于第一阶段经验设计，缺少完全封存的最终外部测试。因此本文定位为可复核的参考研究，后续应以新增测量、品评锚点和独立实验补充关键证据。")

H("8 结论与建议")
P(f"在当前附件尺度上，化合物对感官画像具有一定预测信息：固定配置 GBR 的连续宏平均 MAE 为 {gbr:.4f}，优于均值参照；但原始 0–6 分缺少可识别条件，不应声称已恢复。加入训练、校准和测试隔离的区间示例，可补充连续预测的不确定性表达，仍需遵守边际覆盖及交换性限制。")
P(f"化合物网络可组织共变关系并提供正则化信息，但 150 条 Top-3 仅 {status['comparatively_supported_top3_associations']} 条满足当前相对支持判据，说明稳定解释和化学机制是后续重点。生成比较显示局部凸组合在非复制与风格诊断之间有实用折中，复杂高斯模型尚未表现出全面优势。扩展搜索扩大了模型内可行迁移范围，却不能替代真实浓度阈值、酿造过程或品评验证。")
P("后续研究优先顺序为：获取原始评分或参考酒锚点；复核尺度、检测限与异常；对关键因素开展浓度和文献核查；用封存的新样本验证生成与迁移；最后据实更新论文与 AI 使用记录。")

H("参考文献")
references = [
"[1] Schreurs M, Piampongsant S, Roncoroni M, et al. Predicting and improving complex beer flavor through machine learning. Nature Communications, 15:2368, 2024. DOI:10.1038/s41467-024-46346-0.",
"[2] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12:2825-2830, 2011.",
"[3] Geurts P, Ernst D, Wehenkel L. Extremely randomized trees. Machine Learning, 63:3-42, 2006. DOI:10.1007/s10994-006-6226-1.",
"[4] Friedman J H. Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5):1189-1232, 2001. DOI:10.1214/aos/1013203451.",
"[5] Lei J, G'Sell M, Rinaldo A, et al. Distribution-Free Predictive Inference for Regression. Journal of the American Statistical Association, 113(523):1094-1111, 2018. DOI:10.1080/01621459.2017.1307116.",
"[6] Scikit-learn developers. Permutation Importance with Multicollinear or Correlated Features. https://scikit-learn.org/stable/auto_examples/inspection/plot_permutation_importance_multicollinear.html，访问日期：2026-10-24.",
"[7] Vanbeneden N, Saison D, Delvaux F, Delvaux F R. Decrease of 4-vinylguaiacol during beer aging and formation of apocynol and vanillin in beer. Journal of Agricultural and Food Chemistry, 56(24):11983-11988, 2008. DOI:10.1021/jf8019453.",
"[8] OpenAI Codex，GPT-6 家族（精确可公开版本待核验），OpenAI，使用日期：2026-10-24."
]
for reference in references: P(reference)
H("AI 工具使用声明")
P("本参考研究使用 OpenAI Codex 辅助进行资料检索、代码实现、数据审计、实验记录、图表和稿件编写[8]。正文所述计算为 AI 辅助代码的实际运行结果，不能据此声明核心工作已由参赛队员独立完成。队员后续应独立理解、验证、修改并记录实际贡献；采纳内容须按赛事要求在相应位置标注。模型家族信息可由会话核验，精确版本应按实际界面补充，不臆造版本号。重要交互与采纳修改记录见随附 AI 使用阶段记录，最终提交前须补全规定的 PDF 说明。")
H("附录 复现与结果索引")
P("第一阶段运行 run.py all，第二阶段运行 run_stage2.py，配置分别为 config.json 和 config_stage2.json。原始附件由使用者自行取得并经 source_manifest.csv 核对哈希。Python 与依赖版本锁定在 requirements-lock.txt；生成论文的脚本位于 scripts/，原始数据未随公开仓库分发。")
T([["材料", "路径与内容"], [["数据证据", "outputs/stage2/audit/：字典、原件哈希、尺度和模板检查"], ["第一问", "outputs/stage2/q1/：连续误差、GBR 留出预测、分级敏感性和区间覆盖"], ["第二问", "outputs/stage2/q2/：完整置换贡献与 150 条候选证据；原始网络见 stage1/q2"], ["第三问", "outputs/stage2/q3/：生成对照、局部样本、画像、迁移与缺陷检查"], ["研究源代码", "src/、run.py、run_stage2.py；算法与完整运行代码随支撑包提供"], ["入门教程", "docs/从数据到论文的解题教程.md；按数据、模型、验证与论文顺序阅读"]], [.20,.80])
P("本稿未逐页排入全部源代码，也未填写队号、承诺书和队员独立贡献。若转为正式参赛作品，应依据官方要求把全部实际使用源程序纳入附录，并完成匿名、页数、引用与 AI 披露检查。参考稿中的条件、失败结果和未证实事项不得在修改时无依据删除。")

title = "基于尺度审计与稳定关联的啤酒感官预测及风格约束迁移"
```

```
payload = {"title":title, "abstract":abstract, "keywords":"感官预测；数据尺度；图约束回归；重采样稳定性；条件生成；约束迁移", "blocks":blocks}
(PAPER / "paper_content.json").write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
md = ["# "+title,"","## 摘要",abstract,"","关键词：  "+payload["keywords"],"","## 目录","见 PDF 的自动生成目录。正文以下各节与实际结果文件对应。","",""]
for b in blocks:
    if b["type"]=="heading": md += ["#"*(b["level"]+1)+" "+b["text"],""]
    elif b["type"]=="paragraph": md += [b["text"],""]
    elif b["type"]=="equation": md += [f"公式（{b['number']}）：{b['text']}",""]
    elif b["type"]=="table":
        md += ["| "+" | ".join(b["headers"])+" |","|"+"---|"*len(b["headers"])]
        md += ["| "+" | ".join(map(str,row))+" |" for row in b["rows"]]
        md += [""]
    elif b["type"]=="figure": md += [f"![{b['caption']}]({b['path']})",""]
(PAPER / "论文参考稿.md").write_text("\n".join(md), encoding="utf-8")
print(json.dumps({"blocks":len(blocks),"characters":len('\n'.join(md)),"paper":str(PAPER/'论文参考稿.md')},ensure_ascii=False))
```

源程序 18 scripts/prepare_revised_paper.py

```
"""Build a result-backed revision and transparent AI-use stage record.
AI-assisted, OpenAI Codex, 2026-10-25. No planned result is written as observed.
"""
from pathlib import Path
import hashlib
import json
import pandas as pd

ROOT=Path(__file__).resolve().parents[1]; OUT=ROOT/'outputs/stage3'; PAPER=ROOT/'paper/revised_20261025'; PAPER.mkdir(parents=True,exist_ok=True)
for part in ['q1','q2','q3']:
    if not json.loads((OUT/part/'status.json').read_text())['complete']: raise ValueError('Incomplete experiment '+part)
metrics=pd.read_csv(OUT/'q1/metrics_by_scope.csv'); graph=pd.read_csv(OUT/'q2/graph_comparison.csv'); top=pd.read_csv(OUT/'q2/top3_with_evidence.csv')
gen=pd.read_csv(OUT/'q3/generation_summary.csv'); transfers=pd.read_csv(OUT/'q3/transfer_independent_evaluation.csv'); coverage=pd.read_csv(OUT/'q1/conformal_coverage.csv')
gstatus=json.loads((OUT/'q2/status.json').read_text()); roles=pd.read_csv(OUT/'q3/disjoint_roles.csv'); diagnostics=pd.read_csv(OUT/'q3/real_holdout_diagnostics.csv')
checks=json.loads((OUT/'validation_summary.json').read_text()); tests=checks['tests']['tests']
def mae(model,scope): return float(metrics[(metrics.model==model)&(metrics.scope==scope)].mae.iloc[0])
etcore=mae('extra_trees','core30'); etall=mae('extra_trees','all50'); blendcore=mae('shrunk_target_blend','core30'); blendall=mae('shrunk_target_blend','all50')
r1=transfers[transfers.radius==1]; ht=r1[r1.source_role=='T_real_holdout']; success=int(ht.evaluator_proxy_acceptance.sum()); supported=int(top.evidence_status.str.startswith('comparatively').sum())
blocks=[]
def H(s,level=1): blocks.append({'type':'heading','level':level,'text':s})
def P(s): blocks.append({'type':'paragraph','text':s})
def E(s,n): blocks.append({'type':'equation','text':s,'number':sum(b['type']=='equation' for b in blocks)+1,'asset_number':n})
def T(headers,rows,widths=None): blocks.append({'type':'table','headers':headers,'rows':rows,'widths':widths})
def F(name,caption,height=7): blocks.append({'type':'figure','path':'outputs/stage3/figures/'+name,'caption':caption,'height_cm':height})
references={
 'source':'Schreurs M, Piampongsant S, Roncoroni M, et al. Predicting and improving complex beer flavor through machine learning. Nature Communications, 15:2368, 2024. DOI:10.1038/s41467-024-46346-0.',
 'et':'Geurts P, Ernst D, Wehenkel L. Extremely randomized trees. Machine Learning, 63:3-42, 2006. DOI:10.1007/s10994-006-6226-1.',
 'gbr':'Friedman J H. Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5):1189-1232, 2001. DOI:10.1214/aos/1013203451.',
 'stack':'Breiman L. Stacked Regressions. Machine Learning, 24:49-64, 1996. DOI:10.1007/BF00117832.',
 'graph':'Li C, Li H. Variable selection and regression analysis for graph-structured covariates with an application to genomics. The Annals of Applied Statistics, 4(3):1498-1516, 2010. DOI:10.1214/10-AOAS332.',
 'cp':"Lei J, G'Sell M, Rinaldo A, et al. Distribution-Free Predictive Inference for Regression. Journal of the American Statistical Association, 113(523):1094-1111, 2018. DOI:10.1080/01621459.2017.1307116.",
 'sklearn':'Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12:2825-2830, 2011.',
 'cv':'Bengio Y, Grandvalet Y. No Unbiased Estimator of the Variance of K-Fold Cross-Validation. Journal of Machine Learning Research, 5:1089-1105, 2004.',
 '4vg':'Vanbeneden N, Saison D, Delvaux F, Delvaux F R. Decrease of 4-vinylguaiacol during beer aging and formation of apocynol and vanillin in beer. Journal of Agricultural and Food Chemistry, 56(24):11983-11988, 2008. DOI:10.1021/jf8019453.',
 'iron':'Omur-Ozbek P, Dietrich A M, Duncan S E, Lee Y. Role of lipid oxidation, chelating agents, and antioxidants in metallic flavor development in the oral cavity. Journal of Agricultural and Food Chemistry, 60(9):2274-2280, 2012. DOI:10.1021/jf204277v.',
 'coriander':'Pater A, Satora P, Januszek M. Effect of Coriander Seed Addition at Different Stages of Brewing on Selected Parameters of Low-Alcohol Wheat Beers. Molecules, 29(4):844, 2024. DOI:10.3390/molecules29040844.',
 'mmd':'Gretton A, Borgwardt K M, Rasch M J, et al. A Kernel Two-Sample Test. Journal of Machine Learning Research, 13:723-773, 2012.',
 'ai':'OpenAI. Codex（系统提示模型家族 GPT-6，精确可公开版本待实际界面核验），使用日期：2026-10-24 至 2026-10-25。重要交互、采纳与未完成的人工作业见随附 AI 工具使用详情说明阶段版。',
```

```
}
order=[]
def C(key):
    if key not in order: order.append(key)
    return '['+str(order.index(key)+1)+']'

H('1 问题重述与研究目标')
P(f'本题要求由化合物测量给出感官评分、逐维 Top-3 及稳定性和化学解释，并按风格生成化合物谱和完成缺陷不恶化的迁移。与附件高度吻合的原始研究已用化学资料预测感官并开展加标试验{C("source")}；其数据和已有发现是研究背景，不计作本研究新增贡献。')
P('本文以实际 250 款啤酒、231 个数值输入和 50 个监督目标为基础。保留全部目标输出，另把 A_ 与 F_ 前缀的 30 个核心维度作为预先明确的主要比较范围，其他 20 项按评分卡章节补充报告。这个范围选择不代表官方只要求 30 项，也不把题面“约 40”强行解释为一个与附件不符的固定数量。')
P('三项研究目标分别为：明确连续标签与 0-6 等级的可解释范围；检验网络和候选因素的预测信息是否稳定；把统计可行的风格调整与独立模型支持、真实化学和品评证据区分。无法获得的新测量不以模型输出补造。')

H('2 相关工作、贡献边界与模型假设')
P(f'ExtraTrees{C("et")}、梯度提升回归{C("gbr")}及利用交叉验证预测学习组合权重的 stacking{C("stack")}均为成熟方法。图拉普拉斯约束回归已有系统研究{C("graph")}，分割保形预测也已有理论和方法{C("cp")}。本文采用这些方法并作任务化组合，不主张首次发明基础算法、收缩融合、Pareto 优化或新的保形理论。')
P('本文的可核查贡献定位于应用建模和证据评估：逐字段尺度及评分位置核查；在明确目标范围内对标准模型及收缩融合公平比较；加入无图和打乱图对照，检验网络的额外价值；分离生成搜索资料与评价资料，并如实报告代理判断分歧和失败。贡献是否成立由具体实验决定，不要求所有复杂设计获胜。')
P('假设一，beer 唯一标识相同酒，跨表正确对应，经集合、唯一性和风格一致性支持。假设二，训练内插补、截尾和标准化提供可检验的统计处理，但不证明极端值错误或缺失随机。假设三，相关网络只提供共变结构，系数不是因果效应。假设四，风格内统计插值与经验支持约束不充分保证酿造可行。假设五，覆盖解释依赖相应交换性与测量条件，不能自动推广到自适应搜索后的分布外样本。')
T(['符号','定义'],[['X、Y','231 个输入与 50 个连续目标；生成主模型使用 226 个非聚合字段'],['Z、B、L','训练预处理后的输入、回归系数及有符号图拉普拉斯'],['w_j、η','目标 j 的 GBR 权重及向全局权重收缩的惩罚'],['g、[L_y,U_y]','固定训练分位等级映射与连续预测区间'],['G、J、T','生成/搜索训练组、独立评价训练组、真实留出组'],['r、D','标准化改动半径和四项指定化学缺陷字段集合']],
[.23,.77])

H('3 数据审计与处理')
H('3.1 对齐、字段与评分卡核查',2)
P('实际 1.csv 为 250×234，含 3 列元信息；4.csv 含 50 个目标。beer_id 在感官文件精度受损，不作为主键。程序按 beer 对齐并核对风格；原件以 SHA-256 绑定版本，未修改附件。2.csv 与重算 Spearman 的平均绝对差约 1.58×10^-16，而与 Pearson 约差 0.0873，因此按秩相关解释。')
P('评分卡 summary 与监督目标的三个显式别名为 A_ester_fruity→A_esters_fruity、F_ester_fruity→F_esters_fruity、4vg→X4vg。50 个目标均已对应。评分卡另有 Aroma、Flavour 乙醛位置而没有对应的独立监督列，保留缺标签说明，未补零。12 处 Barnyard 汇总引用 AG76，表单标签却对应 AG90，列为疑似源公式问题；这不自动证明 4.csv 继承该问题。')
T(['资料或范围','数量/用途','处理决定'],[['核心 Aroma / Flavour','15+15 个前缀目标','主比较，全部保留输出'],['其他感官目标','20 个','口感、缺陷、香料、Lambic 与总体等另报'],['输入字段','231 个','预测阶段保持附件尺度'],['生成主输入','226 个非聚合描述字段','五个未核实综合量不独立生成'],['3.xlsx 原评分卡','12 张 beer 页及 summary','输入空白和公式缓存零不作为评分真值']],
[.25,.29,.46])
H('3.2 尺度、缺失和异常处理',2)
P('逐列比对可比的原始正值配对，有 37 个字段支持 log10 关系；其他字段存在多数恒等、少数不匹配或缺参考。正值匹配不证明零替代值、检测限或全部量纲均已恢复，因此不把全表统一反对数。DMS、4VG 的正值变换证据与双乙酰、trans-2-nonenal 的不匹配另列。')
P(f'所有预测预处理只在当前训练部分估计：1%/99% 截尾、中位数插补、均值和标准差标准化，使用 Pipeline 管理{C("sklearn")}。验证和测试只执行已拟合变换。去截尾、2.5% 截尾和模拟缺失作为消融，不能把被截值称为错误测量。')
E('z_ij = [clip(x_ij;a_j,b_j)-m_j]/s_j',1)

H('4 第一问：连续预测、相对分级与不确定性')
H('4.1 绝对评分的信息缺口与交付定义',2)
P('原始研究按品评人员中心化和标准化。若逐人均值、标准差、出席情况及汇总权重缺失，同一聚合连续值可能对应多组原始评分，无法唯一反演真实 0-6 分。空白评分卡不能提供锚点。本文不把经验映射包装成真实品评等级。')
E('z_it=(s_it-μ_t)/σ_t，s_it=μ_t+σ_t z_it',2)
P('实际交付包括全 50 维连续留出预测和带 ASSUMPTION 标记的相对 0-6 等级。相对 0 是训练参考的最低一档，不等于真实未检出；相对 6 也不等于真实压倒性。若题目要求真实绝对等级，仍需原始锚点或转换参数；本文诚实保留这一未解决的测量信息条件。')
H('4.2 模型与严格训练内融合',2)
P('比较均值、岭回归、ET、GBR、固定半数 ET/GBR、全局权重、无收缩逐维权重和收缩逐维权重。ET 200 棵树、最小叶样本 3、特征比例 0.8；GBR 每目标 100 棵深度 2 的树，学习率 0.04、最小叶样本 5、特征比例 0.7。岭回归 α 在 {1,10,100,1000} 中以内层核心 MAE 选择；元信息和风格不作为第一问预测输入。')
E('B_hat=argmin_B ||Yc-ZB||²_F+α||B||²_F',3)
P('融合权重的具体目标是内层留出预测的均方误差加向全局权重的二次惩罚，最终 η 由训练内验证的核心 MAE 选择。设 d=p_GBR-p_ET、e=y-p_ET，得到受限闭式权重：')
E('w_j=clip{[mean_i(d_ij e_ij)+ηw_0]/[mean_i(d_ij²)+η],0,1}',15)
P('η∈{0,0.01,0.1,1,10}。对每个外层训练集，再分 S/V：只在 S 内三折产生基础预测并拟合权重，重训 S 后预测 V、选择 η。V 标签不进入 S 内拟合。选定 η 后在全部外层训练内重新产生 OOF、学习权重，重训基础模型后预测外层测试。该嵌套结构避免直接在外层测试上拼接最优维度。')
H('4.3 同范围验证与实际结果',2)
P(f'按风格三折、五次重复，共 15 个外层划分；内层使用三折 KFold。每个样本每次恰被留出一次，所有方法共享外层角色。报告先对每个 beer 的五次预测平均、再计算逐维宏指标的聚合 OOF 成绩；它不同于未来单次拟合模型的误差，单折成绩另存。重复折共享资料，不能当独立样本简单作 t 检验{C("cv")}。')
E('MAE=(1/nq) Σ_i Σ_j |y_ij-y_hat_ij|',4)
display=[('mean','均值'),('ridge','岭回归'),('extra_trees','ET'),('gbr','GBR'),('fixed_half_blend','固定半数融合'),('global_oof_blend','训练内全局融合'),('unshrunk_target_blend','无收缩逐维融合'),('shrunk_target_blend','收缩逐维融合')]
T(['方法','核心30维 MAE','全50维 MAE'],[[name,f'{mae(key,"core30"):.5f}',f'{mae(key,"all50"):.5f}'] for key,name in display],[.44,.28,.28])
P(f'ET 的核心 MAE 为 {etcore:.5f}，全维为 {etall:.5f}；收缩融合分别为 {blendcore:.5f}、{blendall:.5f}。核心相对 ET 的变化为 {(blendcore/etcore-1):+.2%}。固定半数融合核心 MAE 为 {mae("fixed_half_blend","core30"):.5f}，略优于收缩方案：当前没有证明更复杂的权重学习必要。本文不把全维平均最优推导为核心维度最优。主交付的相对等级与区间采用事先固定的 ET 基线，复杂融合仅在其实际收益和代价范围内评价。')
F('prediction_by_scope.png','图1 相同外层划分的聚合留出误差：核心30维与全50维分开报告',7)
```

```
P('配对比较按 beer 重采样，保持同一酒两模型的误差配对及风格构成。2000 次固定 OOF 条件重采样范围只反映保存预测的样本敏感性，不包含训练和选模不确定性。完整差值、范围、逐维 R² 及失败目标随结果表提供，未把条件范围当作总体算法优势的严格证明。')
F('blend_weights.png','图2 权重完全在外层训练内部学习，反映目标异质性而非外层成绩事后挑选',5.8)
H('4.4 固定相对分级与预测集合',2)
P('每个训练参考的六个边界取 k/7 分位。固定边界定义 g(y)，对留出连续预测给出相对等级。全 250 款酒的交付表使用五次留出等级众数；各折参考稍有差别，因此该表是相对画像，不是共同绝对强度量表。')
E('g(y)=Σ_k I(y>t_k)，G(x)={g(L_y(x)),…,g(U_y(x))}',16)
P('另作五次 150 训练、50 校准、50 测试的分割保形示例。ET 和等级边界只在 150 个训练样本拟合；50 个校准残差的第 ceil[51×0.9]=46 顺序统计量给出逐维连续区间，再映射为相对等级集合包络。重复边界可能让包络包含不可达的中间等级，但不缩窄包络。')
E('r_ij=|y_ij-f_j(x_i)|；q_j=第 ceil[(m+1)(1-α)] 个校准残差',5)
E('C_j(x)=[f_j(x)-q_j,f_j(x)+q_j]',6)
P(f'五次分割、50 个目标的平均连续测试覆盖为 {coverage.continuous_coverage.mean():.2%}，平均区间宽 {coverage.interval_width.mean():.4f}；固定规则下的相对等级集合覆盖为 {coverage.relative_grade_set_coverage.mean():.2%}，平均集合大小 {coverage.mean_grade_set_size.mean():.2f}。这不是原始等级准确率或 50 维同时覆盖保证；同一批酒重复出现也不构成五套外部数据。')
P('平均集合接近全部七档，说明目前很难在可靠性要求下细分感官等级；较高集合覆盖伴随宽集合，不能解读为七级预测很精确。细化等级需要更好的测量锚点、更多样本或更小的连续预测误差。')
F('ordinal_uncertainty.png','图3 五次分割的逐维覆盖与相对等级集合大小；真实绝对评分仍未标定',6.2)
grades=pd.read_csv(OUT/'q1/ASSUMPTION_oof_relative_grades.csv').head(3)
fields=['A_malt_all','A_hops_all','A_esters_all','F_malt_all','F_hops_all','F_esters_all']
T(['beer','麦芽香','酒花香','酯香','麦芽味','酒花味','酯味'],[[str(int(r.beer))]+[str(int(r[f])) for f in fields] for _,r in grades.iterrows()],[.13]+[.145]*6)
P('上表只展示全量输出中的三个样本和六个整体属性，数值均为相对档位；完整 250×50 等级、连续预测和区间角色保存在支撑文件。真实评分锚点获取路径未执行，不虚报绝对等级准确率。')
H('4.5 截尾、缺失压力与失败分析',2)
stress=pd.read_csv(OUT/'q1/missingness_stress.csv'); clip=pd.read_csv(OUT/'q1/clipping_ablation.csv')
st=stress[stress.model=='extra_trees'].groupby(['scenario','extra_missing_rate']).core30_mae.mean().reset_index()
T(['模拟缺失情境','额外比例','ET核心 MAE'],[[r.scenario,f'{r.extra_missing_rate:.0%}',f'{r.core30_mae:.5f}'] for r in st.itertuples()],[.52,.18,.30])
P('低值优先遮蔽使训练 20% 分位以下的观测被遮蔽权重为其他值的四倍；它是检测限相关缺失的压力情境，不是对真实缺失机制的确认。每种情境中 ET 与融合使用完全相同的遮蔽位置。截尾对照使用 ET，未把它冒充全部方法都完成的消融。')
P('截尾比例 0、1%、2.5% 的 ET 单折核心 MAE 均值依次为 '+ '、'.join(f'{v:.5f}' for v in clip.groupby('lower_clip').core30_mae.mean())+'。额外缺失表为逐折误差均值，与前述重复预测平均后的误差不是同一种汇总，不能混作同一成绩提升。')

H('5 第二问：网络消融、稳定因素与化学证据')
H('5.1 描述图与训练折图',2)
P('按给定秩相关矩阵建立 231 节点的描述图，|r|≥0.7 时有 109 条边。预测验证中的网络必须由当前训练数据重算，不能直接使用含外层样本信息的全数据图。符号拉普拉斯及图约束回归定义如下；图用于正则化，不是反应通路。')
E('W_ij=r_ij I(|r_ij|≥τ)，L=diag(Σ_j|W_ij|)-W',7)
E('B_hat=argmin_B {||Yc-ZB||²_F+α||B||²_F+λ tr(B^T L B)}',8)
P('α∈{1,10,100,1000}，λ∈{0,1,5,20}，非零 λ 时 τ∈{0.6,0.7,0.8}，全部在内层以核心 MAE 选择。无图基线使用相同 α 候选；负对照将训练图节点标签随机置换，保留无标签图的边权、符号和度分布。每个拟合使用 10 个随机图，汇总各图的误差而非平均预测形成额外集成。')
H('5.2 网络的额外收益检验',2)
T(['模型','核心30维单折 MAE','全50维单折 MAE'],[[{'none':'无图','real':'真实图','shuffled':'打乱图'}[k],f'{graph[(graph.graph==k)&(graph.scope=="core30")]["mean"].iloc[0]:.6f}',f'{graph[(graph.graph==k)&(graph.scope=="all50")]["mean"].iloc[0]:.6f}'] for k in ['none','real','shuffled']],[.38,.31,.31])
P(f'真实图与无图误差几乎相同，全数据内层选择 α={gstatus["full_graph_params"]["alpha"]}、λ={gstatus["full_graph_params"]["lambda"]}。因此当前数据和网格不支持“图正则化改善预测”的结论。网络仍可组织共变关系，但最终参数退化到普通岭回归时，Top-3 不能归功于图惩罚。该表是单折均值，不能与第一问聚合 OOF 数值直接比较来判断模型优劣。')
F('graph_ablation.png','图4 有图、无图和打乱图的对照及内层选择：真实图未显示实用预测收益',6.8)
H('5.3 跨拟合贡献、风格控制与模块替代',2)
P('每个外层训练内选择每目标三个化合物/元素候选，排除 pH、色度及聚合字段。留出贡献用冻结模型的特征置换 MAE 增量评价，重复 20 次；另在相同风格内置换，以及联合置换整个相关模块、删去模块重训。风格内置换不是完整条件于全部化合物的严格条件置换，模块与单变量贡献也不是因果效应。')
E('π_jk=bootstrap 中入选 Top-3 次数 / 重采样次数',9)
E('Δ_jk=MAE_k(置换后的留出预测)-MAE_k(原留出预测)',10)
P(f'全数据描述模型的 200 次 bootstrap 每次重建预处理与图，但固定全数据选定的超参数，所以频率是该参数条件下的稳定性。当前描述性支持条件为频率≥0.5、至少 5 个外层训练入选、这些入选折的平均普通与风格内贡献均为正，共 {supported}/150 条。它与前阶段 11 条采用的模型和判据不同，不能把数量变化解释为直接的可靠性提升；筛后汇总仍有选择适应性，不作独立显著性检验。')
F('factor_crossfit.png','图5 150 条最终候选的条件重采样稳定性及风格内留出贡献：正贡献不代表因果',6.4)
H('5.4 化学合理性、证据层级与反例',2)
P(f'X4vg 与 X4VG 为较一致的候选。已有啤酒陈化研究观察 4VG 降低并讨论丁香向香草印象变化{C("4vg")}，可支持气味合理性，但不是本数据因果验证。iron 与 metallic 的关系有铁离子诱发口腔脂质氧化的实验依据{C("iron")}，该研究使用金属溶液且不同价态反应不同，因此不能直接给本附件总 iron 套用统一啤酒阈值。')
P(f'coriander 与 linalool 的关联可结合低醇小麦啤酒添加芫荽籽的研究核查；该研究的 GC-O 结果指出芫荽籽中 linalool 等组分有较高气味活性{C("coriander")}。这只支持机制相容性，不能证明本附件 coriander 标签由 linalool 单独造成。酸度与乳酸、醋酸以及酒精感与乙醇可作测量相容性的检查，其余未知联系不补写机理。')
T(['关联示例','可支持的解释','仍缺什么'],[['X4vg / X4VG','啤酒中丁香相关香气有原始研究支持','本批次加标或独立测量'],['metallic / iron','金属离子可诱发口腔金属味机制','价态、基质、浓度和本附件因果'],['coriander / linalool','芫荽籽与啤酒研究的香气依据','标签特异性与浓度验证'],['bitternes / iron','统计贡献可能反映风格或共变','不能据此称铁是苦味的直接来源'],['clove / linalool','可重复关联但不等于典型丁香来源','共线替代与特异性证据']], [.26,.40,.34])
P('完整 150 条结果附方向、频率、入选折数、风格内贡献、单位和附件气味描述。独立文献只支持少量明确案例，其余化学解释逐条标为未充分核实；不以统计支持状态冒充机制通过。论文附录逐维给出 Top-3，弱项和反例均保留。')

H('6 第三问：受约束生成与独立迁移诊断')
H('6.1 三组隔离与226字段模型',2)
P('每个种子将资料按风格分层为 G=100、J=100、T=50。G 拟合生成器、Logistic 风格搜索器、岭感官代理和 ET 感官一致性代理；J 拟合不参与搜索的 ET 风格评价器及另一感官代理；T 只评价真实样本诊断能力。每次 G/J/T 不重叠，但所有酒在前两阶段研究中均已出现，因此只是本次拟合角色隔离，不是新增外部验证。')
P('五个未核实综合量不独立生成，生成器、搜索器与评价器全部重训为相同 226 个非聚合字段。字段仍含理化描述量，不能当作 226 种可任意混合的纯物质；未核实综合量输出为待核实，而不是补造数值。候选及顺序先固定并计算哈希，再拟合 J 评价模型，J/T 结果不用于重新挑候选。'
```

```
)
P(f'五次真实 T 诊断中，搜索器与独立评价器的平均平衡准确率分别为 {diagnostics.searcher_holdout_balanced_accuracy.mean():.3f}、{diagnostics.evaluator_holdout_balanced_accuracy.mean():.3f}。因此独立评价者本身也并非完美风格鉴定。两类模型的概率未经共同校准，不能仅以概率数值高低比较其可靠性；另报目标类别预测比例和代理约束。')
H('6.2 生成方法与相同核带宽比较',2)
P('比较风格内 bootstrap、局部凸组合、十维条件 PCA 高斯及受约束局部凸组合，每种风格每种方法每划分生成 40 个样本。PCA 协方差按 n_c/(n_c+10) 部分汇聚。局部凸组合避免直接复制，但不等于物理混配。')
E('h|c~N(μ_c,w_cΣ_c+(1-w_c)Σ_pool)，w_c=n_c/(n_c+10)',11)
E('x_new=u x_a+(1-u)x_b，u~Uniform(0.1,0.9)',12)
P('受约束版本在同风格最近三个邻居间取权重；DMS 和 4VG 按已支持的正值 log10 关系先回到正值域插值再返回，其他字段仍在附件尺度统计插值。检测限替代值的解释不完整，因此前者也只是有证据条件的建模变换。投影训练范围后，以同风格训练近邻距离的95%分位限制联合支持；每组最多尝试 40×250 次，失败不伪装为新样本。')
P(f'MMD²{C("mmd")} 的表示和核带宽只用 G 训练资料确定，所有方法共用；真实参考为 T 的非缺失测量，另报 KS、复制率、J 模型目标类别判断及范围违反。风格留出常只有两款，不能以小距离或不显著证明分布一致。')
T(['风格','方法','MMD²','J目标类别比例'],[[r.style,{'bootstrap':'重采样','local_convex':'局部凸组合'
,'PCA_Gaussian':'PCA高斯','constrained_local_convex':'受约束凸组合'}[r.method],f'{r.MMD2_fixed_training_bandwidth:.4f}',f'
{r.independent_evaluator_target_prediction_fraction:.1%}'] for r in gen.itertuples()],[.15,.35,.23,.27])
P('受约束方法在已检查的训练范围内违反率为零，未精确复制训练点；但 MMD 未全面优于重采样，且与普通局部凸组合各有取舍。J 的类别判断也不能作为生成酒真实标签。完整生成谱和感官代理画像保存于支撑文件，未将其称为重新测得的感官结果。')
F('generation_independent.png','图6 固定训练核带宽下的分布比较及未参与搜索的J模型判断，五次划分平均',7)
H('6.3 信赖域、Pareto选择与逐项缺陷',2)
P('迁移沿源样本与 Tripel 的 G 训练锚点各取30个权重。候选投影范围后逐项限制 DMS、双乙酰、4VG、trans-2-nonenal 不增加，并同时要求岭与 ET 两个 G 感官代理的 stale_hops 不增加。搜索 Tripel 概率至少0.5，且超过原样和仅截尾对照至少0.01；同时满足目标经验支持与指定改动半径。')
E('x_candidate,d≤x_source,d，d∈D',13)
E('f_stale(x_candidate)≤f_stale(x_source)',14)
E('||Z_candidate-Z_source||₂/√p≤r，r∈{0.25,0.5,1,2}',17)
P('排除违反硬约束的候选后，对标准化改变量、目标训练支持距离、DMS/4VG正值变换下的阈值代理超额作 Pareto 筛选，再从非支配候选选择最小改动者。19 ug/L 与0.2 mg/L 来自附件中位阈值，仅作为这两项正值代理，不替代逐样本浓度和检测限核验。trans-2-nonenal 阈值缺失；双乙酰等配对不匹配未完全解释，所以没有任何候选获“完整绝对化学认证”。')
P('源缺陷缺失时仅以训练中位数参与条件搜索并单独标记，不能作真实源上界。源值非缺失也不表示已验证浓度、基质或感官阈值。找不到非平凡候选时保留原样且记失败；原样满足不增加不计作迁移成功。')
rows=[]
for radius,a in transfers.groupby('radius'):
    held=a[a.source_role=='T_real_holdout']; rows.append([str(radius),f'{(a.status!="no_feasible_nontrivial_candidate_source_retained").sum()}/{len(a)}',f'{a.evaluator_proxy_acceptance.sum()}/{len(a)}',f'{held.evaluator_proxy_acceptance.sum()}/{len(held)}'])
T(['改动半径','搜索非平凡可行','全部记录J代理支持','T源记录J代理支持'],rows,[.18,.25,.29,.28])
P(f'半径1的搜索可行记录中，{int((r1.status=="conditional_on_imputed_source_defect").sum())}条依赖缺失源值插补条件，{int((r1.status=="model_feasible_nontrivial").sum())}条具有四项附件源字段观测。后者仍不等于原浓度、全部阈值或真实缺陷已认证。')
P(f'半径1时，T源样本的 {len(ht)} 条重复划分记录对应 {ht.beer.nunique()} 款不同酒，只有 {success} 条同时满足 J 的 Tripel 概率改善、目标类别判断及 stale_hops 代理不增加。G/J训练源记录另报，不与T源合并冒充独立结果。这个比例是跨模型代理一致性，不是真实品评成功率；不同半径重复使用同一源酒，也不能累加为独立样本。')
F('transfer_independent.png','图7 搜索可行率与独立代理支持率的差异；分母为重复划分记录而非新增酒款',6.7)
P('与前阶段只追求单个分类器高分相比，本阶段增加了最小改动、双代理、联合支持和J评价。这些更严格条件带来更低的通过率，不能与旧稿11/11直接当同定义成绩比较。若某候选利用模型弱点，其失败本身是重要结果。自适应优化后的候选也不能继承普通逐点保形区间的无条件覆盖解释。')

H('7 结果评价、贡献与限制')
P('预测对照揭示目标范围和维度异质性，收缩融合的收益必须与实际误差及复杂度一起判断；网络消融没有支持额外预测优势，因此不把网络图形复杂性当作贡献。独立迁移诊断揭示搜索成绩与跨模型支持之间的明显差距。主要贡献是形成可核查、不过度宣称的任务化证据，而不是创造新的基础算法。')
P(f'当前 {tests} 项程序检查通过，涵盖主键、原件、训练内预处理、嵌套预测角色、图零惩罚等价、风格置换、固定核MMD、生成范围、G/J/T隔离、缺失源标记、失败迁移与校准角色。检查验证实现性质，不能证明全部科学假设成立。配置、源程序与原件哈希、划分和结果均保存。')
P('剩余限制包括：缺少真实绝对评分锚点；原作者预处理和检测限未完全恢复；大部分化学机理尚无目标特异性文献或加标验证；稀有风格留出过少；生成的理化量未形成完整酿造方程；独立评价仍是统计代理且无新外部酒或真实品评。拟议0.5/1/2倍阈值敏感性、所有聚合公式恢复和新增真实锚点并未完成，不能写成已验证。')

H('8 结论与建议')
P(f'在当前附件连续尺度及明确核心范围下，ET 的聚合留出 MAE 为 {etcore:.5f}，收缩融合为 {blendcore:.5f}。相对分级与区间集合给出可复核画像，但不能恢复真实原始0-6分。图约束未提供实用预测增益，因素解释应结合风格内贡献、模块替代和化学证据。')
P(f'受约束局部生成实现了无精确复制和已检范围可行，但分布拟合没有全面超越简单重采样。半径1的T源迁移记录仅 {success}/{len(ht)} 获得独立代理条件共同支持，尚无完整绝对化学或真实品评认证。后续优先获取评分锚点和明确检测限/阈值，补充目标特异性化学证据，再以新酒或盲评确认效果；不建议仅继续增加模型名称。')

ai_number=C('ai')
H('参考文献')
for i,key in enumerate(order,1): P(f'[{i}] '+references[key])
H('AI工具使用与研究贡献声明')
P(f'本参考研究由 OpenAI Codex 辅助资料核对、代码实现、实验运行、图表与稿件编写{ai_number}。所述实验确已运行，但尚无证据表明队员已独立完成或复核核心工作，不能据此作相反声明。精确公开模型版本、队员实际贡献及最终采纳修改需据实补充；随附阶段版说明保留重要提示词、关键回复和实际执行范围。')
H('附录A 完整Top-3、复现与结果索引')
P('以下为全部50个目标的描述性Top-3。顺序按全数据描述模型系数绝对值，括号为固定全数据所选参数条件下的bootstrap入选频率；不等于致因概率。逐折选择与贡献、化学证据层级、全部连续预测和250×50相对等级另存，原始赛题数据不随公开支撑包分发。')
rows=[]
for target,a in top.groupby('target',sort=False): rows.append([target,': '.join(f'{r.field}({r.bootstrap_frequency_conditional_on_full_selected_params:.2f})' for r in a.sort_values('rank').itertuples())])
T(['目标','Top-3 字段及条件入选频率'],rows,[.25,.75])
P('复现顺序：使用官方A附件并核对哈希；安装 requirements-lock.txt；运行 run_stage3.py q1、q2、q3；运行 pytest；然后生成图表与本稿。全部角色和参数位于 outputs/stage3/，执行版配置为 config_stage3.json。软件为Python 3.12、NumPy、pandas、SciPy、scikit-learn、NetworkX、Matplotlib、openpyxl；精确运行版本见各问run_manifest。文档另用ReportLab、pypdf和Pillow。')
```

```
T(['材料','内容与相对路径'],[['评分核查','outputs/stage3_design/scoring_field_map.json 等，只读核查'],['第一问','outputs/stage3/q1：分范围成绩、嵌套权重、相对评分、区间、缺失与截尾'],['第二问','outputs/stage3/q2：图消融、跨拟合Top-3、模块与风格诊断'],['第三问','outputs/stage3/q3：G/J/T角色、候选锁、生成谱、画像、迁移及未知项'],['验证与来源','outputs/stage3/validation_summary.json；docs/SOURCES.md'],['历史记录','stage1、stage2及旧稿保留，不混作同定义新成绩']],[.24,.76])
blocks.append({'type':'pagebreak'})
H('附录B 本项目完整源程序')
P('以下列出本项目自编计算、核查、图表、排版、打包和测试源程序，以及执行配置。外部依赖库不逐行重印，名称版本见锁定文件和环境记录。长行会为版面自动折行；极少数字体不支持字符以Unicode转义显示。可执行原文件随支撑包保留，文件哈希列于附录索引。')
paths=[p.relative_to(ROOT).as_posix() for folder in ['src','scripts','tests'] for p in sorted((ROOT/folder).glob('*.py'))]+[p.name for p in sorted(ROOT.glob('run*.py'))]+[p.name for p in sorted(ROOT.glob('config*.json'))]
blocks.append({'type':'source_files','paths':paths})
title='尺度核查与独立评价下的啤酒感官预测及约束风格迁移'
abstract=f'针对啤酒感官评分、关键因素与风格生成问题，构建尺度审计、分范围预测和独立代理评价流程。以250款酒的231个输入和50个目标建模，明确30个Aroma/Flavour核心维度并保留全表。三折五次重复留出中，ET核心宏平均MAE为{etcore:.4f}，训练内收缩融合为{blendcore:.4f}：五次独立角色分割的平均逐维连续覆盖为{coverage.continuous_coverage.mean():.2%}，但相对等级不等于原始0-6分。加入无图及打乱图对照后，真实图与无图误差几乎相同，全数据内层选择图惩罚为0，不支持额外预测收益。150条因素候选中{supported}条满足当前描述条件，化学证据分级保留。生成阶段去除五个未核实综合量，以100款搜索、100款评价、50款真实留出隔离，比较四类生成方法；非复制优势未转化为全面分布距离优势。半径1下，真实留出源酒的{len(ht)}条重复迁移记录仅{success}条获独立风格及陈旧酒花代理共同支持。研究提供任务化建模与证据边界，不主张新的基础算法、绝对评分恢复或真实品评改善。'
payload={'title':title,'abstract':abstract,'keywords':'感官预测；相对分级；嵌套验证；网络消融；条件生成；独立评价','equation_dir':'paper/revised_20261025/equations','blocks':blocks}
(PAPER/'paper_content.json').write_text(json.dumps(payload,ensure_ascii=False,indent=2),encoding='utf-8')
md=['# '+title,'','## 摘要',abstract,'','关键词：'+payload['keywords'],'']
for b in blocks:
    if b['type']=='heading': md += ['#'*(b['level']+1)+' '+b['text'],'']
    elif b['type']=='paragraph': md += [b['text'],'']
    elif b['type']=='equation': md += [f"公式（{b['number']}）：{b['text']}",'']
    elif b['type']=='table':
        md += ['| '+' | '.join(b['headers'])+' |','|'+'---|'*len(b['headers'])]+['| '+' | '.join(map(str,row))+' |' for row in b['rows']]+['']
    elif b['type']=='figure': md += [f"![{b['caption']}](../../{b['path']})",'']
    elif b['type']=='source_files': md += ['可执行完整源程序已排入PDF附录，并随支撑包提供。文件：'+'、'.join(b['paths']),'']
(PAPER/'论文修订稿.md').write_text('\n'.join(md),encoding='utf-8')

ai_blocks=[]
def AH(s): ai_blocks.append({'type':'heading','level':1,'text':s})
def AP(s): ai_blocks.append({'type':'paragraph','text':s})
AH('1 工具、日期与范围')
AP('工具OpenAI Codex，开发机构OpenAI；系统提示模型家族GPT-6，精确可公开版本待队员以实际界面或配置核验，未臆造。使用日期2026-10-24至2026-10-25。公开阶段记录不含队员身份、签名或私密凭据。')
AP('用途包括公开资料检索、附件结构与尺度审计、参考模型代码、实际实验运行、结果核查、图表、论文和支撑包归档。本记录由AI辅助整理，尚无队员独立复核证据，不能声明核心工作已经由队员独立完成。')
AH('2 重要提示词与关键回复')
interactions=[
 ('用户：你帮我们来做一下A题的三问，至少要吧大致的框架做出来，没疑问的结果要保留，每一问的分析，以及处理过程方法等都要记录。','关键执行内容：先检索公开GitHub与原作者资料，再建立三问可运行基线，保存数据证据、假设分级、探索关联、模型内约束及失败结果。此项为回复/执行摘要，不伪称逐字全文。'),
 ('用户：你帮我完善一下完整的解题过程，需要你推进到写论文的地方，我会把你的解题过程，模型等作为我后续进行解题的一个参考，因为我是第一次参赛。','关键回复节选：已经把 A 题三问推进到完整论文参考稿。每问都记录了问题分析、模型假设、数学公式、求解过程、验证方法、结果与局限。'),
 ('用户：现在你产出的这些东西还缺少什么针对于一篇完整的论文或者说研究。','关键回复节选：目前已经有完整的论文结构和可运行的研究流程，但还缺少足够扎实的证据，把它变成最终参赛论文。'),
 ('用户：你根据现有的这六个缺口，设计相关的实验或者算法来解决他。用户补充：注意同步GitHub。','关键回复节选：目前完成的是设计、算法原型和先导检查，完整第三阶段训练尚未运行。'),
 ('用户：这些算法是创新设计的还是已经有人设计好的，现在这篇文章还有什么问题吗。','关键回复节选：基础算法大多是已有成熟方法；我做的是针对本题的组合、改造和实验设计，目前不能称为已经证实的原创算法。'),
 ('用户：针对你刚才说的这些来改进这篇论文。','关键回复节选：我会先补跑能用现有数据验证的关键实验，重点检查评分范围、网络约束的作用，以及迁移结果能否得到独立评价的支持，再据此修改论文。原始评分锚点等数据缺口会保留条件性结论。')]
for question,reply in interactions: AP(question); AP(reply)
AP('以上保留重要提示词节选和关键回复，未冒充全部对话。正式采纳时应结合原会话补齐队伍实际使用的完整关键交互，尤其是模型选择、修改与人工验证过程；不得将本阶段摘录称为全部聊天记录。')
AH('3 实际完成的辅助工作')
AP(f'已执行三折五次的预测与图对照、五次分割保形与相对等级集合、五次G/J/T生成及迁移独立诊断；已有{tests}项检查通过。核对字段别名和疑似源公式引用，原附件未改。实现采用成熟估计器、正则化组合和标准优化思想，不声明发明全新基础算法。')
AP('实际负结果保留：图约束未见预测收益；生成未全面优于重采样；搜索可行率高于独立代理支持；原始绝对评分、完整浓度阈值、检测限、酿造与真实品评尚未认证。所有数字对应保存结果，而不是事先设计的预期效果。')
AH('4 采纳、人工修改与责任记录')
AP('本阶段代码、计算、图表和参考论文由AI辅助完成。队员独立推导、重跑、化学核对、实际修改和最终采纳尚需真实填写。未发生的验证保持未完成，不填“已人工确认”。')
ai_blocks.append({'type':'table','headers':['内容','AI辅助范围','队员实际验证/修改'],'rows':[['评分与预测','处理、代码、验证与文字','尚未记录'],['网络与化学解释','网络对照、候选与来源核对','尚未记录'],['生成与迁移','约束、搜索、独立代理对照','尚未记录'],['论文与附录','结果整合、图表与排版','尚未记录']],'widths':[.23,.44,.33]})
AH('5 版本、复现与最终提交前补充')
AP('以config_stage3.json及原件/代码哈希、各问run_manifest和结果校验表识别本阶段。正式提交前需核验精确工具版本、队员真实贡献与独立复跑，保留重要原始交互，并按实际采纳在正文、图表和源程序相应位置披露。承诺书、队号命名和签名不由本记录代填。')
ai={'title':'AI工具使用详情说明（阶段版）','front_label':'记录范围','keyword_label':'记录要素','abstract':'本说明记录参考研究中OpenAI Codex的实际辅助范围、重要交互节选、已运行实验和未完成的人工作业。它是可核查的阶段记录，尚未替队员完成精确公开模型版本核验、独立贡献确认和全部关键对话的最终整理。','keywords':'工具信息；重要交互；采纳与修改；实际验证；版本记录','blocks':ai_blocks}
(PAPER/'ai_usage_content.json').write_text(json.dumps(ai,ensure_ascii=False,indent=2),encoding='utf-8')
```

```
print(json.dumps({'blocks':len(blocks),'markdown_characters':len('\n'.join(md)),'source_files':len(paths),'references':len(order),'tests':tests},ensure_ascii=False))
```

源程序 19 scripts/render_equations.py

```
"""Typeset reference-paper equations; AI-assisted, OpenAI Codex, 2026-10-24."""
from pathlib import Path
import matplotlib
matplotlib.use("Agg")
from matplotlib.mathtext import math_to_image

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "paper/equations"
OUT.mkdir(parents=True, exist_ok=True)
EQUATIONS = [
    r"z_{ij} = \frac{\operatorname{clip}(x_{ij};a_j,b_j)-m_j}{s_j}",
    r"z_{it}=\frac{s_{it}-\mu_t}{\sigma_t},\qquad s_{it}=\mu_t+\sigma_t z_{it}",
    r"\widehat{B}=\arg\min_B\left\{\Vert Y_c-ZB\Vert_F^2+\alpha\Vert B\Vert_F^2\right\}",
    r"\operatorname{MAE}=\frac{1}{nq}\sum_{i=1}^{n}\sum_{k=1}^{q}|y_{ik}-\widehat{y}_{ik}|",
    r"r_{ik}=|y_{ik}-f_k(x_i)|,\quad r_k^*=r_{(\lceil(m+1)(1-\alpha)\rceil),k}",
    r"C_k(x)=\left[f_k(x)-r_k^*,\ f_k(x)+r_k^*\right]",
    r"W_{ij}=r_{ij}\,\mathbf{1}(|r_{ij}|\geq\tau),\quad L=\operatorname{diag}\left(\sum_j|W_{ij}|\right)-W",
    r"\widehat{B}=\arg\min_B\left\{\Vert Y_c-ZB\Vert_F^2+\alpha\Vert B\Vert_F^2+\lambda\operatorname{tr}(B^\top LB)\right\}",
    r"\pi_{jk}=\frac{1}{R}\sum_{b=1}^{R}\mathbf{1}\!\left(j\in\operatorname{Top3}_k^{(b)}\right)",
    r"\Delta_{jk}=\operatorname{MAE}_k(\widehat{Y}^{\operatorname{perm}(j)})-\operatorname{MAE}_k(\widehat{Y})",
    r"h\mid c\sim\mathcal{N}\!\left(\mu_c,w_c\Sigma_c+(1-w_c)\Sigma_{\mathrm{pool}}\right),\quad w_c=\frac{n_c}{n_c+10}",
    r"x_{\mathrm{new}}=u x_a+(1-u)x_b,\quad u\sim\operatorname{Uniform}(0.1,0.9),\quad a,b\in c",
    r"x_{\mathrm{candidate},d}\leq x_{\mathrm{source},d},\qquad \forall d\in D",
    r"f_{\mathrm{stale}}(x_{\mathrm{candidate}})\leq f_{\mathrm{stale}}(x_{\mathrm{source}})",
]
for number, expression in enumerate(EQUATIONS, 1):
    math_to_image("$" + expression + "$", OUT / f"eq_{number:02d}.png", dpi=250, format="png", color="black")
print(f"Typeset {len(EQUATIONS)} equations")
```

源程序 20 scripts/revised_equations.py

```
"""Typeset new equations and retain the verified historical equations."""
from pathlib import Path
import shutil
import matplotlib
matplotlib.use('Agg')
from matplotlib.mathtext import math_to_image
ROOT=Path(__file__).resolve().parents[1]; out=ROOT/'paper/revised_20261025/equations';
out.mkdir(parents=True,exist_ok=True)
for p in (ROOT/'paper/equations').glob('*.png'): shutil.copyfile(p,out/p.name)
expressions={15:r'w_j=\operatorname{clip}\!\left(\frac{\operatorname{mean}_i(d_{ij}e_{ij})+\eta w_0}{\operatorname{mean}_i(d_{ij}^2)+\eta},0,1\right)',
16:r'g(y)=\sum_{k=1}^{6}\mathbf{1}(y>t_k),\quad\mathcal{G}(x)=\{g(L_y(x)),\ldots,g(U_y(x))\}',
17:r'\frac{\Vert Z_{\mathrm{candidate}}-Z_{\mathrm{source}}\Vert_2}{\sqrt{p}}\leq r,\quad r\in\{0.25,0.5,1,2\}'}
for number,expression in expressions.items(): math_to_image('$'+expression+'$',out/f'eq_{number:02d}.png',dpi=250,format='png',color='black')
print('Prepared 17 math figures')
```

源程序 21 scripts/stage2_figures.py

```
"""AI-assisted scientific plots: OpenAI Codex / GPT-6, 2026-10-24."""
from pathlib import Path
import json
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "outputs/stage2"
FIG = OUT / "figures"
FIG.mkdir(exist_ok=True)
plt.rcParams.update({"font.size": 10, "axes.spines.top": False, "axes.spines.right": False, "savefig.facecolor": "white"})

def finish(fig, name):
    fig.savefig(FIG / name, dpi=180, bbox_inches="tight")
    plt.close(fig)

```

```
metrics = pd.read_csv(OUT / "q1/metrics_summary.csv").set_index("model")
names = ["mean", "ridge", "pls", "extra_trees", "gbr_fixed", "fixed_ridge_tree_blend"]
fig, ax = plt.subplots(figsize=(9.5, 4), layout="constrained")
ax.bar(np.arange(6), metrics.loc[names, "mae"], color=["#999999", "#3974a6", "#549978", "#bb8439", "#a45f87", "
#728c99"])
ax.set_xticks(range(6), ["Mean", "Ridge", "PLS", "ExtraTrees", "GBR", "Fixed blend"])
ax.set(ylabel="Macro MAE on supplied continuous targets", title="Identical repeated held-out splits")
finish(fig, "model_comparison.png")

g = pd.read_csv(OUT / "q1/grade_rule_sensitivity.csv")
c = pd.read_csv(OUT / "q1/conformal_coverage.csv")
fig, ax = plt.subplots(1, 2, figsize=(10, 4), layout="constrained")
ax[0].bar(np.arange(6) + 1, 100 * g.any_rule_disagreement_fraction, color="#3974a6")
ax[0].set(xlabel="Held-out fold", ylabel="Disagreement across grading assumptions (%)", ylim=(0, 100))
ax[1].hist(c.empirical_test_coverage, bins=np.linspace(.6, 1, 9), color="#549978", edgecolor="white")
ax[1].axvline(.9, ls="--", color="black", label="Nominal marginal coverage")
ax[1].set(xlabel="Empirical coverage (50 test beers)", ylabel="Number of sensory targets")
ax[1].legend(fontsize=8)
finish(fig, "calibration_uncertainty.png")

t = pd.read_csv(OUT / "q2/top3_evidence_review.csv")
good = t.evidence_status == "comparatively_supported_association"
fig, ax = plt.subplots(figsize=(9.5, 4), layout="constrained")
ax.scatter(t.loc[~good, "bootstrap_top3_frequency"], t.loc[~good, "heldout_permutation_MAE_increase"], s=23,
alpha=.65, label="Weak or unstable")
ax.scatter(t.loc[good, "bootstrap_top3_frequency"], t.loc[good, "heldout_permutation_MAE_increase"], s=45,
color="#b26044", label="Passed current descriptive criteria")
ax.axhline(0, color="grey", lw=.8)
ax.axvline(.5, color="grey", lw=.8, ls="--")
ax.set(xlabel="Bootstrap Top-3 frequency", ylabel="Held-out MAE increase after permutation", title="Association
evidence for 150 selected factors")
ax.legend(fontsize=8)
finish(fig, "factor_evidence.png")

d = pd.read_csv(OUT / "q3/generator_comparison_by_split.csv")
fig, ax = plt.subplots(1, 2, figsize=(10, 4), layout="constrained")
styles = ["Tripel", "Kriek", "Lambic"]
methods = ["bootstrap", "local_convex", "PCA_Gaussian"]
for j, method in enumerate(methods):
    group = d[d.method == method].groupby("style").agg(mmd=("MMD2", "mean"), sd=("MMD2", "std"),
prob=("target_style_probability", "mean")).reindex(styles)
    xx = np.arange(3) + (j - 1) * .23
    ax[0].bar(xx, group.mmd, width=.22, yerr=group.sd, capsize=3, label=method)
    ax[1].bar(xx, group.prob, width=.22, label=method)
for a in ax:
    a.set_xticks(range(3), styles)
ax[0].set(ylabel="MMD squared (mean and split SD)", title="Held-out distribution distance")
ax[1].set(ylabel="Target-style model probability", title="Internal classifier diagnostic", ylim=(0, 1.05))
ax[0].legend(fontsize=7)
finish(fig, "generator_comparison.png")

a = pd.read_csv(ROOT / "outputs/stage1/q3/transfer_diagnostics.csv").set_index("beer")
b = pd.read_csv(OUT / "q3/expanded_transfer_diagnostics.csv")
fig, ax = plt.subplots(figsize=(10, 4), layout="constrained")
xx = np.arange(len(b))
ax.plot(xx, b.source_trip_probability, "o--", color="#999999", label="Source")
ax.plot(xx, a.loc[b.beer, "candidate_target_probability"], "s-", color="#3974a6", label="PCA direction search")
ax.plot(xx, b.candidate_trip_probability, "^-", color="#b26044", label="Expanded conditional-anchor search")
for i, row in b.iterrows():
    if not row.all_source_defects_observed:
        ax.annotate("missing source diacetyl", (i, row.candidate_trip_probability), xytext=(i-.8, .72-.06*(i%2)),
fontsize=7, arrowprops={"arrowstyle":"-", "lw":.5})
ax.set_xticks(xx, b.beer.astype(str), rotation=45)
ax.set(xlabel="Source Pils/Lager beer", ylabel="Tripel classifier probability", ylim=(0, 1.09), title="Model-based
transfer; actual sensory response remains unverified")
ax.legend(fontsize=8, loc="center right")
finish(fig, "transfer_comparison.png")

counts = pd.read_csv(OUT / "audit/style_counts.csv").sort_values("n")
fig, ax = plt.subplots(figsize=(8, 6), layout="constrained")
ax.barh(counts.tasting_category_fine, counts.n, color="#3974a6")
ax.set(xlabel="Observed beers", title="Small and unbalanced style samples")
finish(fig, "style_counts.png")
print("Created six stage-two figures")
```

源程序 22 scripts/stage3_figures.py

```
"""Publication figures from actual stage-three outputs; AI-assisted, 2026-10-25."""
from pathlib import Path
import numpy as np
import pandas as pd
```

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

ROOT=Path(__file__).resolve().parents[1]; OUT=ROOT/'outputs/stage3'; FIG=OUT/'figures'; FIG.mkdir(exist_ok=True)
plt.rcParams.update({'font.size':10,'axes.spines.top':False,'axes.spines.right':False,'savefig.dpi':200})
def save(fig,name): fig.savefig(FIG/name,bbox_inches='tight'); plt.close(fig)

metrics=pd.read_csv(OUT/'q1/metrics_by_scope.csv'); names=['mean','ridge','extra_trees','gbr','fixed_half_blend','gl
obal_oof_blend','unshrunk_target_blend','shrunk_target_blend']
labels=['Mean','Ridge','ET','GBR','Half blend','Global blend','Target blend','Shrunk blend']
fig,axes=plt.subplots(1,2,figsize=(11,3.5))
for ax,scope in zip(axes,['core30','all50']):
    a=metrics[metrics.scope==scope].set_index('model').loc[names];
ax.bar(np.arange(len(a)),a.mae,color=['#999999','#779ab2','#3978a8','#b36b73','#8ca98c','#87a7b7','#b79a75','#477e69']);
ax.set_xticks(np.arange(len(a)),labels,rotation=40,ha='right'); ax.set_ylabel('MAE on supplied continuous targets');
ax.set_title('Core Aroma/Flavour: 30 targets' if scope=='core30' else 'All 50 targets')
save(fig,'prediction_by_scope.png')

coverage=pd.read_csv(OUT/'q1/conformal_coverage.csv'); fig,axes=plt.subplots(1,2,figsize=(10,3.2))
axes[0].hist(coverage.continuous_coverage,bins=np.linspace(.5,1,11),color='#3978a8');
axes[0].axvline(.9,color='black',ls='--'); axes[0].set_xlabel('Observed marginal continuous coverage');
axes[0].set_ylabel('Target-split combinations')
axes[1].hist(coverage.mean_grade_set_size,bins=10,color='#477e69'); axes[1].set_xlabel('Mean size of relative
ordinal interval hull'); axes[1].set_ylabel('Target-split combinations'); fig.suptitle('Five train/calibration/test
splits; relative grades are not raw intensity')
save(fig,'ordinal_uncertainty.png')

graph=pd.read_csv(OUT/'q2/graph_comparison.csv'); params=pd.read_csv(OUT/'q2/selected_parameters.csv');
fig,axes=plt.subplots(1,2,figsize=(10,3.3))
a=graph[graph.scope=='core30'].set_index('graph').loc[['none','real','shuffled']]; axes[0].bar(['No graph','Real
graph','Shuffled graph'],a['mean'],color=['#999999','#3978a8','#b36b73']); axes[0].set_ylim(0,.45);
axes[0].set_ylabel('Mean outer-fold core MAE')
for j,v in enumerate(a['mean']): axes[0].text(j,v+.006,f'{v:.6f}',ha='center',fontsize=9)
p=params[params.graph=='real']['lambda'].value_counts().sort_index();
axes[1].bar(p.index.astype(str),p.values,color='#477e69'); axes[1].set_xlabel('Graph penalty selected inside training');
axes[1].set_ylabel('Outer folds (15 total)'); fig.suptitle('Graph regularization ablation; no meaningful predictive gain
observed')
save(fig,'graph_ablation.png')

top=pd.read_csv(OUT/'q2/top3_with_evidence.csv'); fig,ax=plt.subplots(figsize=(7,3.6));
good=top.evidence_status.str.startswith('comparatively'); ax.scatter(top.loc[~good,'bootstrap_frequency_conditional_on_fu
ll_selected_params'],top.loc[~good,'within_style_delta_on_selected_folds'],s=16,color='#84a9c2',label='Weak or
incomplete'); ax.scatter(top.loc[good,'bootstrap_frequency_conditional_on_full_selected_params'],top.loc[good,'within_sty
le_delta_on_selected_folds'],s=24,color='#b85e43',label='Current descriptive criteria');
ax.axhline(0,color='gray',lw=.7); ax.axvline(.5,color='gray',ls='--',lw=.7); ax.set_xlabel('Conditional bootstrap Top-3
frequency'); ax.set_ylabel('Within-style held-out permutation MAE change'); ax.legend(fontsize=8);
save(fig,'factor_crossfit.png')

gen=pd.read_csv(OUT/'q3/generation_comparison.csv'); methods=['bootstrap','local_convex','constrained_local_convex','
PCA_Gaussian']; styles=['Tripel','Kriek','Lambic']; colors=['#999999','#3978a8','#477e69','#b38a58'];
fig,axes=plt.subplots(1,2,figsize=(11,3.5)); width=.19
for k,(method,color) in enumerate(zip(methods,colors)):
    a=gen[gen.method==method].groupby('style').mean(numeric_only=True).loc[styles]; x=np.arange(3)+(k-1.5)*width
    axes[0].bar(x,a.MMD2_fixed_training_bandwidth,width,color=color,label=method);
axes[1].bar(x,a.independent_evaluator_target_prediction_fraction,width,color=color)
for ax in axes: ax.set_xticks(np.arange(3),styles)
axes[0].set_ylabel('MMD squared (shared training bandwidth)'); axes[1].set_ylabel('Independent evaluator predicts
target style'); axes[1].set_ylim(0,1.05); axes[0].legend(fontsize=7); save(fig,'generation_independent.png')

transfer=pd.read_csv(OUT/'q3/transfer_independent_evaluation.csv'); fig,ax=plt.subplots(figsize=(7.5,3.6));
radii=sorted(transfer.radius.unique()); values=[]
for r in radii:
    a=transfer[transfer.radius==r]; t=a[a.source_role=='T_real_holdout'];
values.append([(a.status!='no_feasible_nontrivial_candidate_source_retained').mean(),a.evaluator_proxy_acceptance.mean(),
t.evaluator_proxy_acceptance.mean()])
values=np.array(values)
for j,(label,color) in enumerate([('Search model feasible (55 records)','#3978a8'),('Independent proxy acceptance
(55 records)','#b36b73'),('Real-holdout source proxy acceptance (10 records)','#477e69')]):
ax.plot(radii,values[:,j],marker='o',label=label,color=color)
ax.set_xlabel('Standardized change radius'); ax.set_ylabel('Fraction of records (repeated beers across splits)');
ax.set_ylim(-.03,1.03); ax.legend(fontsize=8,loc='upper left'); save(fig,'transfer_independent.png')

weights=pd.read_csv(OUT/'q1/blend_weights.csv'); fig,ax=plt.subplots(figsize=(7,3));
core=weights.target.str.startswith(('A_','F_')); ax.hist([weights.loc[core,'weight_GBR'],weights.loc[~core,'weight_GBR']],
bins=np.linspace(0,1,11),label=['Core 30','Additional 20'],color=['#3978a8','#b38a58'],alpha=.8); ax.set_xlabel('GBR
weight learned inside outer training only'); ax.set_ylabel('Target-fold combinations'); ax.legend();
save(fig,'blend_weights.png')
print('Saved seven stage-three figures')
```

源程序 23 scripts/stage3_preflight.py

```
"""Read-only source audit and conditional analysis of existing OOF predictions.

Run from the bundled document runtime (openpyxl + numpy). No workbook is edited.
This is a protocol preflight, not the full stage-three experiment runner.
"""
from pathlib import Path
import argparse
import csv
import hashlib
import json
import re
import sys
import xml.etree.ElementTree as ET
import numpy as np
from openpyxl import load_workbook

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from src.stage3_algorithms import paired_oof_bootstrap

parser = argparse.ArgumentParser()
parser.add_argument("--data-dir", type=Path, default=ROOT.parent / "materials/A/附件")
parser.add_argument("--out", type=Path, default=ROOT / "outputs/stage3_design")
args = parser.parse_args()
DATA, OUT = args.data_dir, args.out
OUT.mkdir(parents=True, exist_ok=True)


def read_csv(path):
    with path.open(encoding="utf-8-sig", newline="") as stream:
        return list(csv.DictReader(stream))


def write(name, value):
    (OUT / name).write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")


original_hashes = {name: hashlib.sha256((DATA / name).read_bytes()).hexdigest() for name in ["1.csv", "3.xlsx", "
4.csv", "8.xlsx"]}
previous_hashes = {r['file']:r['sha256'] for r in read_csv(ROOT/'outputs/stage2/audit/source_manifest.csv')}
if any(previous_hashes.get(name) != sha for name,sha in original_hashes.items()):
    raise ValueError("Source version differs from the existing stage-two predictions/evidence")
target_rows = read_csv(DATA / "4.csv")
targets = [c for c in target_rows[0] if c not in ["beer", "beer_id", "tasting_category_fine"]]
aliases = {"A_ester_fruity": "A_esters_fruity", "F_ester_fruity": "F_esters_fruity", "4vg": "X4vg"}
workbook = load_workbook(DATA / "3.xlsx", data_only=False, read_only=True)
summary = workbook["summary"]
mapping, anomalies = [], []
for col in range(2, summary.max_column+1):
    original = summary.cell(1,col).value
    target = aliases.get(original,original)
    if target not in targets:
        continue
    formula = str(summary.cell(2,col).value)
    match = re.fullmatch(r"=beer_1!\$([A-Z]+)\$(\d+)",formula)
    if not match:
        raise ValueError(f"Unexpected score formula for {target}: {formula}")
    column, row = match.group(1), int(match.group(2))
    expected_cell = "AG90" if target == "barnyard" else f"{column}{row}"
    section = ("Aroma_core" if target.startswith("A_") else "Flavour_core" if target.startswith("F_") else
               "Faults" if target in ["dms","metallic","X4vg","t2n","diacetyl","stale_hops"] else
               "Spiced_beers" if target in ["orange","coriander","clove"] else
               "Lambic_Geuze" if target in ["lactic","acetic","barnyard"] else
               "Overall" if target == "overall" else "Palate_or_basic_taste")
    mapping.append({"target":target,"summary_header":original,"summary_column":col,"original_formula":formula,
                    "expected_score_cell":expected_cell,"section":section,
                    "formula_status":"suspected_wrong_reference_derived_map_corrects_only" if target=='barnyard'
else "consistent_with_form_position",
                    "official_submission_scope":"requires_explicit_reporting_core_30_and_full_50_not_inferred_from_c
ount"})
    if target == "barnyard":
        assert "Barnyard" in str(workbook["beer_1"]["R89"].value)
        assert str(workbook["beer_1"]["AG90"].value).startswith("=")
        for i in range(2,14):
            actual=str(summary.cell(i,col).value)
            expected=f"=beer_{i-1}!$AG$90"
            if actual != expected:
                anomalies.append({"summary_cell":summary.cell(i,col).coordinate,"target":target,"actual_formula":act
ual,"form_position_expected_formula":expected})
assert len(mapping) == len(targets) == 50 and {r['target'] for r in mapping} == set(targets)
```

```
write("scoring_field_map.json",mapping)
write("scoring_scope_audit.json",{"mapped_targets":50,"core_Aroma":15,"core_Flavour":15,"additional_targets":20,
    "aliases":aliases,"suspected_formula_mismatches":anomalies,
    "card_items_without_separate_supervised_targets":[{"card_label":"Acetaldehyde under
Aroma","score_cell":"Q45"},{"card_label":"Acetaldehyde under Flavour","score_cell":"AG45"}],
    "original_files_edited":False,"absolute_score_anchors_available":False,
    "scope_caveat":"30 is the prefixed core comparison scope, not a ruling that only 30 answers are required."})

scales = {r['field']:r for r in read_csv(ROOT / "outputs/stage2/audit/field_scales.csv")}
chem = {r['field']:r for r in read_csv(ROOT / "outputs/stage2/audit/chemical_dictionary.csv")}
registry=[]
for field in ["DMS","X2.3.butanedione","X4VG","X.E..2.nonenal"]:
    scale,meta=scales[field],chem[field]
    try: threshold=float(meta['threshold_median'])
    except (TypeError,ValueError): threshold=None
    if threshold is not None and not np.isfinite(threshold): threshold=None
    supported=scale['transform_evidence']=='verified_log10_positive_pairs'
    registry.append({"field":field,"unit_in_attachment":meta['unit'],"threshold_median_in_attachment":threshold,
        "transformation_evidence":scale['transform_evidence'],"positive_pair_log10_supported":supported,
        "concentration_rule_if_eligible":"10**x" if supported else "unresolved_per_sample_check_needed",
        "zero_LOD_or_substitution_provenance":"not_fully_verified",
        "absolute_defect_certification":"not_complete_even_if_positive_pairs_match",
        "next_action":"audit_per_sample_calibrated_pairs_and_detection_limits" if supported else "
resolve_mismatching_pairs_and_scale_before_concentration_claim"})
write("defect_registry_preflight.json",registry)

beer_ids=[int(r['beer']) for r in target_rows]
truth=np.array([[float(r[t]) for t in targets] for r in target_rows])
def predictions(path):
    rows=read_csv(path)
    bybeer={int(r['beer']):r for r in rows}
    if len(bybeer)!=len(rows) or set(bybeer)!=set(beer_ids): raise ValueError("OOF keys do not align uniquely")
    return np.array([[float(bybeer[b][t]) for t in targets] for b in beer_ids])
gbr=predictions(ROOT/'outputs/stage2/q1/oof_gbr_continuous.csv')
et=predictions(ROOT/'outputs/stage1/q1/oof_extra_trees_continuous.csv')
labels=[r['tasting_category_fine'] for r in target_rows]
reports=[]
for scope,indices in [('all_50',list(range(50))),('prefixed_core_30',[j for j,t in enumerate(targets) if
t.startswith(('A_','F_'))])]:
    result=paired_oof_bootstrap(truth[:,indices],gbr[:,indices],et[:,indices],labels)
    result.update(scope=scope,comparison='fixed_stage2_GBR_minus_stage1_ExtraTrees',metric='macro_MAE',
                  uncertainty_scope='beer-resampling_conditional_on_existing_predictions_not_fresh_external_validati
on')
    reports.append(result)
write('existing_oof_paired_pilot.json',reports)

test_report=ROOT/'tmp/stage3_tests.xml'
suite=ET.parse(test_report).getroot().find('testsuite')
assert all(int(suite.attrib[k])==0 for k in ['failures','errors','skipped'])
for name,old in original_hashes.items():
    assert hashlib.sha256((DATA/name).read_bytes()).hexdigest()==old
manifest={'date':'2026-10-25','stage':'design_and_preflight','complete_stage3_training_run':False,
   'source_sha256':original_hashes,'protocol_sha256':hashlib.sha256((ROOT/'config_stage3_design.json').read_bytes())
.hexdigest(),
    'algorithm_tests':{k:int(suite.attrib[k]) for k in ['tests','failures','errors','skipped']},
    'no_human_validation_claimed':True,'no_original_workbook_edited':True,
    'full_experiments_status':'planned_not_executed; only source mapping and conditional existing-OOF pilot
executed'}
write('preflight_manifest.json',manifest)
print(json.dumps({'mapped_targets':len(mapping),'formula_mismatches':len(anomalies),'paired_pilot':reports,'test_sum
mary':manifest['algorithm_tests']},ensure_ascii=False))
```

源程序 24 scripts/verify_reference.py

```
"""Validate the actual paper and saved evidence, without publishing host identifiers."""
from pathlib import Path
import csv
import hashlib
import json
import xml.etree.ElementTree as ET
from pypdf import PdfReader

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "outputs/stage2"
reader = PdfReader(ROOT / "paper/论文参考稿.pdf")
texts = [page.extract_text() for page in reader.pages]
assert len(reader.pages) == 13
assert "摘 要" in texts[0] and "目 录" in texts[1]
assert "1 问题重述与研究范围" in texts[2]
assert all("\x00" not in text for text in texts)
assert reader.metadata.get("/Author", "") == ""
```

```
assert all("github.com" not in text and "@qq.com" not in text for text in texts)
assert all(abs(float(p.mediabox.width) - 595.2756) < 0.1 for p in reader.pages)
content = json.loads((ROOT / "paper/paper_content.json").read_text(encoding="utf-8"))
for block in content["blocks"]:
    if block["type"] == "heading" and block["level"] == 1:
        title = block["text"]
        expected = next(i-1 for i, page in enumerate(texts[2:], 2) if title in page)
        destination = next(item for item in reader.outline if not isinstance(item, list) and item.title == title)
        assert reader.get_destination_page_number(destination)-1 == expected
with (OUT / "q1/metrics_summary.csv").open(encoding="utf-8-sig", newline="") as f:
    metrics = list(csv.DictReader(f))
gbr = next(row for row in metrics if row["model"] == "gbr_fixed")
assert abs(float(gbr["mae"]) - 0.339759) < 0.000001
suite = ET.parse(OUT / "tests.xml").getroot().find("testsuite")
assert int(suite.attrib["tests"]) == 23
assert all(int(suite.attrib[k]) == 0 for k in ["errors", "failures", "skipped"])
summary = {
    "date": "2026-10-24",
    "tests": {k: int(suite.attrib[k]) for k in ["tests", "errors", "failures", "skipped"]},
    "test_names": [item.attrib["name"] for item in suite.findall("testcase")],
    "paper": {"physical_pages": len(reader.pages), "abstract_page": 1, "contents_page": 2,
              "body_starts_physical_page": 3, "body_starts_printed_page": 1,
              "equations": sum(b["type"] == "equation" for b in content["blocks"]),
              "figures": sum(b["type"] == "figure" for b in content["blocks"]),
              "missing_glyph_nulls": 0, "author_metadata_empty": True,
              "visual_review": "all 13 rendered pages inspected; final amendment on defect names reviewed",
              "sha256": hashlib.sha256((ROOT / "paper/论文参考稿.pdf").read_bytes()).hexdigest()},
    "scientific_scope": "Exploratory AI-assisted reference; no absolute-score recovery, causal or tasting
validation claimed."
}
(OUT / "validation_summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
files = [p for p in OUT.rglob("*") if p.is_file() and p.name not in ["tests.xml", "result_checksums.json"] and "
private" not in p.parts]
checksums = [{"file": p.relative_to(OUT).as_posix(), "sha256": hashlib.sha256(p.read_bytes()).hexdigest()} for p in
sorted(files)]
(OUT / "result_checksums.json").write_text(json.dumps(checksums, ensure_ascii=False, indent=2), encoding="utf-8")
print(json.dumps({"pages": len(reader.pages), "tests_passed": 23, "hashed_results": len(checksums)},
ensure_ascii=False))
```

源程序 25 scripts/verify_revised_paper.py

```
"""Check actual revised PDFs, anonymous metadata, pagination and source appendix."""
from pathlib import Path
import hashlib
import json
import re
from pypdf import PdfReader
import pdfplumber
ROOT=Path(__file__).resolve().parents[1]; PAPER=ROOT/'paper/revised_20261025'
reports=[]
for name in ['论文修订稿.pdf','AI工具使用详情说明_阶段版.pdf']:
    path=PAPER/name; reader=PdfReader(path); texts=[p.extract_text() for p in reader.pages]
    assert reader.metadata.get('/Author','')==''  and '目 录' in texts[1]
    assert texts[2].lstrip().startswith('1\n1 ')
    assert all('\x00' not in t for t in texts)
    assert not any(re.search(r'[A-Za-z0-9_.+-]+@qq\.com',t) for t in texts)
    assert all(abs(float(p.mediabox.width)-595.2756)<.1 for p in reader.pages)
    headings=[p for p in reader.outline if not isinstance(p,list)]
    appendix=next((reader.get_destination_page_number(p)+1 for p in headings if p.title.startswith('附录A')),None)
    overflow=[]
    with pdfplumber.open(path) as pdf:
        for i,page in enumerate(pdf.pages):
            # CJK hanging punctuation includes a wide side-bearing in its advance box.
            bad=[c for c in page.chars if c.get('text','').strip() and (c['x0'] < 67 or c['x1'] >
float(page.width)-(60 if c.get('text') in '。，、；： ' else 67) or c['top']<43 or c['bottom']>float(page.height)-25)]
            if bad: overflow.append({'page':i+1,'count':len(bad),'sample':''.join(c['text'] for c in bad[:60])})
    assert not overflow,overflow
    reports.append({'file':name,'physical_pages':len(reader.pages),'body_starts_page':3,'appendix_A_starts_physical_
page':appendix,'missing_glyph_nulls':0,'text_overflow_pages':0,
                    'sha256':hashlib.sha256(path.read_bytes()).hexdigest()})
content=json.loads((PAPER/'paper_content.json').read_text(encoding='utf-8')); numbers=[b['number'] for b in
content['blocks'] if b['type']=='equation']
assert numbers==list(range(1,len(numbers)+1))
manifest=json.loads((PAPER/'source_appendix_manifest.json').read_text(encoding='utf-8'))
paths=next(b['paths'] for b in content['blocks'] if b['type']=='source_files')
assert {p['file'] for p in manifest}==set(paths)
for row in manifest: assert hashlib.sha256((ROOT/row['file']).read_bytes()).hexdigest()==row['sha256']
report={'client_date':'2026-10-25','documents':reports,'numbered_equations':len(numbers),'source_files':len(manifest)
,'source_lines':sum(r['lines'] for r in manifest),
        'source_appendix_matches_current_files':True,'visual_review':'rendered_pages_must_be_inspected_before_delive
ry'}
```

```
(PAPER/'pdf_validation.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps(report,ensure_ascii=False))
```

源程序 26 scripts/verify_stage3.py

```
"""Cross-check saved scientific artifacts and record exact executed scope."""
from pathlib import Path
import hashlib
import json
import xml.etree.ElementTree as ET
import numpy as np
import pandas as pd
ROOT=Path(__file__).resolve().parents[1]; OUT=ROOT/'outputs/stage3'
suite=ET.parse(ROOT/'tmp/stage3_full_tests.xml').getroot().find('testsuite')
tests={k:int(suite.attrib[k]) for k in ['tests','failures','errors','skipped']}
assert tests['tests']>=40 and tests['failures']==tests['errors']==tests['skipped']==0
status={part:json.loads((OUT/part/'status.json').read_text()) for part in ['q1','q2','q3']}
assert all(a['complete'] for a in status.values())
roles=pd.read_csv(OUT/'q1/outer_roles.csv')
assert len(roles)==1250 and roles['repeat'].nunique()==5
assert (roles.groupby(['repeat','beer']).size()==1).all()
grades=pd.read_csv(OUT/'q1/ASSUMPTION_oof_relative_grades.csv'); a=grades.drop(columns='beer').to_numpy()
assert a.shape==(250,50) and np.isfinite(a).all() and np.all((a>=0)&(a<=6)&(a==a.astype(int)))
weights=pd.read_csv(OUT/'q1/blend_weights.csv'); assert len(weights)==750 and weights.weight_GBR.between(0,1).all()
metadata=pd.read_csv(ROOT/'outputs/stage2/audit/source_manifest.csv')
from sys import path
path.insert(0,str(ROOT))
from src.data import find_data,load_data
original=find_data(); data=load_data(original)
for row in metadata.itertuples(): assert hashlib.sha256((original/row.file).read_bytes()).hexdigest()==row.sha256
scope=pd.read_csv(OUT/'q1/metrics_by_scope.csv'); core=[c for c in data.Y if c.startswith(('A_','F_'))]
for model in scope.model.unique():
    pred=pd.read_csv(OUT/f'q1/oof_{model}.csv').set_index('beer').loc[data.meta.beer]
    for name,fields in [('core30',core),('all50',list(data.Y))]:
        expected=float(np.abs(data.Y[fields].to_numpy()-pred[fields].to_numpy()).mean())
        actual=float(scope[(scope.model==model)&(scope.scope==name)].mae.iloc[0]); assert abs(expected-actual)<2e-8
graph=pd.read_csv(OUT/'q2/graph_fold_metrics.csv'); assert graph.graph.notna().all() and
len(graph[graph.graph=='shuffled'])==300
gen=pd.read_csv(OUT/'q3/generation_comparison.csv'); assert len(gen)==60 and (gen.n_generated==40).all()
assert (gen.provided_bounds_violation_fraction==0).all()
transfer=pd.read_csv(OUT/'q3/transfer_independent_evaluation.csv'); assert len(transfer)==220
assert not transfer.absolute_chemical_certification.any() and not transfer.real_tasting_validated.any()
defects=pd.read_csv(OUT/'q3/transfer_defect_checks.csv'); assert defects.provided_nonincrease.all()
top=pd.read_csv(OUT/'q2/top3_with_evidence.csv'); assert len(top)==150 and (top.groupby('target').size()==3).all()
literature=[]
for r in top.itertuples():
    row={'target':r.target,'field':r.field,'statistical_status':r.evidence_status,'causal_status':'not_identified','
independent_reference':'','chemical_evidence':'attachment_metadata_only_target_specific_mechanism_unverified','limitation
s':'direction_and_style_confounding_require_review'}
    if r.target=='X4vg' and r.field=='X4VG': row.update(independent_reference='https://pubmed.ncbi.nlm.nih.gov/19053
838/',chemical_evidence='beer_research_supports_clove_related_aroma_not_current_dataset_causality',limitations='aging_and
_matrix_conditions_not_a_new_spiking_test')
    elif r.target=='metallic' and r.field=='iron': row.update(independent_reference='https://pubmed.ncbi.nlm.nih.gov
/22304665/',chemical_evidence='metal_solution_and_salivary_lipid_oxidation_mechanism',limitations='iron_speciation_and_be
er_matrix_not_resolved_no_beer_threshold_inferred')
    elif r.target=='coriander' and r.field=='linalool': row.update(independent_reference='https://pubmed.ncbi.nlm.ni
h.gov/38398596/',chemical_evidence='coriander_seed_low_alcohol_beer_primary_research_context',limitations='not_specific_c
ausality_for_this_panel_or_this_concentration')
    elif (r.target=='acidity' and r.field in ['lactic_acid.mg.L.','acetic_acid.g.L.']) or (r.target=='lactic' and
r.field=='lactic_acid.mg.L.') or (r.target=='alcohol' and r.field=='ethanol..v.v.'):
        row.update(chemical_evidence='measurement_compatible_relation_not_independent_causal_validation',limitations
='matrix_mixture_and_source_preprocessing_remain')
    literature.append(row)
pd.DataFrame(literature).to_csv(OUT/'q2/chemical_evidence_ledger.csv',index=False,encoding='utf-8-sig')
summary={'client_date':'2026-10-25','tests':tests,'original_sources_unchanged':True,'prediction_outer_folds':15,'uni
que_beers':250,
 'graph_null_fits_per_outer':10,'generation_disjoint_partitions':5,'generated_records_per_group':40,
 'core_targets':30,'all_targets':50,'relative_grade_cells':12500,
 'complete_absolute_score_calibration':False,'complete_chemical_certification':False,'real_tasting_validated':False,
 'scientific_scope':'exploratory_internal_validation_after_earlier_stages_not_new_external_validation',
 'executed':['nested_prediction_and_shrinkage','fixed_relative_grade_sets','missingness_and_clipping','train_only_gr
aph_ablations','crossfit_within_style_module_effects','disjoint_generation_and_Pareto_transfer','failure_and_unknown_item
_preservation'],
 'still_unresolved':['real_absolute_score_anchors','detection_limits_and_all_units','all_aggregate_formulas','full_t
arget_specific_chemical_mechanisms','new_external_beers_and_tasting','human_independent_review'],
 'source_hashes':[{ 'file':r.file,'sha256':r.sha256} for r in metadata.itertuples()],
 'code_sha256':{p.relative_to(ROOT).as_posix():hashlib.sha256(p.read_bytes()).hexdigest() for p in
sorted((ROOT/'src').glob('stage3*.py'))}}
(OUT/'validation_summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
checks=[{'file':p.relative_to(OUT).as_posix(),'sha256':hashlib.sha256(p.read_bytes()).hexdigest()} for p in
sorted(OUT.rglob('*')) if p.is_file() and p.name!='result_checksums.json' and 'private' not in p.parts]
```

```
(OUT/'result_checksums.json').write_text(json.dumps(checks,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps({'tests':tests,'files_hashed':len(checks),'relative_grade_cells':12500,'all_three_experiments_compl
ete':True},ensure_ascii=False))
```

源程序 27 tests/test_invariants.py

```
"""AI-assisted verification: OpenAI Codex / GPT-6, 2026-10-24."""
import json
from pathlib import Path

import numpy as np
import pandas as pd
import pytest
from sklearn.model_selection import RepeatedStratifiedKFold

from src.common import TrainQuantileClipper, ecdf_grades, sha256
from src.data import CHEMICAL_DEFECTS, NON_MOLECULES, align_by_beer, find_data, load_data
from src.q2 import signed_laplacian
from src.q3 import ConditionalPCAGaussian, chemical_nonincrease


ROOT = Path(__file__).resolve().parents[1]
OUTPUT = ROOT / "outputs/stage1"


def test_join_uses_unique_beer_and_recovers_permuted_rows():
    x = pd.DataFrame({"beer": [1, 2], "beer_id": [100, 101], "tasting_category_fine": ["a", "b"]})
    y = pd.DataFrame({"beer": [2, 1], "beer_id": [100, 100], "tasting_category_fine": ["b", "a"], "score": [9, 4]})
    assert align_by_beer(x, y).score.tolist() == [4, 9]


def test_duplicate_key_is_rejected_before_join():
    a = pd.DataFrame({"beer": [1, 1], "tasting_category_fine": ["a", "a"]})
    with pytest.raises(ValueError, match="unique"):
        align_by_beer(a, a)


def test_preprocessing_bounds_do_not_learn_from_validation_extreme():
    train = np.array([[0.], [1.], [2.], [3.]])
    clip = TrainQuantileClipper(lower=0, upper=1).fit(train)
    assert clip.transform([[1e12]])[0, 0] == 3
    np.testing.assert_array_equal(clip.upper_, [3.])


def test_signed_laplacian_is_psd_with_negative_edges():
    L = signed_laplacian([[1, -.8, .7], [-.8, 1, .1], [.7, .1, 1]], .6)
    assert np.linalg.eigvalsh(L).min() > -1e-10


def test_ecdf_grades_are_bounded_and_monotone():
    ref = np.arange(100.)[:, None]
    g = ecdf_grades(ref, np.array([[-1.], [50.], [1000.]]))[:, 0]
    assert g[0] == 0 and g[-1] == 6 and np.all(np.diff(g) >= 0)


def test_each_defect_is_checked_separately():
    # A falling total must not hide a rise in one defect.
    source, candidate = np.array([2., 2.]), np.array([0., 3.])
    assert candidate.sum() < source.sum()
    assert not chemical_nonincrease(source, candidate, [0, 1])


def test_conditional_generator_is_finite_bounded_and_reproducible():
    rng = np.random.default_rng(4)
    x = rng.normal(size=(30, 7))
    x[0, 0] = np.nan
    gen = ConditionalPCAGaussian(3).fit(x, np.array(["a"] * 15 + ["b"] * 15))
    a = gen.sample("a", 10, np.random.default_rng(1))
    b = gen.sample("a", 10, np.random.default_rng(1))
    np.testing.assert_allclose(a, b)
    assert np.isfinite(a).all()
    assert np.all(a >= gen.low - 1e-10) and np.all(a <= gen.high + 1e-10)


@pytest.fixture(scope="module")
def real_data():
    try:
        root = find_data()
    except FileNotFoundError:
        pytest.skip("Original competition data not available")
    return load_data(root)
```

```


def need_results():
    if not (OUTPUT / "q3/status.json").exists():
        pytest.skip("Run all stages before artifact validation")


def test_original_sources_unchanged(real_data):
    need_results()
    manifest = pd.read_csv(OUTPUT / "audit/source_manifest.csv")
    for row in manifest.itertuples():
        assert sha256(real_data.root / row.file) == row.sha256


def test_oof_mean_matches_actual_training_only_mean(real_data):
    need_results()
    cfg = json.loads((ROOT / "config.json").read_text())
    Y = real_data.Y.to_numpy()
    expected = np.zeros_like(Y)
    counts = np.zeros(len(Y))
    splitter = RepeatedStratifiedKFold(n_splits=cfg["outer_folds"], n_repeats=cfg["outer_repeats"],
random_state=cfg["seed"])
    for train, test in splitter.split(real_data.X, real_data.meta.tasting_category_fine):
        expected[test] += Y[train].mean(axis=0)
        counts[test] += 1
    saved = pd.read_csv(OUTPUT / "q1/oof_mean_continuous.csv")[real_data.Y.columns].to_numpy()
    np.testing.assert_allclose(saved, expected / counts[:, None], atol=1e-7)


def test_each_sample_held_out_once_per_repeat(real_data):
    need_results()
    s = pd.read_csv(OUTPUT / "q1/split_assignments.csv")
    assert not s.duplicated(["repeat", "beer"]).any()
    for _, part in s.groupby("repeat"):
        assert set(part.beer) == set(real_data.meta.beer)


def test_top3_has_unique_eligible_factors_per_target(real_data):
    need_results()
    top = pd.read_csv(OUTPUT / "q2/top3_by_sensory_dimension.csv")
    assert len(top) == 3 * real_data.Y.shape[1]
    assert set(top.target) == set(real_data.Y.columns)
    assert not top.duplicated(["target", "field"]).any()
    assert not set(top.field) & NON_MOLECULES
    assert top.bootstrap_top3_frequency.between(0, 1).all()


def test_transfer_failure_is_not_hidden_as_success():
    need_results()
    t = pd.read_csv(OUTPUT / "q3/transfer_diagnostics.csv")
    failed = t[t.status == "no_nontrivial_candidate_found_source_retained"]
    assert np.allclose(failed.supplied_scale_change_l2, 0)
    success = t[t.status == "nontrivial_model_feasible_candidate"]
    assert (success.candidate_target_probability > success.source_target_probability + .01).all()
    assert (success.candidate_target_probability > success.cleaning_only_target_probability + .01).all()
    assert t.chemical_nonincrease.all() and t.predicted_stale_hops_nonincrease.all()
    checks = pd.read_csv(OUTPUT / "q3/defect_nonincrease_checks.csv")
    assert checks.nonincrease.all()
    assert set(checks.defect) == set(CHEMICAL_DEFECTS) | {"stale_hops"}


def test_calibration_remains_explicitly_unresolved():
    need_results()
    status = json.loads((OUTPUT / "q1/status.json").read_text())
    assert status["absolute_0_6_prediction"].startswith("UNRESOLVED")
    grades = pd.read_csv(OUTPUT / "q1/ASSUMPTION_percentile_grades_0_6.csv")
    assert grades.status.str.startswith("ASSUMPTION_ONLY").all()


def test_generator_holdout_does_not_overlap_training(real_data):
    need_results()
    split = pd.read_csv(OUTPUT / "q3/generator_split.csv")
    train = set(split[split.role == "train"].beer)
    heldout = set(split[split.role == "holdout"].beer)
    assert train.isdisjoint(heldout)
    assert train | heldout == set(real_data.meta.beer)


def test_generation_diagnostics_do_not_hide_missing_reference_values():
    need_results()
    diagnostics = pd.read_csv(OUTPUT / "q3/generation_diagnostics.csv")
    assert np.isfinite(diagnostics.mean_marginal_KS_holdout).all()
```

```
    assert diagnostics.mean_marginal_KS_holdout.between(0, 1).all()
    assert (diagnostics.n_KS_fields_observed > 0).all()
```

源程序 28 tests/test_stage2.py

```
"""AI-assisted validation: OpenAI Codex / GPT-6, 2026-10-24."""
import json
from pathlib import Path

import numpy as np
import pandas as pd
import pytest

from src.stage2 import assumption_grades, conformal_quantile, local_convex_samples

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "outputs/stage2"


def test_conformal_quantile_uses_finite_sample_rank():
    errors = np.arange(1, 51)[:, None]
    assert conformal_quantile(errors, .1)[0] == 46
    with pytest.raises(ValueError):
        conformal_quantile(errors[:2], .01)


def test_three_grade_assumptions_remain_bounded():
    rng = np.random.default_rng(3)
    ref = rng.normal(size=(50, 4))
    grades = assumption_grades(ref, np.array([[-100.] * 4, [100.] * 4]))
    assert grades.shape == (3, 2, 4)
    assert np.all((grades >= 0) & (grades <= 6))
    assert np.all(grades[:, 0] <= grades[:, 1])


def test_local_convex_samples_stay_inside_training_hull_box():
    real = np.array([[0., 1.], [1., 2.], [2., 3.]])
    generated = local_convex_samples(real, 50, np.random.default_rng(4))
    assert np.all(generated >= real.min(0)) and np.all(generated <= real.max(0))
    assert np.allclose(generated[:, 1] - generated[:, 0], 1.)
    assert not np.any(np.all(np.isclose(generated[:, None], real[None]), axis=2))


def need_stage2():
    if not (OUT / "status.json").exists():
        pytest.skip("Stage two has not been run")


def test_conformal_roles_are_disjoint():
    need_stage2()
    frame = pd.read_csv(OUT / "q1/conformal_split.csv")
    assert len(frame) == frame.beer.nunique() == 250
    assert frame.role.value_counts().to_dict() == {"train": 150, "calibration": 50, "test": 50}


def test_attribution_flags_do_not_claim_causality():
    need_stage2()
    top = pd.read_csv(OUT / "q2/top3_evidence_review.csv")
    assert len(top) == 150
    selected = top[top.evidence_status == "comparatively_supported_association"]
    assert (selected.bootstrap_top3_frequency >= .5).all()
    assert (selected.heldout_permutation_MAE_increase > 0).all()
    assert (selected.positive_fold_fraction >= 2 / 3).all()
    assert (top.causality == "not_established").all()


def test_repeated_generator_comparison_has_all_methods_and_splits():
    need_stage2()
    d = pd.read_csv(OUT / "q3/generator_comparison_by_split.csv")
    assert len(d) == 5 * 3 * 3
    assert set(d.method) == {"bootstrap", "local_convex", "PCA_Gaussian"}
    assert np.isfinite(d[["KS_mean", "MMD2", "target_style_probability"]]).all().all()


def test_extended_transfer_preserves_failure_and_each_defect():
    need_stage2()
    d = pd.read_csv(OUT / "q3/expanded_transfer_diagnostics.csv")
    good = d[d.status == "nontrivial_model_feasible_candidate"]
    assert (good.candidate_trip_probability > good.source_trip_probability + .01).all()
    assert (good.candidate_trip_probability > good.cleaning_only_probability + .01).all()
    bad = d[d.status != "nontrivial_model_feasible_candidate"]
```

```
    assert np.allclose(bad.change_l2, 0)
    assert pd.read_csv(OUT / "q3/expanded_transfer_defect_checks.csv").nonincrease.all()


def test_stage2_keeps_absolute_calibration_unidentified():
    need_stage2()
    assert json.loads((OUT / "status.json").read_text())["absolute_score_calibration"].startswith("not_identifiable")
```

源程序 29 tests/test_stage3_algorithms.py

```
"""Meaningful property tests for the new mathematical helpers."""
import numpy as np
import pytest
from src.stage3_algorithms import (fixed_grade_boundaries, grade_with_fixed_boundaries,
    ordinal_interval_image, shrunk_blend_weights, paired_oof_bootstrap, pareto_minimize, threshold_excess)


def test_fixed_ordinal_map_preserves_contained_outcome_with_tied_boundaries():
    boundaries = fixed_grade_boundaries(np.array([[0,0],[0,1],[0,2],[1,3],[1,4],[1,5],[2,6]], float))
    lower, upper = np.array([[-1,1.5],[0,2]]), np.array([[2,4],[0,2]])
    lo, hi = ordinal_interval_image(lower, upper, boundaries)
    for weight in np.linspace(0,1,21):
        inside = grade_with_fixed_boundaries(lower+weight*(upper-lower),boundaries)
        assert np.all((lo <= inside) & (inside <= hi))


def test_invalid_interval_is_rejected():
    with pytest.raises(ValueError):
        ordinal_interval_image(np.ones((1,2)), np.zeros((1,2)), np.zeros((6,2)))


def test_blend_recovers_opposite_best_models_and_shrinks_to_global():
    y = np.zeros((20,2)); et = np.tile([0,1.],(20,1)); gbr=np.tile([1.,0],(20,1))
    w,_ = shrunk_blend_weights(y,et,gbr,0)
    np.testing.assert_allclose(w,[0,1])
    w,global_w = shrunk_blend_weights(y,et,gbr,1e8)
    np.testing.assert_allclose(w,global_w,atol=1e-7)


def test_fixed_prediction_resampling_uses_paired_errors_and_declares_scope():
    y=np.zeros((12,2)); a=np.zeros_like(y); b=np.ones_like(y)
    r=paired_oof_bootstrap(y,a,b,np.repeat(['A','B'],6),100)
    assert r['difference_A_minus_B'] == -1
    assert r['conditional_percentile_95'] == [-1,-1]
    assert 'does_not_include_model_retraining' in r['scope']


def test_pareto_retains_tradeoffs_and_rejects_nonfinite_candidate():
    keep=pareto_minimize([[0,2],[1,1],[2,0],[2,2],[np.nan,0],[1,1]])
    np.testing.assert_array_equal(keep,[True,True,True,False,False,True])


def test_unknown_threshold_and_source_values_never_become_safe_zero():
    result=threshold_excess([.5,2,2,np.nan,-1],[1,1,np.nan,1,1])
    np.testing.assert_allclose(result[:2],[0,1])
    assert np.isnan(result[2:]).all()
```

源程序 30 tests/test_stage3_research.py

```
"""Check actual isolation and mathematical invariants of the executed study."""
from pathlib import Path
import json
import numpy as np
import pandas as pd
import pytest
from src.stage3_prediction import learn_blend_nested
from src.stage3_graph import fit_tuned_graph, within_style_indices
from src.stage3_generation import mmd_fixed_bandwidth, constrained_convex
from src.q3 import ConditionalPCAGaussian

ROOT=Path(__file__).resolve().parents[1]


def test_nested_blend_base_predictions_never_predict_their_own_training_rows(monkeypatch):
    import src.stage3_prediction as module
    calls=[]
    def fit(X,Y,cfg): return set(X[:,0].astype(int))
    def predict(seen,X):
        assert not seen.intersection(X[:,0].astype(int)); calls.append((seen,set(X[:,0].astype(int))))
```

```
        return np.zeros((len(X),2)),np.ones((len(X),2))
    monkeypatch.setattr(module,'fit_pair',fit); monkeypatch.setattr(module,'predict_pair',predict)
    X=np.c_[np.arange(12),np.arange(12)**2]; Y=np.c_[np.arange(12)/12,np.arange(12)/24]
    w,_,_,eta,_=learn_blend_nested(X,Y,{'seed':1,'blend_penalties':[0,.1]},np.array([0]),0)
    assert len(calls)==12 and np.all((w>=0)&(w<=1)) and eta in [0,.1]


def test_zero_penalty_graph_and_permuted_graph_reduce_to_same_ridge():
    rng=np.random.default_rng(5); X=rng.normal(size=(18,4)); Y=rng.normal(size=(18,2))
    cfg={'ridge_alphas':[10],'graph_lambdas':[0],'graph_thresholds':[.7],'graph_null_repeats':2}
    fits=[fit_tuned_graph(X,Y,cfg,k,np.array([0]),17) for k in ['none','real','null']]
    for f in fits:
        for b in f['beta']: np.testing.assert_allclose(b,fits[0]['beta'][0],atol=1e-12)


def test_style_permutation_preserves_each_style_and_singleton():
    labels=np.array(['A','A','B','B','B','C']); order=within_style_indices(labels,np.random.default_rng(9))
    np.testing.assert_array_equal(labels[order],labels); assert sorted(order)==list(range(6)) and order[-1]==5


def test_fixed_bandwidth_mmd_is_symmetric_and_zero_for_identical_samples():
    rng=np.random.default_rng(8); a=rng.normal(size=(12,4)); b=rng.normal(size=(7,4))
    assert abs(mmd_fixed_bandwidth(a,a,2))<1e-12
    assert np.isclose(mmd_fixed_bandwidth(a,b,2),mmd_fixed_bandwidth(b,a,2))


def test_constrained_convex_respects_fitted_bounds_and_budget():
    rng=np.random.default_rng(3); X=rng.normal(size=(12,4)); g=ConditionalPCAGaussian(3,2).fit(X,np.repeat('A',12))
    real=g.prep.named_steps['scale'].inverse_transform(g.prep.transform(X))
    a,info=constrained_convex(real,g,[0],20,rng)
    assert len(a)==20 and info['attempts']<=5000
    assert np.all(a>=g.low-1e-9) and np.all(a<=g.high+1e-9)


def test_saved_graph_comparison_contains_all_15_folds_and_ten_null_replicates():
    path=ROOT/'outputs/stage3/q2/graph_fold_metrics.csv'
    if not path.exists(): pytest.skip('Third-stage graph study not present')
    d=pd.read_csv(path); assert d.fold.nunique()==15
    shuffled=d[d.graph=='shuffled']; assert len(shuffled)==300
    assert (shuffled.groupby(['fold','scope']).size()==10).all()


def test_saved_search_evaluator_and_test_roles_are_disjoint():
    path=ROOT/'outputs/stage3/q3/disjoint_roles.csv'
    if not path.exists(): pytest.skip('Third-stage generation study not present')
    d=pd.read_csv(path)
    for _,a in d.groupby('repeat'):
        assert len(a)==a.beer.nunique()==250
        assert sorted(a.groupby('role').size())==[50,100,100]


def test_saved_failed_transfers_are_not_counted_as_improvements():
    path=ROOT/'outputs/stage3/q3/transfer_independent_evaluation.csv'
    if not path.exists(): pytest.skip('Third-stage transfer study not present')
    d=pd.read_csv(path); failed=d.status=='no_feasible_nontrivial_candidate_source_retained'
    assert not d.loc[failed,'evaluator_proxy_acceptance'].any()
    assert (d.loc[failed,'standardized_change']==0).all()


def test_missing_sources_are_never_reported_as_unconditional_feasible():
    path=ROOT/'outputs/stage3/q3/transfer_independent_evaluation.csv'
    if not path.exists(): pytest.skip('Third-stage transfer study not present')
    d=pd.read_csv(path)
    assert not ((~d.all_provided_source_defects_observed)&(d.status=='model_feasible_nontrivial')).any()
    assert not d.absolute_chemical_certification.any() and not d.real_tasting_validated.any()


def test_saved_ordinal_calibration_roles_are_disjoint():
    paths=list((ROOT/'outputs/stage3/q1').glob('conformal_roles_*.csv'))
    if len(paths)!=5: pytest.skip('Third-stage conformal experiments not complete')
    for path in paths:
        d=pd.read_csv(path); assert len(d)==d.beer.nunique()==250
        assert d.groupby('role').size().to_dict()=={'calibration':50,'test':50,'train':150}


def test_saved_prediction_study_is_complete_and_absolute_scores_remain_unidentified():
    path=ROOT/'outputs/stage3/q1/status.json'
    if not path.exists(): pytest.skip('Third-stage prediction study not complete')
    d=json.loads(path.read_text()); assert d['complete'] and d['outer_folds']==15
    assert not d['absolute_anchors_observed']
```

源程序 31 run.py

```
"""AI-assisted orchestration: OpenAI Codex / GPT-6, 2026-10-24."""
import argparse
import json
import os
from pathlib import Path

os.environ.setdefault("MPLBACKEND", "Agg")
os.environ.setdefault("OMP_NUM_THREADS", "2")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "2")

from src.common import environment, progress, write_json
from src.data import find_data, load_data


def main():
    parser = argparse.ArgumentParser(description="Reproducible A-track research baselines")
    parser.add_argument("stage", choices=["audit", "q1", "q2", "q3", "report", "all"])
    parser.add_argument("--data-dir")
    parser.add_argument("--out", default="outputs/stage1")
    parser.add_argument("--config", default="config.json")
    args = parser.parse_args()
    cfg = json.loads(Path(args.config).read_text(encoding="utf-8"))
    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)
    data = load_data(find_data(args.data_dir))
    if args.stage in ["all", "audit"]:
        from src.audit import run_audit
        run_audit(data, out, cfg)
    if args.stage in ["all", "q1"]:
        from src.q1 import run_q1
        run_q1(data, out, cfg)
    if args.stage in ["all", "q2"]:
        from src.q2 import run_q2
        run_q2(data, out, cfg)
    if args.stage in ["all", "q3"]:
        from src.q3 import run_q3
        run_q3(data, out, cfg)
    if args.stage in ["all", "report"]:
        from src.reporting import make_report
        make_report(data, out, cfg)
    manifest = environment()
    manifest["stage"] = args.stage
    manifest["config"] = cfg
    manifest["files"] = sorted(str(p.relative_to(out)).replace("\\", "/") for p in out.rglob("*") if p.is_file()
and "private" not in p.parts and p.suffix in {".csv", ".json", ".md", ".png"})
    write_json(out / "run_manifest.json", manifest)
    progress(f"Completed {args.stage}; outputs: {out.resolve()}")


if __name__ == "__main__":
    main()
```

源程序 32 run_stage2.py

```
"""AI-assisted research extension, OpenAI Codex / GPT-6, 2026-10-24."""
import argparse
import json
import os
from pathlib import Path

os.environ.setdefault("MPLBACKEND", "Agg")
os.environ.setdefault("OMP_NUM_THREADS", "2")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "2")

from src.common import environment, write_json
from src.data import find_data, load_data
from src.stage2 import run_extension

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--data-dir")
    parser.add_argument("--out", default="outputs/stage2")
    parser.add_argument("--base", default="outputs/stage1")
    args = parser.parse_args()
    cfg = json.loads(Path("config.json").read_text())
    ext = json.loads(Path("config_stage2.json").read_text())
    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)
```

```
    run_extension(load_data(find_data(args.data_dir)), Path(args.base), out, cfg, ext)
    write_json(out / "run_manifest.json", {**environment(), "base_config": cfg, "extension_config": ext})
```

源程序 33 run_stage3.py

```
"""Run saved, explicitly scoped third-stage experiments; AI-assisted, 2026-10-25."""
import os
os.environ.setdefault("OMP_NUM_THREADS", "2")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "2")
os.environ.setdefault("MKL_NUM_THREADS", "2")
os.environ.setdefault("MPLBACKEND", "Agg")
import argparse
import json
from pathlib import Path
from src.common import environment, write_json, sha256
from src.data import load_data, find_data

if __name__ == '__main__':
    parser=argparse.ArgumentParser()
    parser.add_argument('part',choices=['q1','q2','q3'])
    parser.add_argument('--data-dir')
    parser.add_argument('--out',default='outputs/stage3')
    args=parser.parse_args()
    cfg=json.loads(Path('config_stage3.json').read_text(encoding='utf-8'))
    data=load_data(find_data(args.data_dir)); out=Path(args.out); out.mkdir(parents=True,exist_ok=True)
    expected={r['file']:r['sha256'] for r in __import__('pandas').read_csv('outputs/stage2/audit/source_manifest.csv').to_dict('records')}
    for file in ['1.csv','2.csv','3.xlsx','4.csv','8.xlsx']:
        if sha256(data.root/file)!=expected[file]: raise ValueError('Original source version changed: '+file)
    if args.part=='q1':
        from src.stage3_prediction import run_prediction
        run_prediction(data,out,cfg)
    elif args.part=='q2':
        from src.stage3_graph import run_graph
        run_graph(data,out,cfg)
    else:
        from src.stage3_generation import run_generation
        run_generation(data,out,cfg)
    write_json(out/f'{args.part}_run_manifest.json',{**environment(),'config':cfg,'config_sha256':sha256('config_stage3.json'),'sources_verified':expected})
```

源程序 34 config.json

```
{
  "seed": 20261024,
  "outer_folds": 3,
  "outer_repeats": 2,
  "inner_folds": 3,
  "ridge_alphas": [1.0, 10.0, 100.0, 1000.0],
  "pls_components": [3, 8],
  "extra_trees": 200,
  "bootstrap_repeats": 100,
  "graph_threshold": 0.7,
  "graph_ridge_alpha": 20.0,
  "graph_ridge_lambda": 5.0,
  "generation_holdout_fraction": 0.2,
  "generation_components": 10,
  "generation_samples_per_style": 40,
  "generation_styles": ["Tripel", "Kriek", "Lambic"],
  "transfer_source": "Pils/Lager",
  "transfer_target": "Tripel",
  "missing_rates": [0.0, 0.05, 0.1, 0.2],
  "export_assumption_grades": true
}
```

源程序 35 config_stage2.json

```
{
  "seed": 20261024,
  "gbr_estimators": 100,
  "gbr_learning_rate": 0.04,
  "gbr_depth": 2,
  "permutation_repeats": 5,
  "conformal_alpha": 0.1,
  "generation_repeats": 5,
  "generation_n": 40,
  "generation_styles": ["Tripel", "Kriek", "Lambic"],
```

```
  "transfer_weights": 30
}
```

源程序 36 config_stage3.json

```
{
  "date": "2026-10-25",
  "seed": 20261025,
  "outer_folds": 3,
  "outer_repeats": 5,
  "inner_folds": 3,
  "primary_targets": "explicit_A_and_F_prefixed_30_not_official_scope_ruling",
  "blend_penalties": [0, 0.01, 0.1, 1, 10],
  "ridge_alphas": [1, 10, 100, 1000],
  "extra_trees": 200,
  "gbr_estimators": 100,
  "graph_thresholds": [0.6, 0.7, 0.8],
  "graph_lambdas": [0, 1, 5, 20],
  "graph_null_repeats": 10,
  "bootstrap_repeats": 200,
  "permutation_repeats": 20,
  "conformal_repeats": 5,
  "conformal_alpha": 0.1,
  "generation_repeats": 5,
  "generation_n": 40,
  "generation_styles": ["Tripel", "Kriek", "Lambic"],
  "generator_n": 100,
  "evaluator_n": 100,
  "real_holdout_n": 50,
  "transfer_weights": 30,
  "trust_radii": [0.25, 0.5, 1, 2],
  "transfer_target_probability_floor": 0.5,
  "transfer_min_probability_gain": 0.01,
  "claim_scope": "exploratory_internal_validation_after_stages_one_and_two_no_new_external_or_tasting_data",
  "execution_differences_from_design": [
    "Clipping and missingness ablations use fixed ExtraTrees, not all nested fusion variants; reported separately.",
    "Chemical literature evidence is graded and incomplete; no fabricated absolute-score anchors or physical
validation."
  ]
}
```

源程序 37 config_stage3_design.json

```
{
  "stage": "stage3_design",
  "date": "2026-10-25",
  "seed": 20261025,
  "status": "protocol_and_preflight_only_not_completed_stage3_experiments",
  "validation": {
    "outer_folds": 3,
    "outer_repeats": 5,
    "inner_folds": 3,
    "stratification": "beer_style",
    "unit": "unique_beer",
    "primary_metric": "macro_MAE_on_explicitly_mapped_Aroma_Flavour_30_targets",
    "secondary_scope": "all_50_targets_and_card_sections_reported_separately",
    "paired_bootstrap_repeats": 2000,
    "bootstrap_scope": "conditional_on_saved_OOF_predictions_not_full_algorithm_uncertainty",
    "practical_relative_MAE_margin": 0.01,
    "new_external_data": "not_available_existing_data_are_exploratory"
  },
  "grading": {
    "primary_without_anchors": "training_only_ECDF_relative_rank_bins_0_to_6",
    "conformal_alpha": 0.1,
    "absolute_grade_accuracy": "not_estimable_without_observed_absolute_anchors",
    "warning_if_grade_set_size_at_least": 4,
    "anchor_analysis": "conditional_on_new_observed_same_protocol_pairs_only"
  },
  "models": {
    "baselines": ["mean", "ridge", "extra_trees", "gbr_fixed"],
    "proposed": "inner_OOF_per_target_GBR_ET_blend_shrunk_to_global_weight",
    "blend_shrinkage_grid": [0, 0.01, 0.1, 1, 10],
    "clip_quantiles": [0, 0.01, 0.025],
    "missing_rates": [0, 0.05, 0.1, 0.2],
    "mask_scenarios": ["MCAR", "preferentially_mask_low_training_quantile_values"]
  },
  "graph": {
    "thresholds": [0.6, 0.7, 0.8],
    "ridge_alphas": [1, 10, 100, 1000],
```

```
    "graph_lambdas": [0, 1, 5, 20],
    "null_graph": "permute_node_labels_preserving_unlabelled_weighted_signed_graph",
    "bootstrap_repeats": 200,
    "permutation_repeats": 20,
    "inference": "descriptive_training_selection_with_heldout_effects_no_causal_or_p_value_claim"
  },
  "generation": {
    "repeat_seeds": [20261025, 20261026, 20261027, 20261028, 20261029],
    "generator_search_training_n": 100,
    "external_evaluator_training_n": 100,
    "real_holdout_n": 50,
    "samples_per_style": 40,
    "styles": ["Tripel", "Kriek", "Lambic"],
    "primary_fields": "all_nonaggregate_fields_226",
    "unknown_aggregate_policy": "exclude_from_primary_models_until_formula_verified",
    "methods": ["bootstrap", "local_convex", "conditional_PCA_Gaussian", "constrained_local_convex"],
    "MMD_bandwidth": "fit_on_generator_training_data_once_per_split_shared_by_all_methods"
  },
  "transfer": {
    "source": "Pils/Lager",
    "target": "Tripel",
    "chemical_fields": ["DMS", "X2.3.butanedione", "X4VG", "X.E..2.nonenal"],
    "trust_region_radii": [0.25, 0.5, 1, 2],
    "objective": "Pareto_minimize_standardized_change_target_distance_and_defect_excess",
    "threshold_sensitivity_multipliers": [0.5, 1, 2],
    "missing_defect_source": "unknown_not_certified_against_median",
    "stale_hops": "multiple_model_proxy_consensus_not_real_tasting_certification",
    "failure": "preserve_source_and_record_no_feasible_nontrivial_candidate"
  }
}
```